

A Compiler for Multiplatform Behaviour-Driven Development UI Testing

Master thesis in partial fulfilment of the requirements for
the degree of

Master of Science

Author: Denis Schüle, BSc

Submitted to the Master degree program MultiMediaTechnology, Salzburg University of Applied Sciences

Advisor: FH-Ass.Prof. Andreas Bilke, MSc
Second Advisor: FH-Prof. DI Brigitte Jellinek, MSc

Salzburg, Austria, 30.08.2026

Affidavit

I herewith declare on oath that I wrote the present thesis without the help of third persons and without using any other sources and means than those cited herein. I further declare that I observed the guidelines for scientific work in the quotation of all unprinted sources, printed literature and phrases and concepts taken either word for word or according to meaning from the Internet and that I referenced all sources accordingly.

This thesis has not been submitted as an exam paper of identical or similar form, either in Austria or abroad and corresponds to the paper graded by the assessors.

Date

Signature

First Name *Last Name*

Kurzfassung

Die Überführung plattformunabhängiger Verhaltensszenarien in ausführbare mobile UI-Tests erfordert anwendungs- und plattformspezifisches Automatisierungswissen. Diese Arbeit stellt MobileBDD, eine kontrollierte Behaviour-Driven Development (BDD)-inspirierte domänenspezifische Sprache, und einen Compiler vor, der ihre Szenarien in Maestro-YAML-Flows für Android und iOS übersetzt. Der Compiler validiert jedes Szenario anhand eines abstrakten Interaktionsmodells (Abstract Interaction Model, AIM). Dieses trennt die fachliche Testabsicht von anwendungs- und plattformspezifischen Selektoren und Ausführungsregeln. Untersucht wird, ob diese Trennung ausführbare plattformübergreifende UI-Tests ermöglicht.

Evaluiert wurde der Workflow mit der Anwendung SalzburgCard, einem eigens entwickelten Prototyp auf Basis von Kotlin Multiplatform und Compose Multiplatform mit einer gemeinsamen Codebasis für Android und iOS. Der Gesamtkorpus umfasste 64 Szenarien, von denen 32 den Evaluationskorpus bildeten. Diese Szenarien ergaben 16 Paare aus acht Interaktionskategorien. Die beiden Szenarien eines Paares verwendeten denselben Interaktionspfad und unterschieden sich nur in der erwarteten Nachbedingung. Für die Evaluationsszenarien wurden sowohl die vom Compiler erzeugten Flows als auch die separat implementierten manuellen Vergleichsflows unter einer festen Konfiguration jeweils zehnmal auf einem Android-Emulator und einem iOS-Simulator ausgeführt. Die Wiederholungen dienten dazu, mögliche Abweichungen zwischen den Ausführungen zu erfassen.

Alle Evaluationsszenarien kompilierten in sämtlichen Wiederholungen. Über alle Wiederholungen hinweg entsprachen die Ergebnisse der generierten Flows in 318/320 Android-Ausführungen (99,375 %) und 320/320 iOS-Ausführungen (100 %) dem jeweils erwarteten Ausgang (bestanden/fehlgeschlagen). Die Ergebnisse der generierten und manuellen Flows stimmten in 318/320 Android-Ausführungspaaren überein; ein Paar wich ab, ein weiteres war nicht vergleichbar. Unter iOS stimmten die Ergebnisse in allen 320 Ausführungspaaren überein.

Die hohe Übereinstimmung der Ausführungsstatus ist jedoch nicht mit vollständiger semantischer Äquivalenz gleichzusetzen: Bei 28/32 Evaluationsszenarien prüften die generierten und manuellen Flows dasselbe abschließende UI-Merkmal; bei zwei Varianten eines Rezensionsszenarios wurden zudem nicht alle in der Nachbedingung genannten Merkmale unabhängig überprüft. Die Ergebnisse belegen daher die Machbarkeit und Wiederholbarkeit innerhalb des evaluierten Korpus und der festen Konfiguration. Ob der Ansatz auf unbekannte Szenarien übertragbar ist und praktische Vorteile hinsichtlich Wiederverwendung, Lesbarkeit, Erstellung oder Wartung bietet, muss unabhängig untersucht werden.

Schlüsselwörter: BDD, mobile UI-Tests, Testgenerierung, domänenspezifische Sprache, plattformübergreifendes Testen

Abstract

Turning platform-independent behavioural scenarios into executable mobile UI tests requires application- and platform-specific automation knowledge. This thesis presents MobileBDD, a constrained domain-specific language inspired by Behaviour-Driven Development (BDD), and a compiler that translates its scenarios into Maestro YAML flows for Android and iOS. The compiler validates each scenario against an Abstract Interaction Model (AIM). This model separates test intent from application- and platform-specific selectors and execution rules. The thesis investigates whether this separation enables executable cross-platform UI tests.

The workflow was evaluated using SalzburgCard, a purpose-built Kotlin Multiplatform and Compose Multiplatform prototype with a shared codebase for Android and iOS. The full corpus comprised 64 scenarios, 32 of which formed the evaluation corpus. These scenarios formed 16 pairs across eight interaction categories. The two scenarios in each pair followed the same interaction path and differed only in their expected postcondition. For the evaluation scenarios, both compiler-generated flows and separately implemented manual comparator flows were executed ten times on one Android emulator and one iOS simulator under a fixed configuration. The repetitions served to capture possible variation between executions.

All evaluation scenarios compiled in each of the ten repetitions. Across all repetitions, the generated-flow outcomes matched the expected result (pass/fail) in 318/320 Android executions (99.375%) and 320/320 iOS executions (100%). The generated and manual flow outcomes agreed in 318/320 Android execution pairs; one execution pair differed, while another was not comparable. On iOS, the outcomes agreed in all 320 execution pairs.

However, the high agreement in execution statuses should not be equated with full semantic equivalence: in 28/32 evaluation scenarios, the generated and manual flows checked the same terminal UI observable; in two variants of a user-review scenario, not all properties stated in the postcondition were independently asserted. The results therefore demonstrate feasibility and repeatability within the evaluated corpus and fixed configuration. Whether the approach transfers to unseen scenarios and offers practical benefits in terms of reuse, readability, authoring, or maintenance requires independent investigation.

Keywords: BDD, mobile UI testing, test generation, domain-specific language, cross-platform testing

Table of Contents

1	Introduction	1
2	Related Work	3
2.1	Mobile Application Testing	3
2.1.1	Automated Test Generation for Mobile GUIs	3
2.1.2	Black-Box GUI Testing and Observable Behaviour	4
2.1.3	Cross-Platform Context of Mobile Applications	5
2.1.4	Cross-Platform Test Reuse	6
2.2	BDD-Inspired Behaviour Specifications for Test Generation	6
2.2.1	BDD as Example-Based Specification	7
2.2.2	From Scenarios to Executable Acceptance Tests	7
2.2.3	Limitations of Conventional BDD Automation	9
2.2.4	Connecting BDD Scenarios with UI Artefacts and Test Automation	10
2.3	Domain-Specific Language for Compiler-Based UI Test Generation	11
2.3.1	A Constrained Source Language	11
2.3.2	Testing DSLs as Abstractions over Automation Details	12
2.3.3	From Concrete Syntax to Validated Interaction Intent	13
2.3.4	Intermediate Representations and Model-Based Generation	13
2.4	Evaluation of Generated UI Tests	14
2.4.1	Source and Target Validity	14
2.4.2	Execution Readiness and Platform Execution	14
2.4.3	Expected Outcomes and Test Oracles	15
2.5	Positioning Against Closely Related Approaches	15
3	Methods	17
3.1	Evaluation Design	17
3.1.1	Evaluation Boundaries	17
3.1.2	System Under Test	18
3.2	Scenario Corpus	19
3.2.1	UI Interaction Taxonomy	19
3.2.2	Construction and Quality Review	21
3.2.3	Audit Trail	23
3.2.4	Development–Evaluation Assignment	23
3.3	Operationalisation and Measures	24
3.3.1	Evaluation Criteria	24
3.3.2	Observation Units	24
3.3.3	Measures and Reporting	25
3.3.4	Diagnostic Ordering and Decision Labels	25
3.4	Manual Comparator Construction for RQ3	25
3.5	Evaluation Procedure	26
3.5.1	Fixed Execution Environment	26
3.5.2	Repeated Pipeline Execution	27

3.5.3	Evidence Collection and Failure Classification	27
4	Implementation	28
4.1	Architecture	28
4.1.1	C4 Architecture	28
4.2	Core Compiler Artefacts	31
4.2.1	MobileBDD Grammar	31
4.2.2	Typed Kotlin AST	34
4.2.3	Abstract Interaction Model	35
4.3	Pipeline Stages	38
4.3.1	Pipeline Orchestration and Execution Configuration	39
4.3.2	S_0 Configuration Readiness	40
4.3.3	S_1 Analysis and Semantic Resolution	40
4.3.4	S_2 Maestro YAML Synthesis	41
4.3.5	S_3 Runtime Execution and Classification	42
4.3.6	S_4 Runtime-Outcome Comparison	43
4.3.7	Report Aggregation	44
4.3.8	Worked End-to-End Compilation and Execution Example	44
4.4	Implementation Verification	46
4.5	Implementation Decisions and Lessons	46
4.6	Platform-Specific Runtime Stabilisation	48
5	Results	50
5.1	Combined Evaluation Runs	50
5.2	Shared Compilation Success (RQ1)	52
5.3	Expected Terminal-Status Conformity (RQ2)	53
5.4	Differential Raw-Status Consistency (RQ3)	53
5.5	Results Summary and Decision Labels	55
6	Discussion	56
6.1	Interpretation of Findings	56
6.2	Relation to Prior Work	57
6.3	Threats to Validity	59
6.3.1	Construct Validity	59
6.3.2	Internal Validity	60
6.3.3	Conclusion Validity	61
6.3.4	External Validity	61
6.3.5	Reliability and Reproducibility	61
7	Summary & Outlook	62
7.1	Findings and Contributions	62
7.2	Limitations	63
7.3	Outlook	63

Appendices	71
A Use of Generative AI	71
A.1 Tools and Purposes	71
A.2 Verification and Limitations	71
A.3 Representative Prompts	71
B Git Repositories	72
C Fixed Execution Environment	73
D Supplementary Implementation Listings	75
D.1 Typed Kotlin AST	75
D.2 AIM Configuration	76
D.2.1 Semantic Matching and Configuration Precedence	76
D.3 Pipeline Types, Commands, and Evidence	77
D.3.1 Report Pairing and S ₄ Case Fields	81
E Corpus Provenance and Set-Membership Audit	82
E.1 Case-Level Set Contrasts	82
E.2 Selected Artefact Chronology	83
F Scenario Corpus Summary	85
F.1 Development Corpus ($\mathcal{D}$)	86
F.2 Evaluation Corpus ($\mathcal{E}$)	89
G S₄ Manual Reference-Flow Artefacts	92
G.1 Development-Corpus Reference Flows ($\mathcal{D}$)	92
G.2 Fixed Evaluation-Corpus Reference Flows ($\mathcal{E}$)	94
G.3 Representative Evaluation-Corpus Reference Flow	96

List of Figures

1	Representative Android and iOS screens of the system under test	20
2	BDD-to-Maestro system context diagram	28
3	BDD-to-Maestro container architecture diagram	29
4	CLI application components	30
5	Language-core components Diagram	30
6	BDD-to-Maestro compiler and evaluation pipeline	39
7	Android first-start dialog observed during pilot execution.	48
8	Password-manager prompt interrupting the application.	48
9	Multilingual keyboard tip observed in the simulator.	49
10	Continuous-path keyboard tip observed in the simulator.	49

List of Listings

1	Selected implemented MobileBDD lexer rules	31
2	MobileBDD document and feature-header structure	32
3	MobileBDD constrained Gherkin-style scenario structure	33
4	AIM declarations used by the worked example	36
5	MobileBDD source used for the end-to-end flow	44
6	Maestro flow generated in stage S_2 for the worked scenario	45
7	Seeding selected iOS Simulator keyboard preferences before text-input flows	49
8	Selected implemented Kotlin AST types	75
9	Representative AIM recipe, language-alias, and operation declarations	76
10	Result variants and pipeline outcomes of the S_1 gate	77
11	Generated platform-specific Maestro branch	78
12	Simplified S_2 validation evidence	78
13	Android and iOS execution-target discovery	79
14	Tag-filtered standalone generation	79
15	Standalone CLI invocation for S_3	79
16	Shortened S_3 report for an expected-fail flow	79
17	Shortened S_4 report for one eligible agreeing pair	81
18	Representative expected-pass S_4 evaluation-corpus reference flow	96
19	Representative expected-fail S_4 evaluation-corpus reference flow	96
20	Shared interaction path used by both representative evaluation flows	96
21	Reusable helper invoked by the representative evaluation action	97

List of Tables

1	Conceptual relationship between BDD, Gherkin, and Cucumber.	9
2	Comparison of closely related approaches	16
3	Case distribution by interaction category and capability	23

4	Pipeline prerequisite and evaluation criteria	24
5	Success rule and denominator for each research question	25
6	Implemented configuration groups of the Abstract Interaction Model	36
7	Maestro CLI commands integrated by the pipeline runner.	40
8	Implemented raw status and failure classification of an S_3 report.	42
9	Resolution of the worked scenario into Maestro effects	45
10	Measurement-boundary cases covered by source tests	46
11	Completeness of the retained campaign evidence	51
12	Ten retained combined cross-platform evaluation runs	51
13	Disposition of the repeated S_0 Abstract Interaction Model (AIM) findings	52
14	RQ2 results by platform	53
15	Aggregate differential raw-status consistency	54
16	Answers to RQ1–RQ3	55
17	Git repositories used in this thesis.	72
18	Fixed execution environment for status evaluation	73
19	Representative syntax and semantic diagnostics produced by S_1	78
20	Principal fields of a completed S_3 report.	80
21	Implemented S_4 case classification.	81
22	Evaluation-corpus cases and their development-corpus contrasts	82
23	Selected Git chronology of corpus, comparator, AIM, pipeline, and result artefacts	84
24	Development corpus derived from source files	86
25	Evaluation corpus derived from source files	89
26	Manual reference-flow artefacts for the development corpus $\mathcal{D}$	93
27	Manual reference-flow artefacts for the evaluation corpus $\mathcal{E}$	95

Abbreviations

ADB	Android Debug Bridge
AIM	Abstract Interaction Model
ANTLR	ANother Tool for Language Recognition
AST	Abstract Syntax Tree
BDD	Behaviour-Driven Development
BDDTS	BDD Transition System
CMP	Compose Multiplatform
DSL	Domain-Specific Language
DSTL	Domain-Specific Test Language
IBDD	Intermediate Behaviour-Driven Development
KMM	Kotlin Multiplatform Mobile

KMP Kotlin Multiplatform
MAT Mobile Application Testing
MBT Model-Based Testing
NL Natural Language
R&R record-and-replay
SUT System Under Test
STS Symbolic Transition System
TDD Test-Driven Development
UBL User Behavior Language
UX User Experience
ViMoTest ViewModel Test

1 Introduction

Internet use and mobile access are widespread in the EU. In 2025, approximately nine in ten individuals used the internet daily, and a similar share used a mobile phone or smartphone to access it (Eurostat, 2025a, 2025b). Smartphones provide several ways to access internet content, including dedicated applications tailored to particular services and mobile-device use (Fichte & York, 2025, p. 151). Nitze and Schmietendorf (2015) surveyed mobile application users and identified usability, performance, and reliability as important quality criteria. As mobile apps continue to grow in complexity (Inukollu et al., 2014, p. 32), their feature-rich GUIs become more challenging to explore and understand, while new mobile development concepts and interaction gestures further complicate GUI testing (Nie et al., 2023, p. 262).

GUI testing examines the application at the point where users interact with it. Muccini et al. (2012, p. 34) identify two mobile GUI-testing challenges: checking whether data are rendered adequately and whether native applications are displayed correctly across different devices. They propose automatically replaying captured interaction scripts on different devices. However, high code coverage of applications only indicates that code was executed during testing and does not guarantee the absence of bugs (Eke et al., 2025, p. 320). Establishing correctness requires an oracle that compares observed with expected behaviour (Baresi & Young, 2001, p. 48). Consequently, beyond executing many scripted interactions, a key question is how to specify intended behaviour so that tests capture the right requirements and remain communicable and maintainable as systems evolve.

BDD addresses this aspect of test specification by shifting the focus from tests towards describing system behaviour (Terhorst-North, 2006). BDD evolved from Test-Driven Development (TDD), which focuses on a test-first software development approach (Beck, 2002, p. 18). BDD promotes a shared, business-readable vocabulary for expressing requirements and acceptance criteria (Binamungu et al., 2018, p. 175). These acceptance criteria can be formulated as structured scenarios and mapped to executable specifications (Binamungu et al., 2018, p. 175). Prior research has demonstrated the usefulness of BDD to verify the compliance of software products with their requirements (Binamungu et al., 2018, p. 183).

While BDD provides a platform-independent way to express expected system behaviour, executing these behavioural specifications as tests introduces platform-specific challenges. Reusing UI tests across platforms remains constrained by differences in how platforms represent interfaces (Menegassi & Endo, 2020, pp. 27–28). Prior work shows that high-level behavioural models can support reuse across Android and iOS (Karlsson et al., 2021, p. 135), while generating executable Android UI tests from abstract interaction models still requires mappings to concrete UI elements (Turner & Bowen, 2024, p. 7). Together, these findings motivate separating shared scenario intent from platform-specific bindings. None of the reviewed approaches covers the full workflow investigated in this thesis, from mobile-specific test specification and validation to cross-platform test generation, execution, and evaluation (Table 2). Rather than indicating a need for an entirely new approach, the reviewed work provides related concepts and foundations that are combined and adapted in this thesis using a current mobile testing framework.

The objective of this thesis is to design and evaluate a workflow that compiles scenarios written in MobileBDD, a constrained, BDD-inspired Domain-Specific Language (DSL), into executable Maestro YAML flows. The compiler validates and resolves the scenarios using application-specific interaction mappings. The generated flows are executed on Android and iOS against the System Under Test (SUT) described in Section 3.1.2. The thesis contributes the implemented transformation workflow and its staged evaluation.

The evaluation uses the corpus of MobileBDD scenarios $\mathcal{E}$, defined in Section 3.2.4, to assess compilation and execution on both platforms. For each generated flow, the observed execution status is compared with the expected status declared in the scenario and with the status of the corresponding manual flow. These aspects are addressed by the following research questions:

- **RQ1:** What compilation success rate does the proposed compiler achieve when transforming the MobileBDD scenarios in $\mathcal{E}$ into syntactically valid Maestro flows?
- **RQ2:** What expected terminal-status conformity rate do the generated Maestro flows for $\mathcal{E}$ achieve on Android and iOS?
- **RQ3:** What differential raw-status consistency is observed between the generated flows for $\mathcal{E}$ and corresponding manual comparator flows on Android and iOS?

The remainder of this thesis moves from the conceptual foundations of the approach to its implementation and empirical evaluation. Relevant work on mobile UI testing, behaviour-driven specification, DSLs, test generation, and compiler validation is reviewed in Chapter 2, positioning the proposed workflow within existing research. Building on this foundation, Chapter 3 defines the evaluation design, introduces SalzburgCard as the SUT, presents the scenario corpus, and explains how the three research questions are operationalised. The architecture and implementation of the MobileBDD compiler and its execution pipeline follow in Chapter 4. Results for compilation, platform execution, and comparison with the manual flows are presented in Chapter 5 and interpreted in relation to prior work and the limitations of the study in Chapter 6. Finally, Chapter 7 summarises the contribution and identifies directions for further evaluation and development.

2 Related Work

This chapter reviews the concepts and approaches relevant to transforming behavioural specifications into executable mobile UI tests. It first considers mobile GUI testing, black-box validation, and cross-platform test reuse before examining BDD-inspired behavioural specifications and their connection to executable tests. It then reviews DSLs, compiler-based transformation, and intermediate representations used to bridge behavioural intent and concrete automation artefacts. Finally, prior work on the evaluation of generated tests is considered, and the relation to this work.

2.1 Mobile Application Testing

This section reviews the literature relevant to the proposed cross-platform mobile GUI-testing pipeline. It examines automated test generation and black-box validation before considering the cross-platform application context and the reuse of shared behavioural specifications across Android and iOS test artefacts.

2.1.1 Automated Test Generation for Mobile GUIs

In mobile applications, the GUI is the primary point of user interaction, and incorrect application behaviour can therefore become observable at this level (Lin et al., 2021, p. 1079). Testing such behaviour can be challenging because GUI actions and reactions are closely connected to application logic and may additionally be affected by runtime interruptions such as system events (Kong et al., 2019, p. 52). These practical challenges are reflected in developers' choice of testing approach. Lin et al. (2021, p. 1085) report that many developers prefer unit tests over UI tests because of their speed, scope, and simplicity. Developers who prefer UI tests describe them as an interactive form of end-to-end testing. This highlights a trade-off: UI tests can capture observable end-to-end interaction behaviour, but their practical use is affected by the effort required to create and maintain them. Furthermore, performing mobile UI testing manually is costly and difficult to scale, motivating automated execution for repeatable runs across mobile configurations (Nie et al., 2023, pp. 249–250). Beyond automated execution, reducing the manual effort involved in creating unit tests remains necessary (Lin et al., 2021, p. 1088).

Automated execution should be distinguished from automated test generation. The former runs already defined test artefacts, whereas the latter derives such artefacts from models, specifications, or other higher-level representations. Nie et al. (2023, p. 262) survey automated techniques for mobile test-case generation. They identify test-case generation, test minimisation, and understanding application flows as relevant research directions (Nie et al., 2023, p. 262). Within Mobile Application Testing (MAT), model-based test generation remains a recurring research direction, particularly where higher-level representations are used to derive executable tests (Eke et al., 2025, p. 320).

Generation approaches can use models or specifications of the SUT to automate the creation of test scripts, reducing manual rewrites when specifications or the UI changes (Turner & Bowen,

2024, pp. 2–3). For Android, Turner and Bowen (2024, p. 7) generate Espresso¹ test code by mapping widgets represented in abstract interaction-sequence models to concrete Android UI widgets.

2.1.2 Black-Box GUI Testing and Observable Behaviour

Eke et al. (2025, p. 317) distinguish black-box testing from white-box and grey-box testing according to whether source code is available when creating test cases (Eke et al., 2025, p. 312). Drawing on the broader landscape of automated mobile GUI-testing approaches surveyed by Nie et al. (2023, pp. 261–262), this subsection reviews three black-box testing strands relevant to the proposed pipeline: record-and-replay, runtime exploration, and scenario-guided testing.

Record-and-replay (R&R) testing captures already observed UI actions as traces and replays them to execute scenarios or reproduce bugs (Song et al., 2025, p. 32). This makes R&R a direct form of black-box GUI validation. The limitation is that the recorded trace is only useful if the concrete interaction path remains reproducible across later executions, devices, and interface changes. Replayability problems were most frequent for crashing bugs (44%), followed by non-crashing failures (38%) and ordinary user scenarios (17%). A related maintenance problem appears in Espresso capture/replay tests (Song et al., 2025, p. 32). Turner and Bowen (2024, p. 1) note that recorded scripts are tightly coupled to the concrete UI, so interface changes can make them fragile. Their response is a model-based generator that uses explicit UI mappings to produce Espresso tests (Turner & Bowen, 2024, pp. 5, 7).

Exploratory GUI testing generates event sequences dynamically at runtime, without previous recording. Although effective for broad exploration and crash detection, this approach can be costly when applications must be retested after each revision. To address this limitation, Lv et al. (2022, p. 1) introduce Fastbot2. The approach reduces repeated exploration by retaining what earlier tests learned about which events lead to particular activities. This knowledge is represented by a probabilistic model, while reinforcement learning guides multi-step event selection (Lv et al., 2022, p. 2).

ARES is an automated Android GUI testing approach that uses deep reinforcement learning to guide runtime exploration. It represents the observed state using the current activity and its available widgets, assigning a large positive reward when an action reaches an activity not yet visited during the current episode or triggers a crash (Romdhana et al., 2022, pp. 5–7). Because comparing and tuning deep reinforcement learning algorithms through repeated interaction with real applications is expensive, the authors also introduce FATE, a model-based environment that simulates application-navigation constraints using finite-state models. FATE supports algorithm comparison and hyperparameter tuning before the selected configurations are transferred to ARES for testing real applications (Romdhana et al., 2022, pp. 8–10).

Fastbot2 and ARES guide runtime exploration through reused transition knowledge or learned event-selection policies. Both remain discovery-oriented, and can not ensure specific scenarios.

¹Espresso is an Android framework for writing UI tests; see the official Android documentation <https://developer.android.com/training/testing/espresso> (Accessed 15.08.2026).

Yu et al. (2024, pp. 1949, 1951) show that coverage-driven automated exploration can miss business-relevant behaviour because it does not explicitly target scenario completion. Their login example shows that broad exploration can still fail at behaviourally simple but semantically structured tasks, such as completing a login flow (Yu et al., 2024, p. 1951). To address this limitation, Yu et al. propose SCENTEST, an automated mobile-app testing approach that reconstructs user-level scenarios from human testing knowledge, including test reports and screenshots. SCENTEST encodes this information as Event Knowledge Graphs (EKGs) and uses them to select actions needed to complete a target scenario rather than maximize general application coverage (Yu et al., 2024, pp. 1956, 1963). The success criterion therefore shifts from explored states or detected crashes to completion of a specified user-level scenario.

2.1.3 Cross-Platform Context of Mobile Applications

Mobile application development primarily targets Android and iOS, which together account for virtually all mobile operating-system usage worldwide (Statista, 2023, p. 14). Because native applications are tied to a platform’s SDK, programming language, frameworks, and UI elements, supporting both platforms requires separate implementations (Jošt & Taneski, 2025, p. 4; Heitkötter et al., 2013, p. 122). This separation can also require distinct platform-specific skill sets, as Martinez and Lecomte (2017, p. 185) illustrate through Facebook’s return to native development. Cross-platform frameworks address this duplication by enabling shared source material to target multiple platforms (Frattaroli et al., 2023, p. 4). This motivation is reflected in cross-platform development frameworks, which aim to improve efficiency, reduce development costs, and support deployment across Android and iOS (Jošt & Taneski, 2025, p. 3).

Cross-platform development can take different technical forms: shared artefacts may run through a common runtime, be wrapped into platform-specific containers, or be transformed into platform-specific applications. These approaches carry different benefits, constraints, and implementation assumptions (Biørn-Hansen et al., 2019, p. 10). Heitkötter et al. (2013, pp. 122–123) distinguish native applications, which are tied to one platform and require separate implementations for each target system, from cross-platform applications, which use a shared code base to target multiple platforms. They further classify cross-platform approaches according to how shared code becomes executable: runtime-based approaches include mobile Web apps, hybrid apps, and self-contained runtimes, whereas generator-based approaches transform common source material into platform-specific applications. The suitability of these approaches depends on the specific development context and requirements (Jangassiyev et al., 2024, p. 109). This distinction matters because it determines where platform-specific behaviour can arise despite a shared code base (Jošt & Taneski, 2025, p. 5).

Framework choice affects architecture and end-product characteristics (Majchrzak et al., 2017, p. 6169). Abstractions may be realised through platform-specific UI components, runtimes, or mappings (Martinez & Lecomte, 2017, p. 185; Biørn-Hansen et al., 2019, p. 8). Kotlin Multiplatform (KMP)² is one of the current approaches to share application logic while permitting platform-specific implementations, and Compose Multiplatform (CMP)³ to share UI definitions

²Official Kotlin Multiplatform website: <https://kotlinlang.org/multiplatform> (Accessed 15.08.2026).

³Official Compose Multiplatform website: <https://kotlinlang.org/compose-multiplatform> (Accessed 15.08.2026).

across concrete platform environments (Stanić & Ćirković, 2024, pp. 54–55).

2.1.4 Cross-Platform Test Reuse

Even when equivalent functionality exists on Android and iOS, differences in platform-specific UI representations can require separate automation artefacts (Menegassi & Endo, 2020, pp. 27–28). Approaches to cross-platform test reuse can be distinguished by where reuse is introduced in the testing process. One direction starts from an existing platform-specific test and migrates it to another platform. Another shares behavioural descriptions or interaction models before platform-specific executable tests are created.

Cross-platform GUI test migration is a downstream form of test reuse: it transfers tests between platform implementations of the same mobile application after a platform-specific test already exists (Ji et al., 2023, p. 1417). MAPIT by Talebipour et al. (2021, p. 756) follows this strategy. It takes the binaries of sibling Android and iOS applications together with a pre-existing test for one platform and generates an equivalent test for the other platform. During execution of the source-platform test, MAPIT dynamically extracts a GUI model containing screen bitmaps, widget information, and events that trigger screen transitions (Talebipour et al., 2021, p. 756). The migration relies on similarities between sibling applications, including comparable UI elements, text, and layout positions (Talebipour et al., 2021, p. 757).

Appium⁴ provides a common WebDriver-based interface for mobile automation. Script reuse still depends on how actions, selectors, assertions, and platform-specific capabilities are represented. In particular, differences between Android and iOS UI representations affect element location and test execution across configurations (Menegassi & Endo, 2020, pp. 28, 34).

Model-Based Testing (MBT) places the shared artefact at the level of interaction states and transitions. In the industrial case by Karlsson et al. (2021, pp. 131, 135), a high-level MBT model was reused for both Android and iOS. The model represents the perceived state of the application from the user’s perspective, while actions connect these states. This moves test design away from platform-specific scripts towards reusable behavioural models.

Reuse at the behavioural-model level does not eliminate platform-specific execution constraints, as differences in UI structures and runtime behaviour can still affect element location and test execution across Android and iOS (Menegassi & Endo, 2020, pp. 28, 34).

2.2 BDD-Inspired Behaviour Specifications for Test Generation

For test generation, this thesis focuses on BDD’s readable context–action–outcome notation. Prior work connects this notation with example-based specification, runtime step binding, UI models, and design artefacts.

⁴Official Appium documentation: <https://appium.io/docs/en/latest/intro/appium> (Accessed 15.08.2026).

2.2.1 BDD as Example-Based Specification

BDD emerged from agile and test-first software development, particularly from attempts to make TDD easier to understand and apply. Beck (2002, p. 18) describes TDD as a test-first feedback loop in which a failing automated test guides implementation and refactoring preserves externally observable behaviour. Terhorst-North (2006) later introduced BDD as a response to practical difficulties in teaching and applying TDD. Shifting the emphasis from tests to expected behaviour.

Practitioner accounts present BDD as a practice centred on concrete examples, shared language, and collaboration across roles. For example, Smart (2015, pp. 12, 268) describes BDD as a set of practices for delivering valuable software through a shared, domain-oriented language, whereas Lawrence and Rayner (2019, pp. 2, 47) distinguish collaborative discovery from the subsequent formalisation and automation of behavioural examples. The Cucumber project's historical account identifies Terhorst-North as the pioneer of BDD, notes the early contributions of Keogh and Matts, and connects this period with the development of the Given–When–Then template.⁵ Peer-reviewed research places these practices within a broader development lifecycle: Solis and Wang (2011, p. 384) characterise BDD as encompassing requirements elicitation, analysis, design, and implementation.

Terhorst-North (2006) frames a story's behaviour through acceptance criteria expressed as scenarios with separate context, event, and outcome fragments. Solis and Wang (2011, p. 385) describe a related decomposition in which business outcomes are refined into feature sets, features, user stories, and scenarios as concrete instances of a user story in a particular context. In this sense, BDD scenarios can function as both behavioural requirements and acceptance-test artefacts when connected to executable test code (Binamungu et al., 2018, p. 175).

Concrete examples instantiate expected behaviour as observable cases instead of leaving it at the level of abstract requirements. Each fixes a context, an action or event, and an expected outcome. Once the scenario language is sufficiently constrained, these cases can be transformed into executable actions and checked against observed results. In BDD, this structure organises information around context, action, and expected result (Smart, 2015, pp. 121–122; Binamungu et al., 2018, p. 175).

2.2.2 From Scenarios to Executable Acceptance Tests

BDD scenarios operationalise behavioural requirements in a form that is both readable and structured. At the requirement level, this relates to user stories, which Cohn (2004, pp. 4, 13–15) describes as short descriptions of user- or customer-oriented functionality that can be supported by acceptance tests. User stories commonly express a requirement in terms of the user role, the desired capability, and the intended benefit (Cohn, 2004, pp. 80–81). BDD refines this user-centred framing into concrete acceptance scenarios. In the original formulation of BDD, a story's acceptance criteria are expressed as Given–When–Then scenarios (Terhorst-North, 2006):

Scenario: <scenario title>

⁵See the Cucumber project's "History of BDD" <https://cucumber.io/docs/bdd/history> (Accessed 15.08.2026).

```
Given <initial context or precondition>
And <additional context>
When <event or action>
Then <expected outcome>
And <additional expected outcome>
```

In a Gherkin scenario, *Given* defines the starting situation, *When* identifies the event or behaviour being performed, and *Then* specifies what should be observable afterward (Binamungu et al., 2018, p. 175). Additional *And* and *But* steps continue the preceding *Given*, *When*, or *Then* sequence, with *But* typically emphasising a contrasting condition, action, or outcome. Practitioner literature presents this as the canonical BDD scenario structure (Smart, 2015, pp. 121–122), while peer-reviewed research also connects these elements to executable or model-based representations (Zameni et al., 2024, p. 204).

Executable specifications require a link between scenario text and executable behaviour. This link can be provided through bindings, generated artefacts, or other automation mechanisms. Smart (2015, p. 115) describes executable specifications as stakeholder-readable, semi-structured scenarios whose format can be automated using BDD tools. Through glue code, such examples become executable and can serve as both requirements specifications and acceptance tests (Binamungu et al., 2018, p. 175). However, Natural Language (NL) steps do not execute by themselves. In conventional runtime-binding approaches, step definitions provide the required mapping by connecting NL scenario sentences to executable code (Soeken et al., 2012, p. 274). Within this mapping, nouns can identify system objects, while verbs can indicate actions (Soeken et al., 2012, p. 275).

More broadly, peer-reviewed research places BDD automation within automated acceptance testing. Solis and Wang (2011, pp. 385–386) describe BDD tool support as mapping scenarios to executable tests, while Irshad et al. (2022, p. 2) treat BDD specifications as artefacts intended to run automatically when the codebase changes. Model-based approaches instead formalise scenario clauses as preconditions, transitions, and postconditions in intermediate representations (Zameni et al., 2024, p. 204). Generation-oriented work further shows that producing executable acceptance artefacts from higher-level behavioural specifications can involve explicit mappings, intermediate representations, or tool-specific transformations (Kamalakar et al., 2013, p. 241; Nguyen et al., 2025, pp. 1327, 1331). The source scenarios thereby retain a recurring human-readable structure while becoming suitable for parsing, semantic validation, and transformation through a dedicated DSL.

Common BDD frameworks such as Cucumber⁶ and JBehave⁷ implement scenario execution through runtime binding. Lawrence and Rayner (2019, pp. 76–77) describe Cucumber projects as separating Gherkin⁸ feature files from step-definition files, which contain the code executed when matching scenario steps are encountered. Smart (2015, p. 143) describes step definitions as

⁶Official Cucumber website: <https://cucumber.io> (Accessed 15.08.2026).

⁷Official JBehave website: <https://jbehave.org> (Accessed 15.08.2026).

⁸Official Gherkin reference: <https://cucumber.io/docs/gherkin/reference> (Accessed 15.08.2026).

the bridge between textual scenarios and concrete execution. In this model, a scenario remains a textual artefact whose steps are matched at runtime to manually implemented glue code; the executable behaviour is supplied by the step definitions and the automation libraries they call.

For clarity, Table 1 distinguishes BDD as a development and specification practice, Gherkin as a scenario notation, and Cucumber as an automation framework. These concepts are related but operate at different levels and should therefore not be treated interchangeably.

Table 1: Conceptual relationship between BDD, Gherkin, and Cucumber.

Concept	General role	Key characteristic
BDD	Development and specification practice based on concrete examples of expected behaviour.	Uses concrete behavioural examples and shared, domain-oriented language to support specification and communication.
Gherkin	Structured notation for expressing behavioural scenarios.	Organises scenarios using keywords such as Given, When, Then, And, and But, while permitting open-ended NL step text.
Cucumber	Automation framework for executable behavioural specifications.	Connects Gherkin scenario steps to executable code through runtime matching against step definitions.

2.2.3 Limitations of Conventional BDD Automation

The following discussion focuses on three limitations of conventional BDD automation that are particularly relevant for this thesis: open-ended scenario text, manually implemented runtime bindings, and platform-specific UI details.

First, although recurring Gherkin keywords provide a common scenario structure, individual step texts remain open-ended and phrasing-dependent. They may therefore be ambiguous or omit information required for automated testing (Zameni et al., 2024, pp. 199–200). For UI test generation, scenarios must identify supported actions, interaction targets, relevant navigation state, and observable outcomes with sufficient precision. More constrained interaction languages can improve the consistency and automated processability of such specifications (T. R. Silva, 2022, p. 52).

Second, runtime-binding approaches rely on manually implemented glue code. Step definitions connect textual steps with executable behaviour, but the underlying interactions, selectors, navigation operations, and assertions still have to be implemented and maintained. For GUI testing specifically, Bündler and Kuchen (2019a, p. 1742) identify insufficient integration with GUI descriptions and the manual implementation of executable test methods as shortcomings of conventional BDD tools. Maintenance difficulties can additionally arise at both the specification and UI-automation levels: similar or duplicated specifications reduce maintainability and motivate

refactoring (Irshad et al., 2022, p. 4), while concrete UI test scripts can become fragile when interface structures change (Turner & Bowen, 2024, p. 1).

Third, mobile and cross-platform UI testing compounds this binding problem because equivalent scenarios may require different selectors, UI representations, and automation operations on Android and iOS (Menegassi & Endo, 2020, pp. 27–28, 34). A shared behavioural specification alone therefore does not eliminate platform-specific test details; it still requires a mechanism for resolving abstract interactions into platform-specific executable artefacts.

2.2.4 Connecting BDD Scenarios with UI Artefacts and Test Automation

Prior work on BDD-based UI testing addresses a central transformation problem: how to turn stakeholder-readable behavioural scenarios into precise and executable UI interactions. The proposed approaches differ mainly in where they supply the information that open-ended scenario text does not encode itself. Runtime-binding approaches place it in step definitions, whereas model-driven approaches obtain it from external UI artefacts or intermediate representations. Constraining the scenario vocabulary is not a separate transformation mechanism but a refinement that can make either approach more amenable to automated processing. Reviewing these alternatives clarifies which parts of the transformation can be derived from the behavioural specification and which still depend on manually maintained mappings or supplementary models.

Silva and collaborators examine how scenario-based BDD artefacts can connect requirements with interactive system artefacts. In their earlier scenario-based work, T. R. Silva and Winckler (2017, p. 3) propose user stories and acceptance scenarios as a common representation for connecting requirements with UI design artefacts. Their approach supports consistency checking among requirements and design artefacts; runtime test generation against a completed SUT lies outside its scope (T. R. Silva & Winckler, 2017, p. 3). The scenarios therefore act as a bridge between requirements and model-based design artefacts, while a shared vocabulary supports automated checks across these artefact types (T. R. Silva & Winckler, 2017, pp. 5–6).

A related contribution by T. Silva et al. (2019, p. 485) adapts BDD to automatically assess intermediate UI design artefacts, such as task models and UI prototypes, before the interactive system is completed. The proposed approach uses user-story scenarios as the central artefact for checking consistency across task models, prototypes, and final UI artefacts (T. Silva et al., 2019, p. 486). In subsequent work, T. R. Silva and Fitzgerald (2021, p. 266) distinguish requirements traceability from consistency assessment: traceability establishes whether an artefact contains a given requirement, but not whether that requirement is represented correctly and consistently. Based on their empirical findings, they argue for more structured BDD stories, suggesting that predefined structures and controlled vocabulary can improve the identification of elements used for automated artefact assessment (T. R. Silva & Fitzgerald, 2021, p. 270).

Bünder and Kuchen (2019a, p. 1742) present a method that derives runnable GUI tests from interface wireframes and feature specifications written in a BDD style. In Bünder and Kuchen (2019a, p. 1749)’s the approach produced entire test methods from scenario descriptions supplemented with GUI wireframes.

Ali (2019, p. 7) frames the Catroid/Pocket Code case study as a basis for cross-platform mobile UI test automation using Cucumber-based BDD specifications. The scenarios are written in NL and represent how end users use the final product (Ali, 2019, p. 5). However, executable scenarios still require step definitions that convert NL constructs into working code (Ali, 2019, p. 87). As the case study covers only Android, its applicability to iOS remains unverified and would require equivalent Cucumber support (Ali, 2019, p. 98).

Taken together, these studies show that readable behavioural scenarios alone do not provide all information required for automated UI test generation. Runtime step definitions preserve flexible NL but require manually maintained executable mappings, whereas model-driven approaches can generate more of the test by introducing supplementary UI artefacts. A structured vocabulary can refine either mechanism by moving more interaction information into the source specification and making it available for automated analysis. This trade-off motivates the use of a DSL in the following subsection because a constrained language can preserve the context–action–outcome structure of BDD while defining the syntax and semantics needed to validate interaction intent and compile it into platform-specific test artefacts.

2.3 Domain-Specific Language for Compiler-Based UI Test Generation

In compiler-based UI test generation, a DSL serves as the structured source from which executable test artefacts are derived. Its use begins with constraints on test descriptions and extends to the abstraction of automation details, syntactic and semantic validation, and transformation through intermediate representations.

2.3.1 A Constrained Source Language

Compiler-based test generation requires a source representation that can be parsed, semantically checked, transformed, and compiled into executable artefacts. A DSL can provide such a constrained source language by defining the syntax and semantics of valid input. This interpretation is consistent with software-language engineering work that treats syntax and semantics as separate language concerns (Hölldobler et al., 2018, p. 387), as well as with model-driven engineering literature that considers the definition of both syntax and semantics necessary for a well-defined modelling language (Brambilla et al., 2017, p. 63). Hudak (1998, pp. 8, 10) further emphasises that DSL design should begin from the domain and its semantics rather than from surface syntax alone.

Compared with conventional Gherkin-style scenarios, a compiler-oriented source language may impose stronger constraints on step vocabulary, argument structure, and supported interaction semantics. Such constraints introduce a trade-off between linguistic flexibility and automated processability. Gherkin-style scenarios combine a fixed keyword frame with step text written in ordinary domain language, which keeps scenarios readable for stakeholders while leaving substantial freedom in phrasing (Smart, 2015, pp. 121–122). This openness can also permit ambiguity, inconsistent wording, and implicit assumptions. Stricter specifications reduce this freedom while supporting automated checks against the language and related artefacts (T. R. Silva, 2022, p. 52).

For UI-test generation, interaction descriptions additionally need to represent actions and their targets in a processable form. Related work addresses this through compact action–target structures or command vocabularies that preserve interaction intent while supporting automated generation (Le-Khanh et al., 2025, pp. 99–100).

2.3.2 Testing DSLs as Abstractions over Automation Details

The testing literature supports the use of DSLs to separate domain-level test intent from low-level automation details. Low-level test procedures can obscure the intended system behaviour by foregrounding implementation and automation details over domain-level actions (Bussenot et al., 2018, p. 433). Domain-Specific Test Languages (DSTLs) address this problem by adapting test languages to domain-specific tester practices and constraints (Bussenot et al., 2018, p. 435). They can also replace NL test procedures with more structured artefacts while preserving domain vocabulary and test intent (Bussenot et al., 2018, p. 436). Similar abstraction mechanisms appear in Teasy, a DSTL for automated functional web-application testing that organises tests around configuration, page components, page actions, and flow definitions, from which executable test scripts can be generated (Lima et al., 2021, pp. 43–44). It hides repetitive test-code details so that testers only provide information relevant to the test flow (Lima et al., 2021, p. 44). Taken together, these approaches illustrate how DSTLs separate test intent from automation details by combining abstract test commands with domain and configuration models (Santiago et al., 2013, p. 45).

For UI-test generation, related approaches abstract interaction behaviour at different levels. ViewModel Test (ViMoTest) specifies UI behaviour at the ViewModel level, independently of concrete GUI widgets, and uses logical widget representations and scenario DSLs to support readable specifications and code generation (Fuksa et al., 2025, pp. 753–754). Similarly, Le-Khanh et al. (2025, pp. 99–100) define User Behavior Language (UBL) as a DSL for abstract UI interactions in which commands combine user actions with information about the target elements. At a broader level, UI models can represent interface composition, content, events, actions, and data independently of the implementation technology (Brambilla et al., 2017, p. 114).

The broader DSTL landscape extends beyond recent UI-oriented script-generation approaches. Earlier work on specification-driven GUI testing represented test cases as high-level specifications from which executable event code and oracle logic could be generated (Assem et al., 2007, p. 84). Such model-driven approaches also allow test specifications to encode environment, platform, and harness information alongside user-action sequences (Santiago et al., 2013, pp. 45, 46). Bussenot et al. (2018, p. 433) further broaden this perspective by using BDD-style scenarios to orchestrate multiple DSTLs through intermediate transformation infrastructure. Work on executable-DSL testing and coverage also shows that testing support can be defined at the language level, including execution traces, coverage rules, and fault-localisation support (Khorram et al., 2024, pp. 3–4, 8). These approaches broaden the literature base from individual test-script DSLs to specification-driven, model-driven, and language-parametric testing infrastructures.

The reviewed approaches are better understood as addressing complementary stages in the transformation from behavioural specifications to executable UI automation. At the source level,

testing DSLs and BDD-style scenarios structure interaction intent. At the semantic level, step definitions, ViewModel abstractions, UI models, and intermediate representations connect that intent to supported actions and interface elements. At the target level, generation infrastructure translates the resolved representation into executable artefacts.

2.3.3 From Concrete Syntax to Validated Interaction Intent

Using a DSL to separate test intent from automation details shifts part of the engineering effort from writing test scripts to defining and validating the source language. A grammar can establish whether a scenario belongs to the concrete syntax of the language. In an ANOther Tool for Language Recognition (ANTLR)-based⁹ implementation, the grammar is operationalised by generated lexer and parser components that tokenise the input and check whether it conforms to the specified syntax (Parr, 2013, p. 22). For DSL-based test generation, however, syntactic validity alone is not sufficient. A scenario may conform to the grammar while still referring to an unsupported UI action, an unresolved interaction target, or a parsed assertion that cannot be represented in the target automation model. Compiler literature therefore distinguishes syntactic validation from semantic validation: parsing establishes the structural form of the input, while semantic checks determine whether the parsed constructs are meaningful (Fischer et al., 2009, p. 17). Parr (2010, p. 146) similarly frames semantic analysis as the phase that validates parsed input against language-specific meaning rules. This distinction also aligns with model-driven work that treats well-formedness checks as insufficient for semantic correctness (Brambilla et al., 2017, p. 170).

To support subsequent semantic validation, the result of parsing must be represented in a form that later compiler phases can inspect. The Abstract Syntax Tree (AST) serves this purpose by preserving the scenario elements and structural relationships relevant to those phases while omitting concrete-syntax details that are no longer needed (Fischer et al., 2009, pp. 15, 46; Parr, 2010, pp. 89, 93). Validating this representation before output generation reflects the compiler principle that target-independent analysis should be separated from target-specific generation (Fischer et al., 2009, pp. 17, 24).

2.3.4 Intermediate Representations and Model-Based Generation

Model-based generation separates source-language processing from target-artefact generation by introducing an intermediate representation between them. Parr (2010, pp. 292–293) notes that syntax-directed generation is limited for more complex translations because output is produced during parsing and cannot easily accommodate non-local semantic information or differences between source and target structure. A model-driven strategy can instead enrich an intermediate model before output is emitted (Parr, 2010, pp. 295–296). This is consistent with model-to-text approaches in which higher-level models are transformed into executable or analysable artefacts (Brambilla et al., 2017, pp. 141, 147). Related approaches apply this separation at different transformation boundaries. Domain-specific test descriptions can be transformed into test-bench artefacts through intermediate language infrastructure (Bussenot et al., 2018, pp. 433, 435–437);

⁹Official ANTLR website: <https://www.antlr.org/> (Accessed 30.08.2026).

ViMoTest generates executable code at the ViewModel layer (Fuksa et al., 2025, p. 754); and the UBL-based approach combines abstract UI-interaction specifications with automated test-case generation (Le-Khanh et al., 2025, p. 99).

Intermediate representations can support separation between source semantics and target-specific generation, but correctness still depends on the source model and the transformations applied to it, as defects may propagate into generated artefacts (Brambilla et al., 2017, pp. 169–171). Model- and DSL-based approaches can additionally support abstraction and reuse, while introducing costs related to language definition, implementation, maintenance, and tooling (Rodrigues Da Silva, 2015, p. 145). These dependencies across source processing, semantic validation, transformation, and target generation motivate an evaluation of both the generated artefacts and their execution.

2.4 Evaluation of Generated UI Tests

Generated UI tests can be evaluated across several stages from source specification through runtime execution and outcome assessment. Relevant concerns include source validity, target-artefact validity, execution readiness, and assessment of observed outcomes against expected results. The following sections review existing work as a basis for the evaluation methodology defined in Chapter 3.

2.4.1 Source and Target Validity

Source and target validity concern the constraints applied before and after generation. For specification-based generation, the source specification must first satisfy the constraints of the input language. Prior work on intermediate BDD representations shows that scenario text supplies some required elements, while domain-specific information supplies others (Zameni et al., 2024, pp. 200–201). For UI-oriented test generation, scenario-level behaviour must also connect to concrete interaction elements. Turner and Bowen (2024, p. 7) require explicit mappings between abstract interaction-sequence widgets and concrete Android UI widgets to enable correct code generation. Bündler and Kuchen (2019b, p. 7) show that references from scenario steps to UI artefacts can be validated before execution. After generation, the target artefact can itself be validated. Syntactically well-formed output is not necessarily a valid target artefact, as validity can also depend on semantic constraints (Georgescu et al., 2024, pp. 199–200).

2.4.2 Execution Readiness and Platform Execution

Generated test artefacts must be executable within their intended runtime environment before their outcomes can be assessed. Executable specifications can form part of automated builds (Smart, 2015, p. 322), while command-line execution supports repeatable runs in build and integration environments (Smart, 2015, p. 325). Execution readiness can depend on assumptions outside the generator. Turner and Bowen (2024, p. 11) report that generated Espresso tests require import steps, package-name updates, and completed prompts before execution, while ViMoTest requires application-specific setup classes to connect generated context data to the SUT despite producing JUnit test output (Fuksa et al., 2025, pp. 754–755). These examples

distinguish generation from execution readiness, as generated artefacts may still require manual completion, integration, or environment-specific setup before they can be executed.

In mobile UI testing, execution also depends on the platform environment. Cross-platform test-generation work reports configuration-dependent failures, for example when older operating-system versions expose UI structures that differ from reference devices (Menegassi & Endo, 2020, p. 33). Menegassi and Endo (2020, p. 34) further identify dynamic screens and platform-specific UI-attribute collisions as causes of element-selection failure during execution. These findings show that artefact validity alone does not establish successful mobile execution.

2.4.3 Expected Outcomes and Test Oracles

Executing a generated test does not by itself establish whether the observed behaviour is correct. Generated UI tests can encode behavioural expectations as assertions over observable interface states. Turner and Bowen (2024, pp. 9–10) use activity and mode assertions while inserting tester prompts when the models lack a concrete expected output. Their design illustrates both the role of executable assertions and the need for oracle information that may be absent from the source representation. Evaluation against expected outcomes may also include cases in which rejection or failure is the intended result. In grammar-based testing, SUT output is compared against an expected result, with positive inputs expected to be accepted and negative inputs expected to be rejected (Rossouw & Fischer, 2024, p. 3). Rossouw and Fischer (2024, p. 8) show that positive generated tests alone can be insufficient when the SUT accepts invalid inputs. Although this work is not mobile-specific, it illustrates the broader role of explicit expected outcomes when assessing generated tests.

2.5 Positioning Against Closely Related Approaches

ViMoTest, Slang, Silva et al., and Zamani et al. were retained as the closest comparisons because they overlap with central aspects of the approach investigated in this thesis. Table 2 summarises their respective abstraction levels, generated artefacts, evaluation strategies, and relation to this work.

Table 2: Comparison of closely related approaches along abstraction, generation, intermediate representation, and evaluation dimensions.

Approach	Input / level	Semantic abstraction	Generated / executed artefact	Evaluation	Relation to this work
ViMoTest Fuksa et al.	BDD scenarios specify GUI behaviour at the ViewModel abstraction level (Fuksa et al., 2023, p. 821).	ViewModels provide a GUI-framework-independent representation of rendering-relevant state. The ViMoTest-DSL separates View Description from Test Suite Specification (Fuksa et al., 2023, pp. 821, 823–824).	The approach transforms view descriptions and related test suites into executable code through model-to-model and model-to-text generation. The code generator targets Java (Fuksa et al., 2023, pp. 822, 825).	Fuksa et al. (2023, p. 830) evaluated their approach on five real-world applications by modelling 17 GUI views and creating 38 corresponding test scenarios.	Shares the use of behaviour-oriented BDD scenarios but applies them at the ViewModel level. This avoids rendering-framework dependence while leaving the final rendered interface unvalidated (Fuksa et al., 2023, pp. 821, 823–824).
Slang Bünder and Kuchen	Slang combines BDD-like feature descriptions, wireframe models, and mapping definitions (Bünder & Kuchen, 2019a, pp. 1743–1744).	Wireframe elements are linked to implementation elements through a mapping description, while scenario text can reference GUI model elements (Bünder & Kuchen, 2019a, pp. 1743–1744).	The generators produce executable GUI test scripts and page objects (Bünder & Kuchen, 2019a, p. 1745).	The evaluation uses an AB/BA crossover experiment against JBehave and reports average specification times for generated test cases (Bünder & Kuchen, 2019a, pp. 1747–1748).	Demonstrates how structured behavioural descriptions can be transformed into executable GUI tests, although Slang additionally relies on wireframes and mapping definitions (Bünder & Kuchen, 2019a, pp. 1743–1745).
Silva research line	User stories and interaction-level BDD scenarios connect business models, task models, prototypes, and final UIs (T. R. Silva & Winckler, 2017, pp. 5–6). The later DSL focuses on interaction scenarios for web GUIs (T. R. Silva, 2022, pp. 48–49).	A behaviour-based ontology provides a platform-independent vocabulary of UI interaction elements and supported behaviours (T. R. Silva et al., 2017a, pp. 250–251). Scenarios represent transitions between UI states (T. R. Silva et al., 2017a, pp. 252–253).	The approaches assess the consistency of UI-related artefacts; standalone test generation is outside their scope. Prototype and final-UI assessment is supported, with Selenium WebDriver used for final web interfaces (T. R. Silva et al., 2017a, pp. 255–256).	The early proof of concept detected inconsistencies across the analysed UI design artefacts (T. R. Silva & Winckler, 2017, p. 10). The later DSL was evaluated through valid and invalid combinations of widget actions, states, and properties (T. R. Silva, 2022, pp. 51–52); evaluation with end users remains open (T. R. Silva, 2022, p. 53).	The approach makes interaction-level UI scenarios machine-checkable; an end-to-end transformation into executable Android and iOS flows remains outside its implementation.
BDDTS / IBDD Zameni et al.	Gherkin scenarios are transformed into BDDTSs, formal MBT representations based on STSs with Given-preconditions and Then-postconditions (Zameni et al., 2023, pp. 37, 39–40).	BDDTS and IBDD act as intermediate formalisations that translate textual BDD scenarios into behavioural representations suitable for MBT. IBDD also makes missing, implicit, and ambiguous scenario information explicit (Zameni et al., 2024, pp. 199–200).	The transformation chain uses existing STS algorithms to generate test cases from the intermediate representations (Zameni et al., 2024, pp. 200, 204).	The approach is illustrated with seven scenarios whose BDDTSs are composed through pre- and postconditions; an example test trace is derived using an existing STS test-generation algorithm (Zameni et al., 2023, pp. 41–43). No systematic empirical evaluation is reported, while a general definition of composition and automated translation from textual scenarios remain open (Zameni et al., 2023, pp. 41, 43).	Supports the need for an intermediate representation that makes BDD intent precise before generation. Its formal MBT target provides conceptual support for the intermediate representation; direct mobile UI automation lies outside its scope.

3 Methods

The research questions are operationalised through a reproducible, bounded technical feasibility study of the compiler-based MobileBDD-to-Maestro workflow. The study examines whether the scenarios in the evaluation corpus $\mathcal{E}$, defined in Section 3.2.4, can be analysed and transformed into valid Maestro artefacts, and whether the generated flows produce the declared terminal outcomes in the configured Android and iOS environments. Eligible generated-flow outcomes are then compared with those of the corresponding manual Maestro flows.

For concise reference throughout this chapter, the following shorthand labels are used for the research questions stated in the Introduction:

- **RQ1 – Analysis and synthesis:** analysis of MobileBDD scenarios and generation of valid Maestro artefacts;
- **RQ2 – Platform execution:** conformity of generated-flow terminal outcomes with the declared expectations on Android and iOS;
- **RQ3 – Generated-manual consistency:** consistency between the raw execution outcomes of eligible generated and corresponding manual Maestro flows.

Configuration readiness is assessed first as a prerequisite to the scenario-level evaluation. The evaluation then follows the order represented by these shorthand labels because success at one processing boundary does not imply success at the next. This structure identifies the boundary at which a failure is detected without attributing its cause or claiming semantic equivalence beyond the recorded evidence.

The chapter first establishes the evaluation design and introduces the cross-platform SUT. It then describes the scenario corpus through a study-specific interaction taxonomy, a documented selection process, and an explicit development–evaluation assignment. The subsequent sections define the staged measures for RQ1–RQ3, describe the manual comparators used for RQ3, and specify the fixed ten-run execution and evidence-collection procedure.

3.1 Evaluation Design

The evaluation follows one fixed transformation workflow from controlled MobileBDD scenarios to generated Maestro flows and repeats their execution on Android and iOS. Evidence is recorded at the source, artefact, and runtime boundaries because a scenario may satisfy one stage but fail the next. The fixed configuration supports assessment of within-configuration repeatability only; comparison with alternative tools and generalisation to other applications or devices lie outside the design. The following sections define these boundaries and the SUT before introducing the corpus and measures.

3.1.1 Evaluation Boundaries

The object of evaluation is the transformation chain from controlled BDD-inspired source scenarios to generated Maestro YAML flows. This includes parsing, typed AST construction,

semantic validation against the AIM, Maestro-flow synthesis, target-artefact validation, platform execution followed by comparison of each eligible raw status with the scenario expectation. These operations are implemented by the compiler pipeline described in Chapter 4. The scenario variant is the unit of analysis. For RQ2 and RQ3, platform and repetition distinguish repeated observations of each variant.

The transformation chain is observed at five detection boundaries: source syntax, source semantics, target-artefact validity, expected terminal-status conformity, and eligible raw-status comparison. The first two jointly map to S_1 ; the remaining three map respectively to S_2 , S_3 , and S_4 , as defined in Section 3.3. Configuration readiness S_0 is a run-level prerequisite outside these scenario-level criteria. Recording the first unmet criterion establishes detection order, not causal fault localisation.

The accepted input space comprises the supported MobileBDD language subset and the AIM entries defined for the selected SUT. It excludes arbitrary Gherkin or unrestricted NL scenarios, unresolved targets, unsupported expected states, missing selector bindings, and interactions without generation recipes. The empirical scope is limited to the SalzburgCard SUT described in Section 3.1.2.

The purposively constructed corpus comprises 32 behavioural paths and 64 positive and negative variants. All RQ1–RQ3 measures use the 32 variants assigned to the evaluation corpus $\mathcal{E}$. The corpus is neither representative nor exhaustive of SUT behaviour. Sections 3.2.1 and 3.2.2 define its scope, while Section 3.2.4 documents the assignment to $\mathcal{D}$ and $\mathcal{E}$ and the final measurement freeze.

Execution is limited to the fixed Android and iOS environments specified in Section 3.5.1, with results reported separately for each platform. The repeated executions assess within-configuration repeatability only; device and OS-version diversity, physical devices, long-term behaviour, performance, and cloud-scale execution remain outside the evaluation.

3.1.2 System Under Test

Prototype role and construction. The SUT, SalzburgCard, is a purpose-built mobile tourism prototype for exploring and managing visitor activities in Salzburg. It was implemented with KMP for sharing application logic and CMP for sharing UI definitions across Android and iOS. The application uses a bundled local demo dataset and does not depend on an external application backend. These characteristics provide a controlled cross-platform subject for evaluating the MobileBDD-to-Maestro workflow. Development of the application itself lies outside the scope of this thesis; neither its architecture nor the selected multiplatform technologies are evaluated as contributions. The application revision is documented in Section 3.5.1.

Evaluated functionality. The application presents a catalogue of visitor attractions and points of interest in Salzburg, including Getreidegasse, Mozart’s Birthplace, Salzburg Zoo, Hohen-salzburg Fortress, and Mirabell Gardens. Visitors can browse the catalogue, open detail pages, search by name, and filter entries by criteria such as category or city area. These behaviours

constitute the predominantly stateless part of the evaluation subject because they do not depend on persistent user-specific data.

The prototype also provides stateful user-facing functionality. Visitors can create a local account, access their profile, save catalogue entries to a personal list, remove saved entries, submit reviews, and sign out. In these workflows, later assertions depend on earlier user input or persistent application state. Generated scenarios must therefore handle authentication, form input, persistent selections, and state transitions across screens.

Evaluated capability groups. The feature set covers *activity discovery* (browsing, searching, filtering, and opening activities), *activity reviews* (exploring and submitting reviews), *local authentication* (account creation, sign-in, and sign-out), and *saved attractions* (saving, viewing, and removing personal attractions). The selection is deliberately varied and explicitly non-exhaustive.

As Figure 1 illustrates, the shared Compose-based UI and unified design system provide comparable interaction states on Android and iOS. Platform-dependent map rendering, status bars, keyboard behaviour, and runtime details nevertheless require separate execution adaptations.

Evaluation boundary. The SUT is exercised exclusively through its UI using Maestro flows. Each top-level flow starts the application with cleared state so that local credentials, saved attractions, and submitted reviews do not leak between scenarios. Tests interact only with visible UI elements and exposed accessibility identifiers; they neither inspect nor instrument internal application state. Assertions are therefore limited to observable screen content and user-facing application state. At the execution boundary, the SUT is thus treated as a black-box mobile system.

3.2 Scenario Corpus

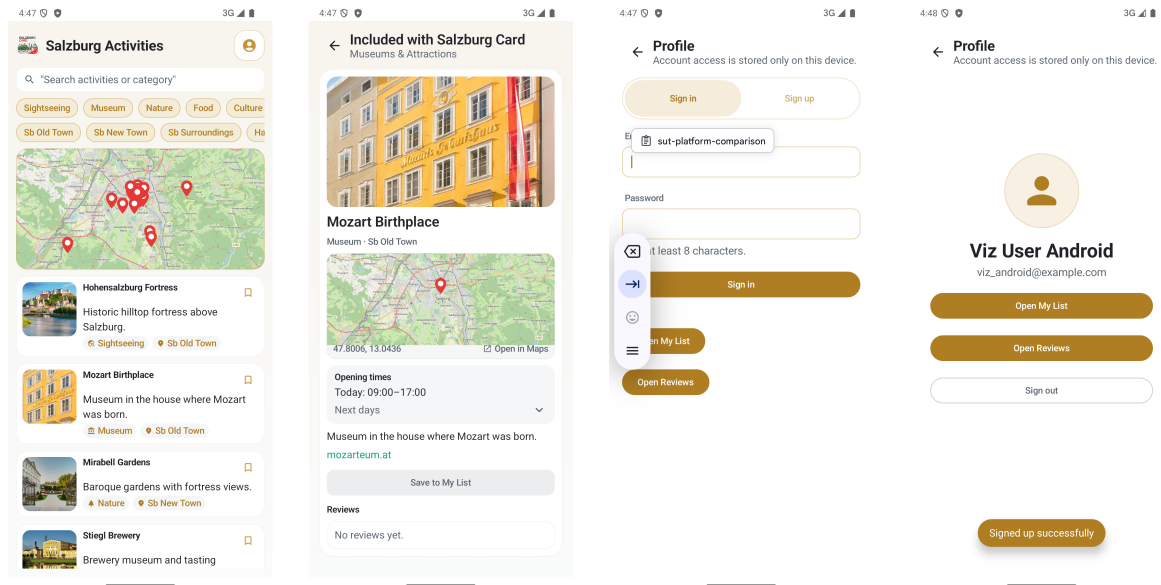
The scenario corpus provides the controlled source cases used to develop and evaluate the MobileBDD-to-Maestro workflow. This section defines the interaction taxonomy used to structure the corpus, documents its construction and audit trail, and specifies the assignment to the development corpus $\mathcal{D}$ and evaluation corpus $\mathcal{E}$ before final measurement.

3.2.1 UI Interaction Taxonomy

Although the literature reviewed in this thesis identifies recurring mobile interaction patterns, it does not provide a consolidated UI-interaction taxonomy suitable for structuring the scenario corpus. This thesis therefore defines eight study-specific interaction categories, informed by Deka et al. (2016, pp. 771–773) and Umar et al. (2024, pp. 3–4, 7). Together, these categories form the study-specific UI-interaction taxonomy used in this thesis. The taxonomy is neither independently validated nor intended to be exhaustive of mobile UI interaction.

Each category represents a user-observable interaction purpose. Collectively, the categories (i) reflect patterns recurring in the cited mobile-interaction literature, (ii) span the four SUT

Android



iOS

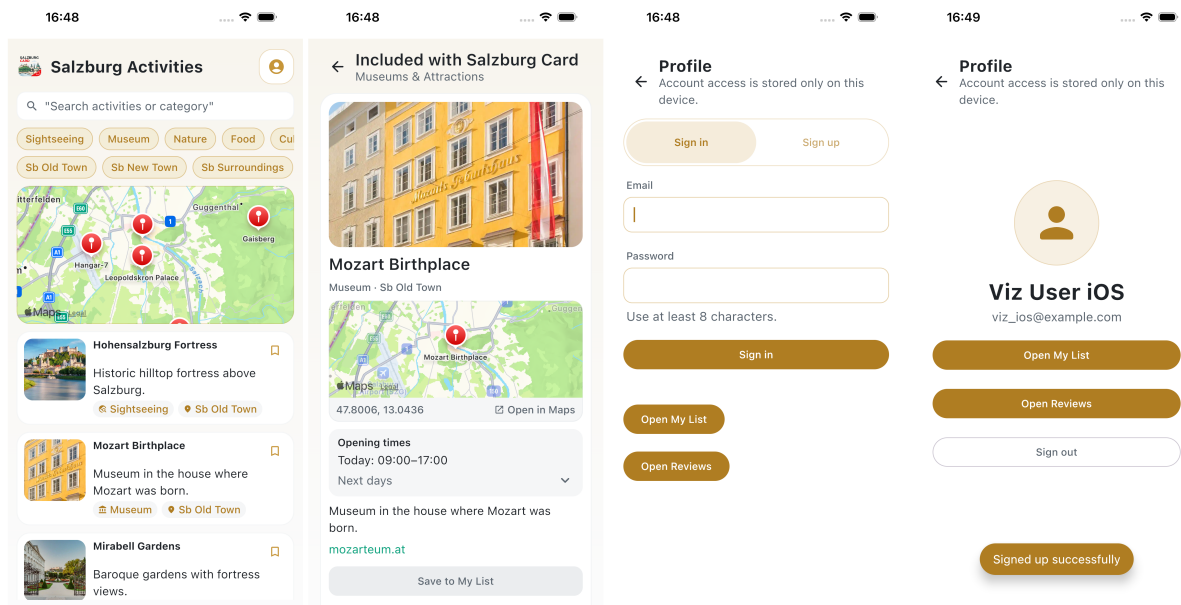


Figure 1: Representative screens of the system under test on Android and iOS, showing the activity overview, activity details, sign-in form, and authenticated profile.

capability groups described in Section 3.1.2, and (iii) distinguish scenarios according to their principal action–observation relationship. Because a scenario may combine several kinds of interaction, it is assigned to the category representing its *dominant* purpose. For example, a review-submission case may involve navigation and element discovery, but user-input composition remains dominant because entering and submitting the review constitutes the behaviour under test. The eight categories are:

- F1 Screen-to-screen Navigation Flow:** Navigating between screens or states to reach a target UI state and advance the task (Deka et al., 2016, p. 770; Umar et al., 2024, p. 6).
- F2 Query-based Retrieval Flow:** Locating content or functionality by issuing a query and interpreting results (Deka et al., 2016, p. 768; Umar et al., 2024, p. 3).
- F3 Content Exploration Flow (Scroll-based):** Exploring or consuming content primarily through scrolling and incremental inspection (Deka et al., 2016, p. 775; Umar et al., 2024, p. 22).
- F4 Element Discovery Flow:** Finding a specific UI element using visual, textual, or layout cues before acting (Deka et al., 2016, p. 772; Umar et al., 2024, p. 6).
- F5 Interaction–Response Flow:** Triggering an interaction and validating the resulting UI feedback or state update, including asynchronous updates (Deka et al., 2016, p. 770; Umar et al., 2024, p. 6).
- F6 Command Invocation Flow:** Executing an intended action by selecting a semantically meaningful control (e.g., share, like, or open) (Deka et al., 2016, p. 773; Umar et al., 2024, p. 7).
- F7 User Input / Composition Flow:** Entering structured or free-form input required to proceed with or complete a task (Deka et al., 2016, p. 770; Umar et al., 2024, p. 3).
- F8 Configuration Flow:** Adjusting settings or preferences (toggles, options, parameters) that change application behaviour or defaults (Umar et al., 2024, p. 7).

These categories provide the interaction-type axis used in the corpus construction described in Section 3.2.2. They support purposive case allocation but do not estimate the prevalence of interaction types in real applications or user journeys. Section 3.2.3 documents the case-level contrasts within each category.

3.2.2 Construction and Quality Review

The implemented SUT defined the behaviours available for corpus construction. The 32 cases were selected purposively from its observable behaviours.

Corpus construction proceeded as follows.

1. The four user-facing capability groups described in the SUT capability overview provided the functional-capability axis. Each group was represented by one feature-level story using

the conventional *As ... I want ... so that ...* template to capture its actor, goal, and benefit.

2. The eight study-specific interaction categories from Section 3.2.1 provided a second, cross-cutting axis. The capability groups identify *what* part of the application is exercised; the categories identify the dominant *kind of interaction*.
3. Candidate cases were required to exercise an observable behaviour within one capability group and to have a deterministic black-box postcondition on both platforms. Within each category, cases differed in interaction path, application state, data condition, or observation.
4. Four behavioural paths were selected per interaction category, with two assigned to $\mathcal{D}$ and two to $\mathcal{E}$. For each path, a positive scenario was written with the correct observable postcondition. A corresponding negative variant was then created by changing only that postcondition to an intentionally incorrect outcome. Each corpus subset therefore contains 16 behavioural paths and 32 scenario variants, giving 32 paths and 64 variants in total.

The resulting source remains domain-level and platform-independent. Concrete selectors, synchronisation decisions, and other operational mechanics are represented separately in the AIM described in Section 4.2.3 and in the manual reference flows specified in Section 3.4. The feature-level stories provide requirements context, while the executable scenarios are the units selected and measured in the evaluation.

Each scenario was assigned a unique identifier following the format $SCN-Fx-yy-P/N$. The prefix SCN denotes a scenario, Fx identifies one of the eight interaction categories defined in Section 3.2.1, and yy denotes the behavioural path within that category. The suffix P marks the valid expected-outcome variant, whereas N marks the corresponding variant with an intentionally incorrect expected outcome. For example, $SCN-F1-01-P$ identifies the valid variant of the first behavioural path in interaction category $F1$.

Tags classify scenarios by interaction category, expected outcome, and execution subset and support reproducible run filtering. The unique $SCN-Fx-yy-P/N$ tag links each source scenario to its generated artefact, validation evidence, runtime reports, and appendix records.

The scenario authoring process followed the question-based checklist of Oliveira et al. (2019, pp. 488–489). At the feature level, the review considered business value and completeness; at the scenario level, completeness, essentiality, uniqueness, singularity, and Gherkin integrity; and at the step level, terminology, focus, essentiality, and clarity. In practical terms, this means that each scenario was written to remain declarative, single-purpose, and platform-independent, while each negative variant preserves the same interaction path as its positive counterpart and differs only in the observable expected outcome. The checklist supports only the quality assessment of the authored BDD artefacts; representativeness of mobile applications or user behaviour remains unevaluated.

3.2.3 Audit Trail

Cases 01 and 02 in each category belong to $\mathcal{E}$, while cases 03 and 04 belong to $\mathcal{D}$. This corpus design is purposive and matched by category, not random. Some contrasts vary only domain data, whereas others cover a different application state or observation boundary. Equal case counts standardise corpus structure without implying equivalent behavioural breadth or representative behaviour. A case-level audit of the membership rule and its contrasts is provided in Appendix E, Table 22. Appendix F summarises the development and evaluation corpora in Tables 24 and 25, respectively.

Table 3 shows how the 32 cases are distributed across interaction categories and SUT capabilities. Each case has one positive and one negative variant, resulting in 64 variants overall.

Table 3: Case distribution by interaction category and capability

cat.	activity discovery	activity reviews	local authentication	saved attractions
F1	2	—	—	2
F2	4	—	—	—
F3	2	2	—	—
F4	—	2	2	—
F5	2	—	—	2
F6	—	—	2	2
F7	—	2	2	—
F8	4	—	—	—
Total	14	6	6	6

The larger activity-discovery share reflects that the evaluated SUT exposes navigation, retrieval, exploration, response, and two kinds of configuration through that capability. It should not be read as an estimate of feature importance or frequency.

Only scenario variants in $\mathcal{E}$ serve as units of analysis in the reported evaluation. Detailed version-control chronology and changes made during corpus preparation are retained as provenance in Appendix E, Table 23; they are not treated as evaluation results.

3.2.4 Development–Evaluation Assignment

The development corpus $\mathcal{D}$ and evaluation corpus $\mathcal{E}$ were stored in separate directory roots and used non-overlapping scenario identifiers, so $\mathcal{D} \cap \mathcal{E} = \emptyset$. Both roots existed before the measurement campaign; no later corpus split was performed. Because $\mathcal{E}$ was available during development, it is treated as a fixed, known in-sample corpus rather than an independently held-out evaluation set.

$\mathcal{D}$ supplied development fixtures and reference flows, while all reported RQ1–RQ3 measures were computed over $\mathcal{E}$. The pre-campaign freeze of the evaluated artefacts and execution protocol is specified in Section 3.5.2.

3.3 Operationalisation and Measures

This section defines the observation units, success criteria, fixed denominators, and decision labels used for RQ1–RQ3. The evaluation corpus $\mathcal{E}$ contains 32 scenario variants, and the evaluation design schedules ten cross-platform repetitions.

3.3.1 Evaluation Criteria

The identifiers S_0 – S_4 are diagnostic labels. S_0 is a run-level prerequisite; S_1 and S_2 operationalise RQ1, S_3 operationalises RQ2, and S_4 operationalises RQ3.

Each scenario declares an expected terminal status of `PASSED` or `FAILED`. Each runtime report supplies a raw status and a failure classification. A report is eligible when its status/classification pair is `PASSED/NONE` or `FAILED/ORACLE`. Status `ERROR`, a missing report, or a failed report classified as `INTERACTION`, `INFRASTRUCTURE`, or `UNKNOWN` is ineligible. Table 4 summarises the resulting pipeline prerequisite and evaluation criteria.

Table 4: Pipeline prerequisite and evaluation criteria

Identifier	Criterion
S_0	The AIM and runtime configuration can be read and validated for the combined run.
S_1	The parser/analyser constructs the typed AST and reports no semantic error (Aho et al., 2007, pp. 4, 8).
S_2	The generator emits a non-empty, complete Maestro YAML artefact that passes the configured target-syntax check (Holmes et al., 2018, p. 62).
S_3	The generated report is eligible and its raw status equals the scenario’s expected terminal status.
S_4	The generated and manual reports are eligible and their raw statuses are equal (Baresi & Young, 2001, pp. 4–5).

The criteria are assessed separately. Thus, S_4 can hold when S_3 does not if the generated and manual flows have the same eligible status but that status differs from the scenario expectation. Runtime classification uses the retained result artefacts, extracted diagnostics, process state, and exit code. Parsed failure or completion evidence takes precedence. Only when neither is available is exit code zero used as a documented fallback for `PASSED`, as described in Section 4.3.5. Equality under S_4 remains weaker than trace equivalence or complete behavioural correctness.

3.3.2 Observation Units

The scenario variant is the primary descriptive case. The 32 variants represent 16 positive/negative pairs that share 16 underlying interaction paths. S_1 is a source-file-level diagnostic; for RQ1, each scenario inherits the S_1 result of its containing file and succeeds only when its generated artefact also satisfies S_2 . One repetition therefore schedules 32 shared RQ1 observations.

For RQ2, the observation is one generated scenario–platform flow-run instance. For RQ3, it is the corresponding generated/manual scenario–platform pair. One repetition schedules 32 observations per platform for each RQ. Repeating the design ten times does not enlarge $\mathcal{C}$ or create statistically independent cases; the repetitions are reported descriptively.

3.3.3 Measures and Reporting

The design assigns the same number of scheduled observations to each repetition. S_0 remains a run-level prerequisite: if it fails, the affected scheduled scenario observations cannot count as successes. Table 5 defines the observation counted as successful and its fixed denominator for each RQ.

Table 5: Success rule and denominator for each research question

RQ	Observation counted as successful	Denominator
RQ1	The scenario’s source file satisfies S_1 , and its generated artefact satisfies S_2 .	32 per run; 320 total
RQ2	S_3 holds for the generated scenario–platform flow-run instance.	32 per platform and run; 320 per platform
RQ3	S_4 holds for the corresponding generated/manual scenario–platform pair.	32 per platform and run; 320 per platform

Compilation failures, missing reports, and ineligible runtime evidence remain in the denominator and therefore count as non-successes. For S_4 , ineligible pairs count as non-successes, not disagreements.

Because each repetition has the same denominator, the total fraction across ten repetitions equals the arithmetic mean of the run-level rates. Results report the total fraction and the run-level range where variation occurs. Per-scenario repeatability is reported as successful repetitions out of ten, and the paired Android/iOS intersection is a secondary count.

3.3.4 Diagnostic Ordering and Decision Labels

For diagnostic reporting, the first failed identifier is recorded in the order S_1 , S_2 , S_3 , and S_4 . An S_0 failure is recorded at combined-run level and prevents scenario-level classification for that repetition. This ordering identifies where retained evidence first becomes insufficient; it is not causal fault localisation.

H1 labels the RQ2 rate satisfactory when it reaches 90% on each platform; H2 applies the same study-specific threshold to the RQ3 rate. Section 5.5 reports sensitivity to alternative cut-offs.

3.4 Manual Comparator Construction for RQ3

RQ3 compares each generated flow with a platform-executable manual comparator according to the S_4 criterion defined in Section 3.3. Because the MobileBDD scenarios are specified at

the domain level, they do not uniquely determine an executable UI test on a concrete platform. Selector choices, synchronisation decisions, and other operational details must therefore be resolved separately when constructing each comparator. For this reason, the manual Maestro reference flows used in the reported RQ3 evaluation were implemented separately for the evaluation corpus $\mathcal{E}$. Each reference flow represents the intended interaction path on the SUT using only Maestro commands and selector mechanisms described in the official Maestro documentation.¹⁰

The manual reference flows were authored as separate Maestro artefacts without using generated YAML as input. The source MobileBDD scenarios defined the intended interaction paths, while selectors, synchronisation steps, helper flows, terminal observables, and Maestro command structures were selected manually against the running SUT. The purpose of these flows was not to establish an independent ground truth. Instead, they provided a separately implemented executable baseline for detecting raw-status divergences between generated and manually authored realisations of the same intended interaction path. The same researcher developed the compiler and authored the manual flows; consequently, S_4 provides evidence only of differential raw-status consistency. It does not establish test correctness, independent agreement, trace equivalence, or equality of the evaluated assertions. Section 6.3 discusses this limitation.

Before the final measurement campaign, each reference flow was iteratively checked against the running application. Expected-pass flows had to terminate with status `PASSED`; expected-fail mutants preserved the same interaction path and differed only in an intentionally incorrect final observable assertion. Once these conditions were met, the reference flows were included in the pre-campaign freeze defined in Section 3.5.2 and used as the manual comparators for the status-level differential comparison in RQ3.

Reference flows for the development corpus $\mathcal{D}$ were used to check the S_4 comparison workflow. The frozen reference flows for the evaluation corpus $\mathcal{E}$ were used in the reported RQ3 measurement. Both sets contain expected-pass entry flows, expected-fail mutants, shared action flows, and reusable helpers. Appendix G documents their expected terminal statuses, while Tables 26 and 27 provide their repository-relative paths.

3.5 Evaluation Procedure

With the evaluation corpus, measures, and manual comparators established, the final measurement campaign applied a fixed procedure across platforms and repetitions. This section specifies the execution environment, the repeated cross-platform pipeline protocol, and the collection and classification of the resulting runtime evidence.

3.5.1 Fixed Execution Environment

Controlled conditions. To reduce avoidable runtime variation, all reported executions were conducted under the fixed conditions documented in Appendix C, Table 18. The SUT was

¹⁰Official Maestro command reference: <https://docs.maestro.dev/reference/commands-available> (Accessed 15.08.2026).

evaluated on branch `feat/15-poi-review` at commit `cf607d8`. The evaluated compiler and pipeline implementation was fixed at commit `ef4b5a3`. The corresponding repository locations are listed in Appendix B, Table 17.

Android executions used the emulator `Medium_Phone_API_35`, while iOS executions used the simulator `iPhone 16 Pro` with `iOS 18.1`. The same host computer, application and compiler revisions, toolchain versions, platform targets, display configuration, locale, and permission state were retained for every reported repetition.

Before each complete cross-platform pipeline run, the application was installed with the application identifier expected by the generated and manual Maestro flows and restored to a deterministic baseline state. The Android emulator and iOS simulator were selected explicitly through their configured device identifiers. The application uses a bundled local demo dataset, so the evaluated user flows did not depend on an external application backend.

3.5.2 Repeated Pipeline Execution

Development and pilot executions were excluded from the final measurement set $\mathcal{R}$. Before the first retained run on 27 July 2026, the compiler and SUT revisions, source scenarios, AIM and runtime configuration, manual reference flows, platform targets, toolchain and host conditions, and execution protocol were fixed.

The protocol defined ten sequential executions of the complete cross-platform compilation and execution workflow. Each execution comprised all 32 variants in $\mathcal{E}$ and ran the generated and manual Maestro flows separately on Android and iOS. Let $r \in \mathcal{R} = \{1, \dots, 10\}$ denote one complete cross-platform pipeline run. For an interaction or infrastructure failure, the protocol permitted at most one retry. Both attempts were retained in the same report, and a successful retry was classified as a flaky pass.

These controls fixed the executed and recorded material; the final analytical framing of RQ2 and RQ3 remained unsettled. Their retrospective narrowing and its validity implications are disclosed alongside the research questions and further assessed by Section 6.3.

Each repetition started from the controlled application and device state described above and produced a separate version-controlled run bundle with a top-level report.

3.5.3 Evidence Collection and Failure Classification

The packaged pipeline runner performed the configuration checks, compilation, Maestro execution, and terminal-status comparison for every repetition. The same runner was used for generated and manual flows so that report parsing and stage classification followed one implementation path. For each scenario, platform, and repetition, the runner retained the stage status, raw Maestro status, `failureClassification`, `expectedStatus`, `implementationVerdict`, complete attempt history, `flakyPass`, standard output, standard error, screenshots, and machine-readable report artefacts. Section 4.3.1 describes the command-line integration and evidence implementation.

The reports were interpreted using the eligibility, success, and comparison rules from Section 3.3. Assertion details were retained for diagnosis and excluded from the S_4 comparison decision.

4 Implementation

The implemented compiler-based MobileBDD-to-Maestro workflow transforms MobileBDD scenarios through a typed Kotlin AST and the AIM into resolved interactions, generated Maestro YAML, and retained evaluation evidence. C4¹¹ views establish module responsibilities; the staged pipeline, a worked scenario, implementation-level verification, and development findings show how the design is realised.

4.1 Architecture

The implementation translates the controlled MobileBDD scenario corpus from Section 3.2.2 into executable Maestro flows. MobileBDD defines user-visible behaviour, while the AIM supplies the application-specific targets, capabilities, acquisition strategies, and platform bindings needed to execute it. Runtime configuration is a separate input used when Maestro is invoked; it does not influence parsing or semantic resolution.

4.1.1 C4 Architecture

The C4 views describe the static allocation of responsibilities, while Figure 6 represents the temporal S_0 – S_4 flow. The four C4 views consistently place interaction resolution in the language core and target generation and evidence handling in the CLI, keeping the architectural documentation close to the implemented structure (Jongeling et al., 2025, p. 15). This separation keeps module ownership distinct from execution order and covers only the evaluated toolchain.

System context. Figure 2 separates the developed solution from its operational environment. The solution owns transformation and evaluation orchestration. Maestro remains an external dependency that validates and executes the generated Maestro YAML flows against the Android or iOS application.

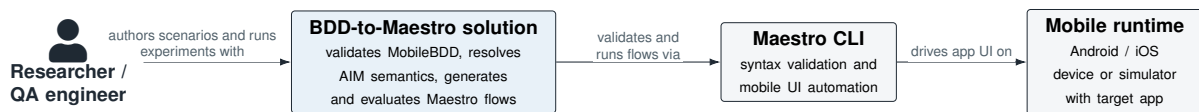


Figure 2: System context diagram of the BDD-to-Maestro solution, following level 1 of the C4 model. The system transforms restricted MobileBDD specifications and AIM application mappings into executable Maestro artefacts and evaluation evidence.

This boundary distinguishes compiler behaviour from target-runtime behaviour. Parsing, semantic resolution, and generation can be assessed before a device is involved; runtime outcomes also depend on Maestro and the selected platform.

Container allocation. Figure 3 allocates the workflow to the CLI, the language core, and the workspace artefacts. The language core transforms the MobileBDD specifications into a

¹¹Official C4 website: <https://c4model.com/> (Accessed 15.08.2026).

resolved interaction plan by parsing the input, constructing the AST, loading and validating the AIM, and performing semantic resolution. The CLI orchestrates the subsequent stages: it handles commands, generates Maestro flows, invokes Maestro for validation and execution, and compares the resulting reports with those of the manually authored flows. The workspace artefacts provide durable inputs and outputs for this process. They contain the MobileBDD specifications, imported AIM files, runtime configuration, and manual comparison flows, while also retaining generated Maestro YAML, execution evidence, reports, and comparison results.

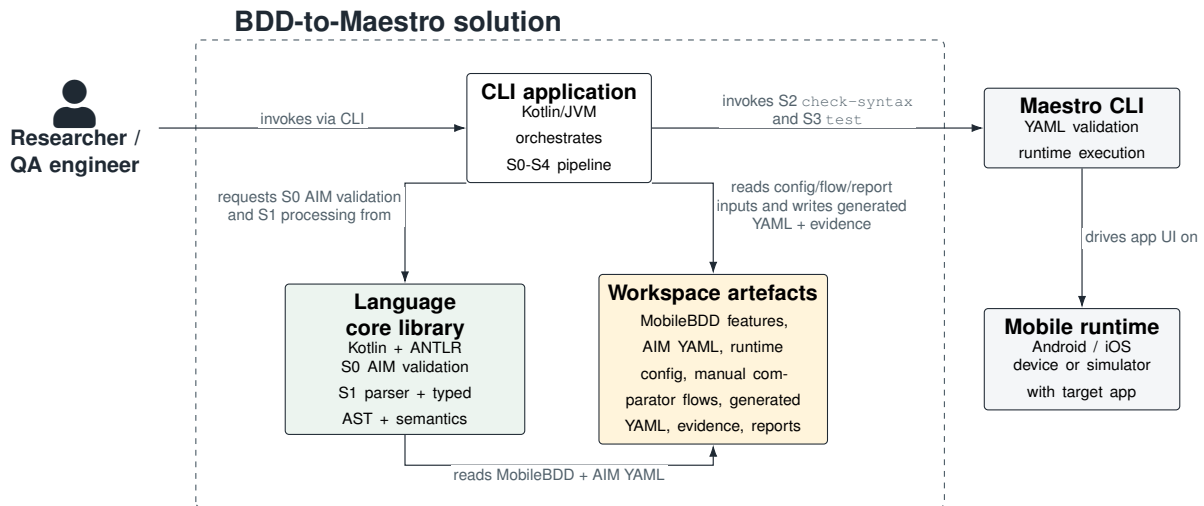


Figure 3: Container diagram of the current implementation, following level 2 of the C4 model. The CLI coordinates S0 configuration validation, requests S0 AIM validation and S1 processing from the language core, invokes Maestro for S2 target validation and S3 execution, and performs S4 comparison internally; workspace artefacts hold the durable inputs and outputs.

The resolved interaction plan is the container boundary that matters for later changes: the CLI does not depend on parse-tree structures or raw AIM configuration, and the language core does not manage Maestro processes or persist evaluation reports.

CLI application components. Figure 4 follows a command-line invocation through the orchestrator. Stage work is delegated to language resolution, YAML generation, syntax validation, runtime execution, evidence writing, and report comparison instead of being implemented in the command adapter.

The orchestrator controls stage order without duplicating parser or resolver logic. Both are requested through the language-core boundary.

Language-core components. Figure 5 exposes two paths. The syntactic path maps MobileBDD through the ANTLR parser and mapper into the typed AST. The model path loads, normalises, and validates the AIM. Both meet in the semantic resolver, which produces the interaction plan returned to the CLI.

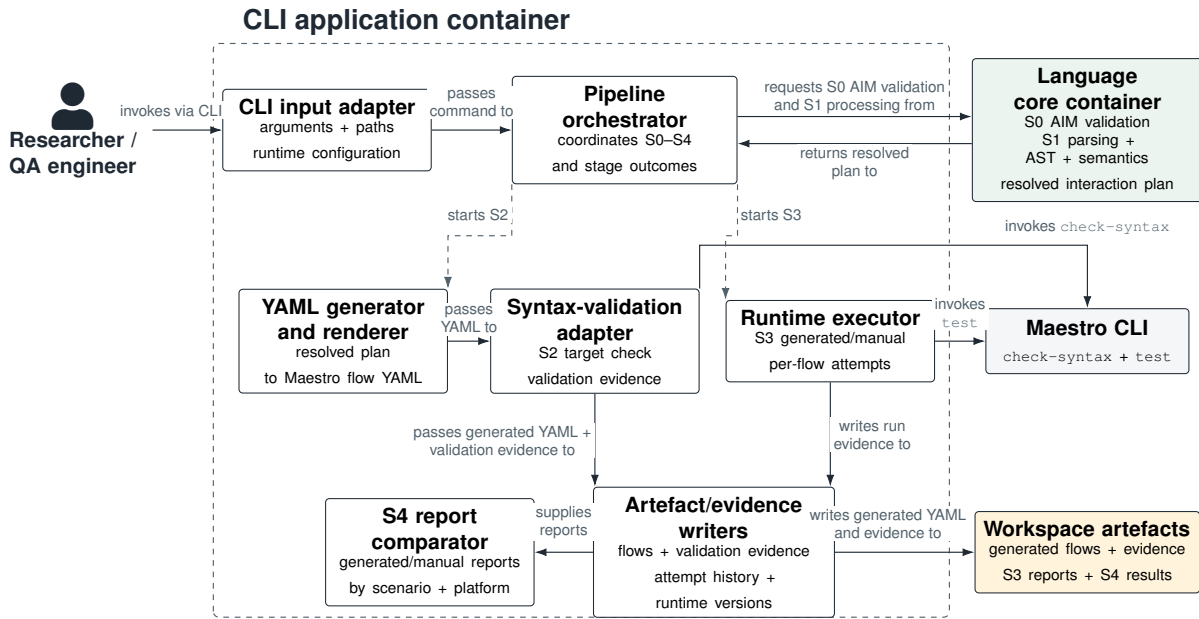


Figure 4: Component diagram of the CLI application, following level 3 of the C4 model. The CLI parses commands and orchestrates S0–S4, requests S0 AIM validation and S1 processing, receives a resolved interaction plan from the language-core container, and owns Maestro generation, target validation, runtime execution, evidence persistence, and S4 report comparison.

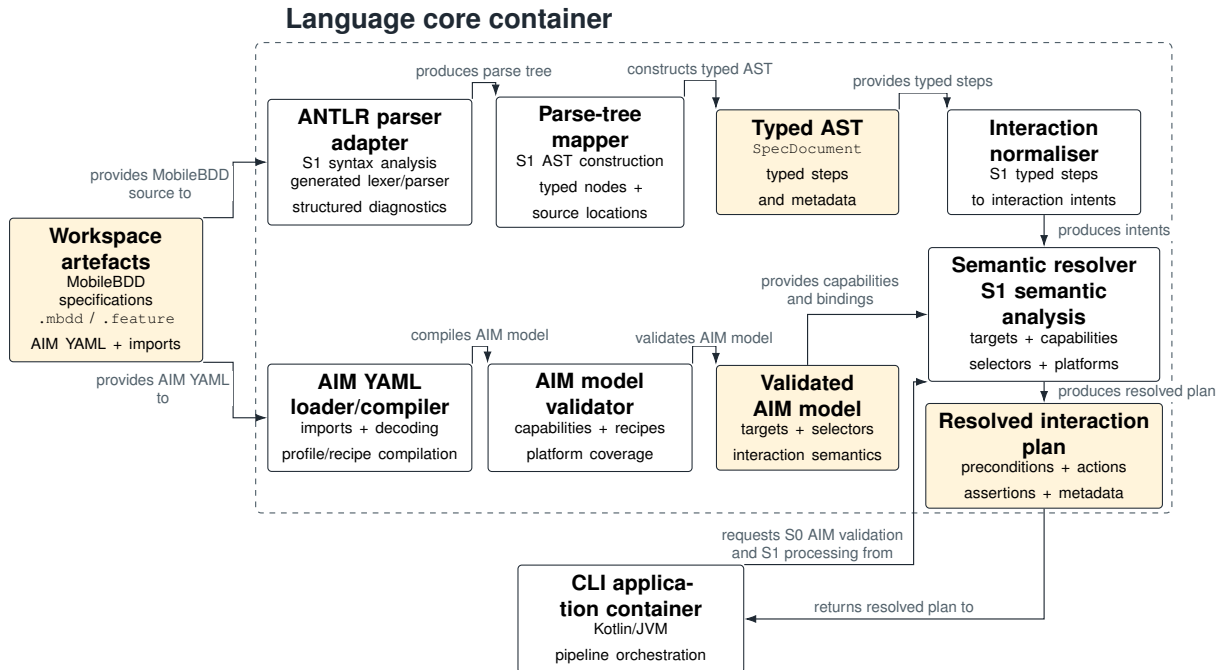


Figure 5: Component diagram of the language core and its immediate container context, following level 3 of the C4 model. The CLI requests S0 AIM validation and S1 processing; the core reads MobileBDD and AIM inputs from workspace artefacts, and its ANTLR adapter, AST mapper, AIM loader, validator, normaliser, and semantic resolver produce a resolved interaction plan for the CLI application.

4.2 Core Compiler Artefacts

The compiler is structured around a sequence of explicit artefacts that progressively transform readable MobileBDD scenarios into generation-ready interactions. The MobileBDD grammar defines the accepted source language, the typed Kotlin AST represents its syntactic structure independently of generated parser classes, and the AIM supplies the application-specific targets and capabilities needed for semantic resolution. Together, they produce a resolved interaction plan that the Maestro backend can translate into executable actions without processing the original scenario text.

These artefacts establish clear boundaries between syntax, application semantics, and target generation. Each stage consumes a validated representation from the preceding stage while retaining source information required for diagnostics. The following subsections describe these artefacts and the contracts through which they support the compiler pipeline.

4.2.1 MobileBDD Grammar

MobileBDD uses separate ANTLR lexer and parser grammars. The lexer recognises keywords, quoted values, placeholders, tags, numbers, and line breaks. The parser combines these tokens into features, user stories, backgrounds, scenarios, phase-ordered steps, and screenshot directives. This separation keeps character-level recognition independent of document structure and follows the conventional division between lexical and syntactic rules in ANTLR (Parr, 2013, pp. 22, 36). Listing 1 reproduces the lexer rules from `modules/language-core/src/main/antlr/MobileBDDLexer.g4` that most directly shape the token stream consumed by the parser; repetitive fixed-keyword and punctuation rules are omitted.

Listing 1: Selected implemented MobileBDD lexer rules

```

1 FEATURE: 'Feature';
2 SCENARIO: 'Scenario';
3 GIVEN: 'Given';
4 WHEN: 'When';
5 THEN: 'Then';
6 AND: 'And';
7 BUT: 'But';
8 STAR: '*';
9 AS: 'As' | 'as';
10
11 SCREENSHOT: '#' [ \t]* 'screenshot' ~[\r\n]*;
12 TAG: '@' [A-Za-z0-9_.-]+;
13 NUMBER: '-'? [0-9]+ ('.' [0-9]+)?;
14 STRING
15     : '"' (ESC | ~["\\r\n])* '"'
16     | '\'' (ESC | ~['\\r\n])* '\''
17     ;
18 WORD: [A-Za-z0-9_]+ ([-./'] [A-Za-z0-9_]+)*;
19

```

```

20 NEWLINE: '\r'? '\n';
21 WS: [ \t\f]+ -> skip;
22 COMMENT: '#' ~[\r\n]* -> channel(HIDDEN);
23
24 fragment ESC: '\\ ' [\'"\\/\bfnrt];

```

The fixed tokens keep document and step keywords distinct from general words; most are case-sensitive, with `As/as` as an explicit implemented exception. Line breaks remain visible because the parser uses them to delimit headers and steps, whereas horizontal whitespace is skipped and ordinary comments are placed on the hidden channel. Rule order is significant for screenshot directives: `SCREENSHOT` precedes `COMMENT` because both match a line beginning with `#`; for an equal-length match, the earlier rule preserves the line as an executable directive instead of hiding it as a comment.

The grammars contain no Kotlin actions and no Maestro-specific branches. ANTLR is used only for recognition and parse-tree construction; a separate mapper creates the handwritten compiler model. Keeping generation code out of the grammar prevents a target-language concern from changing the accepted MobileBDD syntax and allows syntax tests to run independently of the generator (Parr, 2013, p. 112).

The language implements a constrained set of familiar BDD elements. Its document layer provides features, scenarios, tags, and `Given-When-Then` phases. Within executable steps, actions and assertions follow controlled forms that can be mapped to typed compiler data. Application names, labels, user data, and expected messages remain variable scenario values and are not encoded as grammar productions. The grammar constrains sentence structure and remains independent of application-specific knowledge and Maestro commands.

The implemented parser structure is shown in two parts. Listing 2 covers the document and feature header, while Listing 3 covers the constrained Gherkin-style scenario body. Both listings are derived from `modules/language-core/src/main/antlr/MobileBDDParser.g4`. They flatten one-rule line and block wrappers and omit lexical expansion rules.

Listing 2: MobileBDD document and feature-header structure

```

1 spec
2     ::= (feature | NEWLINE)* EOF
3
4 feature
5     ::= tags? NEWLINE* FEATURE COLON titleLine NEWLINE
6         userStory NEWLINE* featureBody
7
8 userStory
9     ::= asLine wantLine soThatLine
10
11 asLine
12     ::= AS article? phraseLine NEWLINE
13

```

```

14 wantLine
15     ::= I WANT TO phraseLine NEWLINE
16
17 soThatLine
18     ::= SO THAT phraseLine NEWLINE
19
20 featureBody
21     ::= (background | scenarioDef | NEWLINE) *

```

Descriptions remain available as non-executable, human-readable context. Backgrounds contribute ordered setup items, tags carry scenario identity and evaluation metadata, and screenshot directives request evidence and remain non-behavioural scenario items. These distinctions allow the parser to retain the document structure while later stages operate only on typed scenario items.

The phase rules distinguish setup, action, and assertion blocks before semantic resolution begins. Continuation steps remain bound to their current phase, and the screenshot directive is represented as a non-interaction scenario item.

Listing 3: MobileBDD constrained Gherkin-style scenario structure

```

1 scenarioDef
2     ::= tags? NEWLINE* SCENARIO COLON titleLine NEWLINE
3         description? scenarioBody
4
5 scenarioBody
6     ::= preconditionPhase actionPhase assertionPhase
7
8 preconditionPhase
9     ::= scenarioInterlude* GIVEN typedAction NEWLINE
10        (scenarioInterlude* continuationStepKeyword
11          (typedAction | typedAssertion) NEWLINE) *
12
13 actionPhase
14     ::= scenarioInterlude* WHEN typedAction NEWLINE
15        (scenarioInterlude* continuationStepKeyword
16          typedAction NEWLINE) *
17
18 assertionPhase
19     ::= scenarioInterlude* THEN typedAssertion NEWLINE
20        (scenarioInterlude* continuationStepKeyword
21          typedAssertion NEWLINE) *
22
23 scenarioInterlude ::= screenshotDirective | NEWLINE
24
25 continuationStepKeyword ::= AND | BUT | STAR
26

```

```

27 typedAction
28     ::= actionHead argument? objectClause*
29
30 typedAssertion
31     ::= assertionSubject argument? objectClause* assertionMarker
32         assertionTail objectClause*
33
34 screenshotDirective
35     ::= SCREENSHOT NEWLINE

```

The representative source in Listing 5 later shows these rules in use. This structure also identifies the stage at which a diagnostic originates. A source file that violates the grammar produces a syntax failure before the AIM is consulted, whereas a syntactically valid step with an unsupported target proceeds to semantic analysis and produces a semantic failure. Section 4.3.3 describes how S_1 represents these two outcomes.

4.2.2 Typed Kotlin AST

The generated parse tree records how the source matched the grammar and remains tied to generated parser classes. The compiler therefore maps it to a handwritten Kotlin AST for use by subsequent stages. It retains features, scenarios, tags, step keywords, typed bodies, arguments, screenshot directives, source locations, and the raw text needed for diagnostics.

The parse-tree mapper is the only handwritten component that depends on generated parser contexts. It visits the recognised document hierarchy, normalises token values, and records one-based source positions on the resulting nodes. Accordingly, parse-tree-to-AST mapping belongs to source-language processing.

Scenarios contain ordered scenario items; executable steps carry their source keyword and a closed step-body type. The implemented type excerpt is reproduced once in Appendix D, Listing 8. Kotlin data classes provide value semantics for compiler data¹², while sealed interfaces keep the supported step-body forms explicit.¹³ The AST records only the accepted action or assertion structure. Application-target resolution, platform binding, and Maestro-command generation occur in later stages. That separation limits the effect of grammar changes on semantic resolution and generation.

Typed action and assertion bodies share the operation phrase, optional value, and object clauses, while assertions retain their assertion phrase separately. The implementation deliberately uses these generic typed bodies instead of one Kotlin class for every surface wording. The finite body hierarchy captures structural differences that later stages must handle; interaction aliases and application vocabulary remain configuration data.

Tags and source locations survive the mapping because they are needed after syntax recognition.

¹²Official Kotlin data-class documentation: <https://kotlinlang.org/docs/data-classes.html> (Accessed 15.08.2026).

¹³Official Kotlin sealed-class and sealed-interface documentation: <https://kotlinlang.org/docs/sealed-classes.html> (Accessed 15.08.2026).

Tags provide scenario identity, selection, and expected outcome metadata. Source locations connect semantic errors back to the original step even though the resolver no longer reads parser contexts. The raw body is kept for diagnostics, not as an alternative input to heuristic generation.

4.2.3 Abstract Interaction Model

The AIM supplies the semantic layer between the typed scenario and the Maestro backend. Each target has a stable identifier, accepted scenario terms, a semantic kind, supported capabilities, argument constraints, screen relations, and platform bindings. Application-specific selector and acquisition knowledge remains outside the grammar.

A target may represent a screen, element, result row, field, or another interaction subject, but its stable identifier is independent of the phrases used in a scenario. Capabilities define the actions and observable conditions supported by the target. This prevents a lexical target match from being treated as sufficient. An element named in the source must also permit the requested interaction in the current context.

A parsed step is resolved by matching its operation and target phrase to an AIM capability. Successful resolution yields a platform-independent interaction with a target, value, expected state, source location, and effective recipe. The generator later applies the Android or iOS binding. Shared bindings produce one Maestro command; differing bindings produce conditional platform-specific commands.

Screen relations add a small state model to this resolution. Targets may limit where they can be addressed, capabilities may limit their allowed source screens, and a successful interaction may set a destination screen. The resolver advances this screen state in scenario order and rejects an otherwise valid target when it is used from an unsupported location.

Configuration structure. The YAML compiler reads the configuration groups shown in Table 6. Together they form the contract between MobileBDD vocabulary, application identity, semantic resolution, and target generation.

This division isolates three decisions. The language layer normalises source wording. Targets and their nested capabilities determine whether an interaction is valid in the current screen context. Recipes specify how the validated interaction is translated into generator primitives. This separation keeps selector, synonym, and acquisition changes independent of the grammar.

Profiles provide the common MobileBDD operation semantics and default recipes. Imports and templates reuse target declarations across application models, while concrete target entries retain final authority over their capabilities and bindings. Loading and validation resolve these layers before scenario analysis, so S_1 gets initialised with an effective model instead of consulting several configuration files for each step.

Model validation checks stable identifiers, referenced templates and recipes, capability shapes, acquisition configuration, screen references, and required platform coverage. Invalid configuration is rejected by S_0 , so a missing selector or malformed recipe is not reported later as a scenario-language failure.

Table 6: Implemented configuration groups of the Abstract Interaction Model

Entry	Responsibility
profile, imports, targetTemplates	Select profile defaults, compose YAML documents, and reuse target declarations through template inheritance.
app.names, app.id	Specify accepted application names and shared or platform-specific application identifiers.
language, interactions. operations	Define action and assertion aliases and map normalised source operations to capabilities, target policies, setup rules, and value extraction.
actions, states, recipes	Provide global action and state capabilities together with named recipes that lower resolved capabilities into generator primitives.
targets, nested capabilities	Describe target identity, accepted terms, capabilities, argument constraints, acquisition, screen relations, platform bindings, and input sequences.

Representative mapping. Listing 4 shows the AIM declarations used by the worked example in Section 4.3.8.

Listing 4: AIM declarations used by the worked example

```

1 profile: mobile-core-v1
2
3 targets:
4   screen_activities:
5     id: "screen.activities"
6     terms: ["Salzburg activities"]
7     capabilities:
8       opened:
9         requiresSelector: false
10        recipe: [no_op]
11
12  activity:
13    id: "element.activity"
14    terms: ["activity"]
15    capabilities:
16      open:
17        acquisition:
18          strategy: "search"
19          query: "{value}"
20        bindings:
21          android:
22            text: ".*Search activities or category.*"
23          ios:
24            text: ".*Search activities or category.*"

```

```

25     bindings:
26         android: { id: "poi_row_.*" }
27         ios:      { id: "poi_row_.*" }
28
29     activity_details:
30         id: "element.activity.details"
31         terms: ["activity details for"]
32         capabilities:
33             displayed: {}
34         bindings:
35             android: { text: "Activity details" }
36             ios:      { text: "Activity details" }

```

The listing illustrates three capability configurations. The `screen.activities` target defines a selector-free assumed state whose `no_op` recipe emits no command. The `element.activity` target combines an open capability with search acquisition and a result binding. The `element.activity.details` target exposes a `displayed` capability with platform bindings for observable assertions. Recipes omitted from individual capabilities are supplied by the selected profile.

Semantic resolution and generation recipes. Matching proceeds from the normalised operation to targets of the required semantic kind. Target terms are compared after case and whitespace normalisation; no match is an error, and multiple matches remain ambiguous. The selected target must expose the required capability and satisfy its argument, screen, and platform-binding constraints. Appendix D.2 records the complete matching and configuration-precedence rules.

Operation semantics provide an indirection between source wording and target capabilities. Several action phrases may normalise to one capability, and a source operation may select different action and state capabilities depending on whether it occurs in an action or assertion body. Irregular forms can also define how their target and value are extracted. This keeps wording variants out of the target catalogue and prevents the generator from inferring semantics from raw text.

Recipes then translate a resolved capability into ordered generator primitives. They may be declared inline or supplied by the selected profile, allowing targets to share lowering behaviour while retaining their own selectors and screen constraints. Acquisition is also explicit: search, scroll, or swipe steps may prepare a target before its action or assertion is emitted. This distinguishes an interaction requested by the scenario from the lower-level steps required to address its target.

Action recipes cover launch, target addressing, taps, input, clipboard, keyboard, and navigation behaviour. State recipes select positive or negative visibility assertions or record a valid assumed state. The generator does not choose among these primitives heuristically: semantic resolution fixes the effective recipe, after which its steps are emitted in order. Representative recipe, language-alias, and operation declarations are shown in Listing 9.

Acquisition bindings and target bindings serve different moments in that sequence. A search acquisition addresses its input field and interpolates the scenario value into the query; the following target binding addresses the result. A scroll or swipe nested under acquisition is preparatory behaviour, whereas the same gesture declared as a target capability represents the interaction requested by the source. This distinction prevents low-level navigation from leaking into MobileBDD.

Selector bindings prefer stable accessibility or test identifiers when the application exposes them, while visible text remains available for interactions and observable assertions.¹⁴ Jetpack Compose semantics¹⁵ and test tags¹⁶ can expose metadata to external tools, and Compose Multiplatform maps semantic properties to native iOS accessibility properties.¹⁷ The relevant boundary is between scenario terminology, AIM targets, and their platform-specific bindings.

Resolved interaction plan. Semantic resolution outputs a backend-independent resolved interaction plan; Maestro YAML is produced later. This separation keeps the language core independent of process execution and allows the backend to translate the same validated intent into platform-specific output.

The plan groups interactions by scenario and preserves their order. Each entry contains the normalised operation, selected target and capability, optional value, effective recipe, source position, and scenario metadata. Resolved selectors remain associated with their platform branches. Equal branches produce a shared command; differing branches produce a conditional command.

This intermediate boundary also makes failure ownership explicit. Syntax and semantic errors occur before a plan is returned. Missing recipe primitives or incomplete emission are compiler backend failures. Maestro syntax rejection belongs to target validation, and device behaviour begins only after the artefact has passed S_2 . Section 4.3.8 demonstrates the complete hand-off.

4.3 Pipeline Stages

The implementation operationalises the evaluation criteria defined in Section 3.3.1 through concrete compiler, runner, and reporting responsibilities. Figure 6 provides an overview of these responsibilities and the artefacts retained at their interfaces.

¹⁴Official Maestro selector documentation: <https://docs.maestro.dev/reference/selectors/core-selectors> (Accessed 15.08.2026).

¹⁵Official Jetpack Compose semantics documentation: <https://developer.android.com/develop/ui/compose/accessibility/semantics> (Accessed 15.08.2026).

¹⁶Official Jetpack Compose testing-interoperability documentation: <https://developer.android.com/develop/ui/compose/testing/interoperability> (Accessed 15.08.2026).

¹⁷Official Compose Multiplatform iOS accessibility documentation: <https://kotlinlang.org/docs/multiplatform/compose-ios-accessibility.html> (Accessed 15.08.2026).

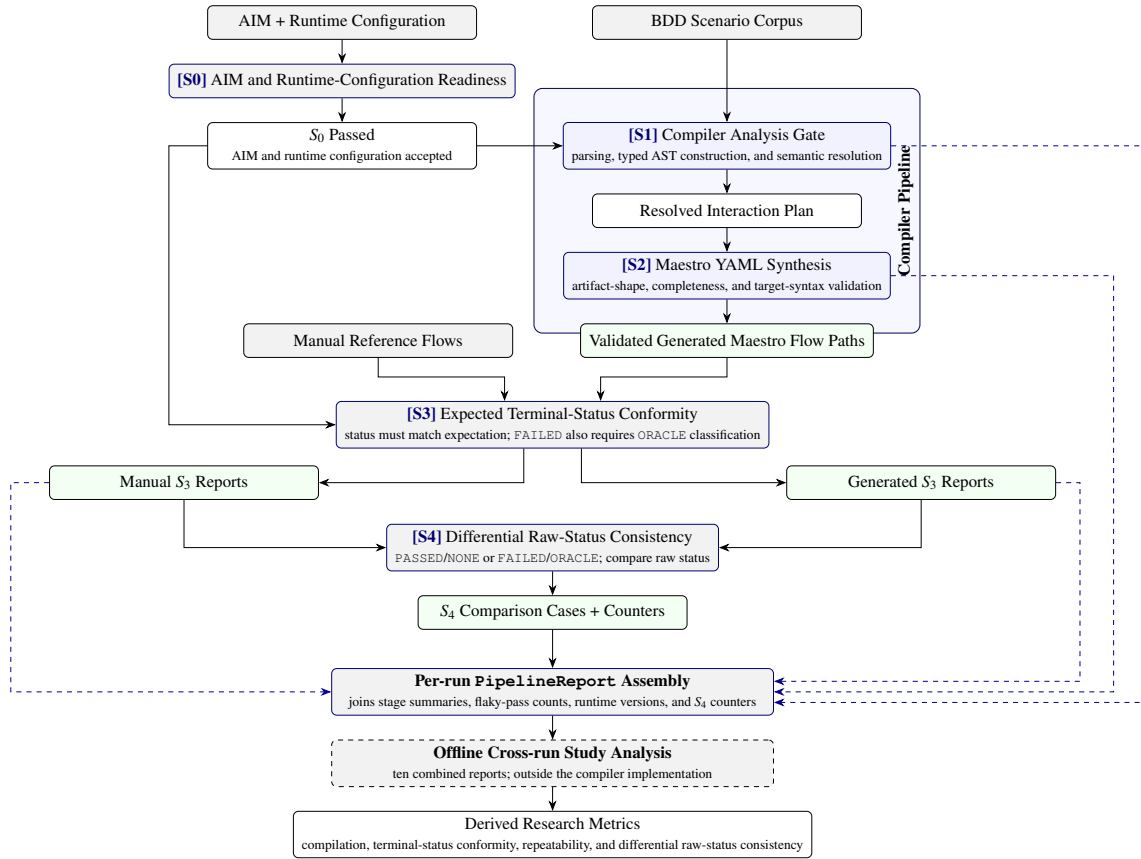


Figure 6: BDD-to-Maestro compiler and evaluation pipeline with implemented per-run reporting and offline cross-run metric derivation.

4.3.1 Pipeline Orchestration and Execution Configuration

The implementation follows the C4 container allocation shown in Figure 3, separating reusable language processing from operational orchestration. The `:language-core` module contains parsing, the typed AST, AIM handling, and semantic resolution. The `:cli` module packages the pipeline as an executable `.jar` and adds generation, Maestro process integration, evidence persistence, and report comparison. Terminal and Makefile entry points call this same artefact, avoiding a separate evaluation implementation.

The runner accepts a source file or directory, expands supported inputs, applies the optional tag expression, creates the output structure, and passes each file to the language core for processing. It then invokes generation and target validation only for successful plans. The integrated and standalone commands share validation logic and differ only in orchestration and retained path context.

The runner invokes `maestro check-syntax` during S_2 and `maestro test`¹⁸ during S_3 , passing the selected platform and device explicitly. After expanding a flow directory, the runner

¹⁸Maestro CLI `test` command and option reference: <https://docs.maestro.dev/maestro-cli/maestro-cli-commands-and-options> (Accessed 15.08.2026).

invokes Maestro separately for each YAML flow. Table 7 summarises the integrated commands. Execution-target discovery, tag-filtered standalone generation, and standalone S_3 execution are shown in Listings 13–15. The reported evaluation uses the complete $\mathcal{E}$ corpus (Section 3.5.2).

Table 7: Maestro CLI commands integrated by the pipeline runner.

Command	Role
<code>maestro check-syntax <flow.yaml></code>	Validate a generated artefact and record S_2 evidence.
<code>maestro test --platform <platform> --device <device-id> [driver flag] -e APP_ID=<id> <flow.yaml></code>	Execute one generated or manual flow as an isolated S_3 attempt.

Execution targets are discovered using the platform-specific terminal commands shown in Listing 13. Runtime configuration supplies the application identifier, platform, device, flow paths, timeout, additional arguments, and environment variables. The runner translates these values into the process invocation; Maestro does not read the configuration file directly. For an integrated run, the effective application identifier is selected from the CLI override, then the platform-specific AIM identifier, and finally runtime configuration. The generated YAML flow consequently remains independent of host-specific device selection.

4.3.2 S_0 Configuration Readiness

S_0 reads the AIM entry point and optional runtime configuration, validates the model invariants and required platform coverage, and retains the effective runtime settings for execution. Scenario sources are independent inputs to S_1 . Source expansion, output preparation, and device selection belong to orchestration and lie outside configuration-readiness checking.

The result distinguishes read or parse errors from validation failures. Warning-level AIM findings are retained and do not cause S_0 to fail. Only a successful result passes the validated model to S_1 and the runtime configuration to S_3 ; otherwise the run stops before scenario-level classification.

The implemented `S0ValidationResult` uses `ERROR` when an input cannot be read or parsed, `FAILED` when the model violates an error-level rule, and `PASSED` when neither condition occurs. Its persisted summary records both input paths, the outcome, warning and error counts, and their messages. This evidence is run-level because no scenario has entered the compiler when the gate fails.

4.3.3 S_1 Analysis and Semantic Resolution

S_1 parses each input file, maps the parse tree to the typed AST from Section 4.2.2, and resolves the selected scenarios against the validated AIM. The resolver checks target identity, capability support, arguments, screen context, and required platform bindings. A successful result contains the source document and a resolved interaction plan. Later stages no longer operate on source phrases or generated parser contexts.

Resolution proceeds in scenario order because screen transitions affect the validity of later steps. The returned plan includes only scenarios retained by the effective tag selection and carries their identifiers, categories, tags, and expected terminal statuses. Within each scenario, the interactions retain source positions so that a backend failure can still be connected to its MobileBDD step.

The sealed result type separates successful resolution, syntax failure, semantic failure, and unreadable input. Diagnostics retain a stable code, message, path, and source position when parsing has started. For example, the malformed input *Giv a visitor has opened the Salzburg activities* produces a step-keyword diagnostic with a `GIVEN` suggestion, whereas the valid input *opens missing activity “Old Town”* produces an `UNRESOLVED_TARGET` semantic error. Listing 10 reproduces the result types, and Table 19 provides the complete diagnostic examples.

S_1 is invoked per expanded input file. A successful plan is mapped to its scenario identifiers; an input-level failure rejects that compilation unit because no partial plan is returned. Tag filtering occurs after parsing and before semantic resolution, so filtering does not bypass syntax checks. The corpus manifest supplies the file-to-scenario mapping required by the measures in Section 3.3.1.

During S_1 , the compiler rejects missing or duplicate `SCN-` identifiers as semantic validation errors, preventing ambiguous artefact names and report mappings. Output names are derived from these workspace-wide identifiers, preventing two selected scenarios from overwriting the same artefact.

4.3.4 S_2 Maestro YAML Synthesis

S_2 consumes a resolved plan and lowers its actions and assertions to Maestro commands. The same plan and generation configuration produce the same YAML. Shared Android and iOS bindings are emitted once; differing bindings are placed in conditional `runFlow` blocks.

The backend maps recipe primitives to the supported Maestro subset, including application launch, target taps, input, navigation, visibility assertions, and target acquisition. Scenario identifiers and effective tags are written to the flow metadata, while the application identifier remains a runtime variable. This lets the same generated file run against the configured application. Machine-specific device and installation details remain outside the source scenario.

Generation also tracks whether every interaction in the plan contributed its declared recipe. This catches an interaction for which no recipe was emitted, even when the resulting YAML is structurally valid. The check covers emitted interactions, not individual components of a source postcondition.

Generated flows pass three checks before execution: the artefact must contain a complete, non-empty command document; every resolved operation must have been emitted; and the installed Maestro CLI must accept the file through `maestro check-syntax`. Using Maestro as the target-syntax validator avoids maintaining a second command schema inside the compiler. Each flow is stored with validation evidence containing its hash, the performed checks, their outcome, and reported issues. Listings 11 and 12 show the platform-branch and evidence examples.

4.3.5 S_3 Runtime Execution and Classification

S_3 starts a separate Maestro process for each flow. Each execution receives its own run directory containing standard output, standard error, and a machine-readable report. A provisional RUNNING report is written before process launch so that interrupted runs still retain their logs and invocation context.

Standard output and standard error are streamed to separate files while the process is active. The process boundary is responsible for concurrent stream consumption, timeout handling, termination, and final exit-code capture. Each flow is executed separately, so one process failure does not remove evidence from an earlier or later flow in the same batch.

Classification uses persisted output, extracted diagnostics, process state, and exit code. A recognised failed command or assertion determines FAILED; parsed completion evidence determines PASSED. ERROR is reserved for cases in which no usable test-level result is available, including launch failures, timeouts, interrupted execution, fatal runner errors, or an unexplained non-zero exit. Exit code zero is used only as a fallback when neither terminal nor failure evidence can be extracted.

The output parser first searches both streams for failure evidence such as a failed command, assertion, or missing element. In its absence, a recognised flow entry and at least one completed command constitute parsed success evidence. Runner exceptions are handled separately because they do not establish a test-level failure. This ordering prevents a zero exit code from hiding an observable failed command and prevents an infrastructure exception from being reported as an expected negative test result.

Table 8 summarises the resulting mapping from runtime evidence to raw status and failure classification.

Table 8: Implemented raw status and failure classification of an S_3 report.

Status	Implementation evidence	Failure classification
PASSED	Parsed completion evidence, with exit code zero as the documented fallback.	NONE
FAILED	A failed command, assertion, missing-element diagnostic, or other recognised test failure.	ORACLE, INTERACTION, INFRASTRUCTURE, or UNKNOWN
ERROR	No usable test-level outcome because execution could not start or complete reliably.	INFRASTRUCTURE

The report stores the raw runtime status, declared expected terminal status, and resulting evaluation verdict as separate fields. Their methodological interpretation belongs to the definition of S_3 in Section 3.3.1. This separation allows S_4 to compare the raw runtime-status field directly; evaluation verdicts remain an S_3 measure. Table 20 summarises the principal report fields, and Listing 16 provides a shortened JSON example.

Expected status is read from the generated flow metadata, with the scenario identifier suffix used as a compatibility fallback. An expected-fail scenario may therefore have raw status =

`FAILED` and `verdict = PASSED`, but only when its failure classification is `ORACLE`. More generally, the verdict passes exactly when the raw status equals the expected status and either the raw status is not `FAILED` or its classification is `ORACLE`. An interaction, infrastructure, or unknown failure cannot satisfy an expected-negative oracle.

An `INTERACTION` or `INFRASTRUCTURE` failure is retried once. The second attempt uses a `retry-1` directory below the first run directory and passes `-no-reinstall-driver`. The completed report keeps both attempts in order; its top-level status, run directory, log paths, command, and exit code describe the final attempt. The `flakyPass` field is true when the retry produces a passing verdict.

Within the integrated pipeline, the first generated flow on each platform requests driver reinstallation. Later generated flows, all retry attempts, and manual flows disable reinstallation. Driver installation is therefore requested only for the first generated flow on each platform.

The completed report also retains the exact command, exit code, timestamps, duration, log and screenshot paths, and extracted failure details. These fields support later inspection, so the aggregation layer need not parse the Maestro output again.

4.3.6 S_4 Runtime-Outcome Comparison

S_4 loads the generated and manual S_3 reports and pairs them by scenario identifier and platform. A report is comparison-eligible when its raw status–classification pair is `PASSED/NONE` or `FAILED/ORACLE`. If one side has no report, the case is `MISSING`. If either present report is ineligible, the case is `NON_EXECUTABLE`. This implementation category includes `ERROR` and terminal `FAILED` reports classified as `INTERACTION`, `INFRASTRUCTURE`, or `UNKNOWN`.

When both reports are comparison-eligible, unequal raw statuses produce `DISAGREED` and equal raw statuses produce `AGREED`. The comparison does not inspect assertion text or a normalised assertion signature. Assertion details remain diagnostic evidence in the S_3 reports. The `verdict` field is likewise excluded from the S_4 decision.

Reports that cannot be read or deserialised fail the standalone comparison configuration. A missing valid report for one member of an otherwise readable pair is instead represented at case level. This preserves the difference between an invalid S_4 invocation and incomplete runtime evidence for one scenario.

Integrated runs pass only the report paths created by the current invocation, which isolates repetitions. Standalone directory evaluation selects one report per scenario and platform using the completed timestamp and a deterministic path tie-breaker. Appendix D.3.1 documents the report-indexing and pairing rules, duplicate-selection policy, S_4 case classification, retained case fields, and aggregation behaviour; Table 21 summarises the implemented case outcomes, and Listing 17 provides a shortened report example.

4.3.7 Report Aggregation

Aggregation starts from all scenarios in $\mathcal{E}$, not from the successfully generated or executed report files, and left-joins the retained stage artefacts by scenario identifier. An absent compilation, execution, or comparison artefact therefore remains a non-success in the fixed denominator. RQ3 additionally distinguishes eligible unequal pairs from ineligible pairs.

The run-level report contains the S_0 summary, S_1 file and scenario counts, paths to S_2 flows and validation evidence, separate generated and manual S_3 summaries for each platform, and the S_4 case counters. Each S_3 summary records its flaky-pass count. The integrated pipeline processes the generated and manual S_3 batches separately on each selected platform and retains their report paths.

After the AIM and runtime configuration have been loaded, the orchestrator also inspects the runtime environment. The `PipelineReport` stores the Maestro version and, per platform, the effective application identifier and version, device identifier, and device-image or simulator-runtime version. Metadata-inspection failures are retained in `runtimeVersions.inspectionErrors`; compilation and execution continue. Stage-local files retain the detailed diagnostics and process output. This avoids duplicating full reports while allowing the evidence chain to be traced from a single run artefact.

4.3.8 Worked End-to-End Compilation and Execution Example

This example follows scenario `SCN-F1-01-P` from the evaluation corpus $\mathcal{E}$ through semantic resolution and generated Maestro YAML to retained runtime evidence. The scenario is used because it combines a state-only precondition, a value-carrying action, an observable assertion, and an OS-specific execution branch.

Listing 5: MobileBDD source used for the end-to-end flow

```

7 @SCN-F1-01-P
8 Scenario: Opening Getreidegasse displays its activity details
9 Given a visitor has opened the Salzburg activities
10 When the visitor opens activity "Getreidegasse"
11 Then activity details for "Getreidegasse" should be displayed

```

During S_1 , the source phrases are resolved against the AIM declarations in Listing 4; Maestro selectors are not embedded in the source or AST. Table 9 shows the three resolution decisions and why they produce the commands in the generated flow. Stage S_2 then emits these commands as the Maestro flow in Listing 6.

Table 9: Resolution of the worked scenario into Maestro effects

Step	Resolved meaning and reason	Generated effect
Given	Capability opened on screen. <code>activities</code> uses <code>no_op</code> because it declares the initial state rather than an interaction.	No command for the step; the configured baseline emits <code>launchApp</code> .
When	Capability open on element. <code>activity</code> uses <code>Getreidegasse</code> as a search query because the named activity must first be located.	Tap the search field, enter the value, submit on iOS, and tap the result.
Then	Capability displayed on element. <code>activity.details</code> uses the scenario value as the observable post-condition.	Assert that <code>Getreidegasse</code> is visible.

Listing 6: Maestro flow generated in stage S_2 for the worked scenario

```

1 appId: ${APP_ID}
2 name: "SCN-F1-01-P Opening Getreidegasse displays its activity details"
3 tags:
4   - "generated-s2"
5   - "capability-activity-discovery"
6   - "SCN-F1-01-P"
7 --
8 - launchApp:
9   clearState: true
10 - tapOn:
11   text: ".*Search activities or category.*"
12 - inputText: "Getreidegasse"
13 - runFlow:
14   when:
15     platform: iOS
16     commands:
17       - pressKey: "ENTER"
18       - hideKeyboard
19 - tapOn:
20   id: "poi_row_.*"
21 - assertVisible:
22   text: "Getreidegasse"

```

Run r_{10} (Table 12) completes the trace: the artefact passed all three S_2 checks; generated and manual flows passed S_3 on Android and iOS; and all 64 S_4 comparisons were AGREED.

4.4 Implementation Verification

Implementation verification follows the pipeline stages from Section 4.3. Source tests cover every pipeline stage. For S_0 – S_2 , they verify model loading, parsing, typed AST mapping, semantic resolution, artefact generation, and syntax validation. For S_3 – S_4 , they verify process handling, outcome interpretation, retries, evidence retention, and report comparison. They also cover cross-stage orchestration and aggregation. At revision 9174479, these tests can be run with `./gradlew :language-core:test :cli:test -rerun-tasks`.

Most source-test outcomes do not enter the reported measures, although the tests protect the underlying implementation behaviour. RQ2 and RQ3, however, depend on the interpretation of retained runtime evidence and the generated/manual report join. Table 10 records the cases exercised around these measurement boundaries. Subsections 4.3.5 and 4.3.6 define how runtime evidence is classified and how generated and manual reports are paired and compared.

Table 10: Measurement-boundary cases covered by source tests

Measurement boundary	Cases exercised
Expected-status resolution	Positive and negative expectation metadata, inline and block tags, filename fallback, conflicting declarations, and final outcomes after retry.
Runtime-outcome interpretation	Zero and non-zero exits combined with recognised command or assertion failures, runner exceptions lacking test-failure evidence, partial output, and timeouts.
S_4 comparison eligibility	Oracle, interaction, infrastructure, and unknown failures; terminal and non-terminal reports; and missing generated or manual counterparts.
Report pairing and decision	Equal and unequal raw statuses, exclusion of assertion details from the decision, scenario/platform keys, repeated batches, and deterministic duplicate selection.

Runtime-classifier fixtures are derived from retained Android and iOS outputs and sanitised to remove application secrets and device identifiers. Platform execution is evaluated separately from these hermetic source tests within the combined evaluation campaign.

4.5 Implementation Decisions and Lessons

Development and pilot runs exposed faults at the grammar, target-generation, and runtime boundaries before the repetition set reported in Section 3.5.2. The following cases are retained because they changed the implementation. Platform-specific runtime incidents and the resulting stabilisation measures are discussed in Section 4.6. The fixed execution environment and version information remain documented in Appendix C.

Step-keyword diagnostics. MobileBDD permits descriptions before scenario steps. Consequently, when `Giv` is written where `Given` is intended, the parser may accept it as description text rather than report it as an invalid step. An early solution used an ANTLR semantic predicate, which coupled the grammar to generated Kotlin parser code.

The final implementation keeps this check in the Kotlin validation layer. The grammar remains responsible for accepted structure; the validator inspects description regions for likely keyword mistakes and emits the structured diagnostic used by S_1 . This keeps the grammar portable and makes the diagnostic testable without target-language actions in the grammar.

Acquisition and selectors in the AIM. Several scenario targets are not addressable in the initial viewport. A direct `tapOn`¹⁹ is therefore insufficient for interactions that require search, scrolling, or repeated swipes. Runtime behaviour also depends on the scrollable region and current screen state. For example, a screen-centred `scrollUntilVisible`²⁰ command can miss a target contained in a smaller results list.

The implementation represents these steps as acquisition strategies and recipes in the AIM. MobileBDD states the requested interaction, while the AIM determines how the application-specific target becomes addressable. Refining a selector or scroll sequence then changes configuration and generation behaviour without extending the source grammar.

Android manual-flow normalisation. Fragment-based manual flows require their referenced action and helper files at runtime. For Android execution, the runtime can copy such a manual-flow workspace into the current run directory and apply three targeted changes to the copy. Because Android application state is reset externally, it removes the `clearState`²¹ option from every `launchApp`²² step. It wraps `scrollUntilVisible` in a `runFlow`²³ guarded by a `notVisible` condition, and inserts `hideKeyboard`²⁴ before the application-specific “Save review” submission sequence when it follows text input. The original manual comparator files remain unchanged. This normalisation is limited to the temporary Android workspace; standalone generated flows continue to execute from their original paths.

Syntax validation. `maestro check-syntax` validates each generated flow separately. Bounded parallelism reduces validation time, while starting a Maestro process for every flow still contributes to feedback time when all scenarios are processed. Using Maestro’s own syntax checker keeps validation consistent with the version used for execution. Tag filtering supports focused development runs, whereas the final evaluation processes all scenarios in $\mathcal{C}$.

¹⁹Maestro `tapOn` API reference: <https://docs.maestro.dev/reference/commands-available/tapon> (Accessed 15.08.2026).

²⁰Maestro `scrollUntilVisible` API reference: <https://docs.maestro.dev/reference/commands-available/scrolluntilvisible> (Accessed 15.08.2026).

²¹Maestro `clearState` API reference: <https://docs.maestro.dev/reference/commands-available/clearstate> (Accessed 15.08.2026).

²²Maestro `launchApp` API reference: <https://docs.maestro.dev/reference/commands-available/launchapp> (Accessed 15.08.2026).

²³Maestro `runFlow` API reference: <https://docs.maestro.dev/reference/commands-available/runflow> (Accessed 15.08.2026).

²⁴Maestro `hideKeyboard` API reference: <https://docs.maestro.dev/reference/commands-available/hidekeyboard> (Accessed 15.08.2026).

Runtime isolation. Runtime execution is serialised per device. Concurrent local Maestro processes compete for the same driver and device channels and produced startup or connection failures during pilot runs. The compiler can generate and validate artefacts independently; reliable parallel execution would require separate devices or external orchestration.

Taken together, the implementation establishes the research artefact as an end-to-end transformation from MobileBDD and AIM inputs to resolved, syntax-validated Maestro flows, with diagnostics, generated artefacts, runtime reports, and comparison results retained as a traceable evidence chain. The self-contained source-level tests verify compiler behaviour and process-boundary logic, while execution still depends on Maestro, the platform drivers, device state, and application state. Chapter 5 evaluates the completed artefact across these compilation, execution, and comparison boundaries.

4.6 Platform-Specific Runtime Stabilisation

Development and pilot runs exposed faults at the grammar, target-generation, and runtime boundaries. They preceded the reported repetition set of Section 3.5.2. The following cases are retained because they changed the compiler, validation, or runtime implementation.

Android emulator state. Pilot execution initially failed when the emulator lacked storage for the Maestro driver application. Resetting the Android Virtual Device and reinstalling the application restored execution. The repository target `make setup-android-reset` automates this reset while preserving and restoring the application under test.

Figures 7 and 8 show how system-level prompts displaced the application as the next interaction surface while the generated application-level steps remained unchanged.

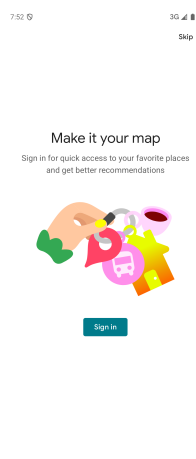


Figure 7: Android first-start dialog observed during pilot execution.

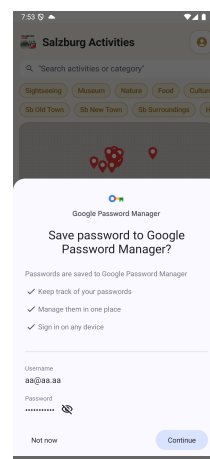


Figure 8: Password-manager prompt interrupting the application.

iOS text input and keyboard state. During pilot runs, the cursor in a Compose Multiplatform TextField could return to the beginning of the field and make scripted input unreliable.²⁵ Figures 9 and 10 show how simulator onboarding overlays introduced a separate source of interference after a flow focused a text field.

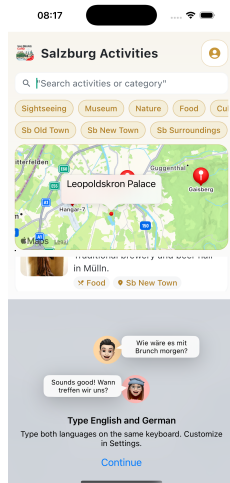


Figure 9: Multilingual keyboard tip observed in the simulator.

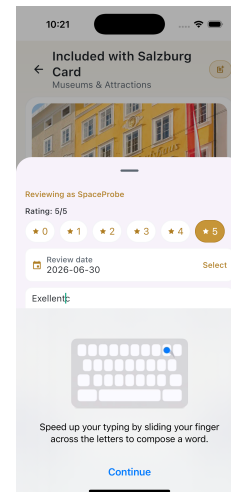


Figure 10: Continuous-path keyboard tip observed in the simulator.

The runner seeds the corresponding simulator preferences before text-input flows. The keys were identified empirically and are not officially documented as simulator interfaces, so they must be checked again after a runtime upgrade.

Listing 7: Seeding selected iOS Simulator keyboard preferences before text-input flows

```
xcrun simctl spawn "$IOS_DEVICE" defaults write \
    com.apple.keyboard.preferences DidShowContinuousPathIntroduction -bool
    YES
xcrun simctl spawn "$IOS_DEVICE" defaults write \
    com.apple.keyboard.preferences MultilingualKeyboardTip -bool YES
```

²⁵JetBrains issue CMP-7312: <https://youtrack.jetbrains.com/projects/CMP/issues/CMP-7312/TextField-cursor-jumps-back-to-start-on-iOS> (Accessed 15.08.2026).

5 Results

The final campaign retained evidence for all ten repetitions of the configured cross-platform pipeline. This chapter first establishes the completeness and configuration readiness of that evidence, then reports the measures for RQ1–RQ3 and the secondary decision labels H1 (RQ2) and H2 (RQ3). Fractions use the fixed scheduled denominators defined in Section 3.3; platform-specific totals and run-level ranges describe overall performance and within-configuration variation without treating the repetitions as independent cases.

5.1 Combined Evaluation Runs

The final campaign comprises ten fully retained combined pipeline repetitions $r \in \{1, \dots, 10\}$. Here, combined means that each repetition includes the shared compilation stage and separate generated and manual flow executions on both Android and iOS. Each repetition is retained in a uniquely named `both-<timestamp>-<random-suffix>` bundle. Its shared `pipeline-report.json` records S_0 , S_1 , and S_2 once, separate S_3 runtime summaries for each flow type and platform, and the resulting S_4 comparisons. The complete run bundles are available under `final_reports/` in the thesis repository listed in Appendix B. All ten bundles passed the configuration prerequisite S_0 and remain in the analysis. Eight bundles passed overall; r_1 and r_9 failed because a generated Android flow did not satisfy S_3 in each respective run.

For each repetition, the shared `pipeline-report.json` anchors the evidence and references the stage-local artefacts. Scenario identifiers link each source scenario to its generated flow, validation evidence, runtime reports, and comparison result, while the run directory identifies the repetition. Cross-run measures aggregate these retained outcomes for the 32 scenario variants in $\mathcal{E}$ across all ten reports, using the fixed denominators defined in Section 3.3.3. Within each report, S_1 and S_2 provide the shared compilation outcomes for RQ1, the generated-flow S_3 results provide the platform-specific outcomes for RQ2, and S_4 provides the generated/manual comparison outcomes for RQ3.

Before interpreting the cross-run measures, the completeness of the retained evidence must be established. Table 11 therefore compares the scheduled observations with the retained records at each stage of the campaign. All ten repetitions and all scheduled compilation and flow-run observations are present, yielding a complete set of 1,280 logical flow outcomes. Four retries increase the number of Maestro process attempts to 1,284; they remain part of existing logical executions and therefore do not change the fixed denominators.

Table 11: Completeness of the retained campaign evidence

Recorded quantity	Scheduled	Observed
Combined pipeline repetitions	10	10 retained; all S_0 passed
Shared compilation observations	320	320 recorded
Generated flow-run instances	640	640 terminal reports
Manual flow-run instances	640	640 terminal reports
Logical flow executions	1,280	1,280 retained outcomes
Maestro process attempts	1,280 minimum	1,284

Retained run inventory. With the campaign-level completeness established, Table 12 identifies the ten retained bundles and reports each S_0 result, warning count, and overall pipeline outcome. The bundle identifiers link these outcomes to the retained evidence.

Table 12: Ten retained combined cross-platform evaluation runs

Run	Bundle identifier	S_0 (warnings)	Pipeline outcome
r_1	both-20260727T171909Z-i7nbdD	PASSED (65)	FAILED (Android S_3)
r_2	both-20260727T191934Z-mTxQFY	PASSED (65)	PASSED
r_3	both-20260727T211831Z-t9ocnK	PASSED (65)	PASSED
r_4	both-20260727T231448Z-iU4ibv	PASSED (65)	PASSED
r_5	both-20260728T011027Z-YFBMKG	PASSED (65)	PASSED
r_6	both-20260728T030615Z-JwWTPj	PASSED (65)	PASSED
r_7	both-20260728T050542Z-f2KLMR	PASSED (65)	PASSED
r_8	both-20260728T070126Z-8Klcgb	PASSED (65)	PASSED
r_9	both-20260728T085950Z-cEb2NY	PASSED (65)	FAILED (Android S_3)
r_{10}	both-20260728T105718Z-ElmW0E	PASSED (65)	PASSED

The following subsections report the shared compilation result for RQ1 and the platform-specific execution and comparison results for RQ2 and RQ3. Table 14 presents the RQ2 totals, rates, run-level ranges, and non-perfect runs for Android and iOS. Table 15 reports the RQ3 generated/manual raw-status comparison by platform, distinguishing equal, unequal, and ineligible pairs.

Configuration-readiness findings. Each retained report contains the same 65 warning-level AIM findings. The validator records severity and message fields, with no stable diagnostic code available for grouping. Table 13 therefore groups the messages by finding pattern. This is one diagnostic set repeated in ten reports, not 650 independent findings, and the findings do not enter the scenario-level denominators.

Table 13: Disposition of the repeated S_0 AIM findings

Finding group	Count	Cause and materiality	Disposition
Text-selector robustness	59	The AIM uses text selectors where the validator recommends stable identifiers. Such selectors can become invalid when labels or localisation change.	Accepted for the fixed SUT and platform configurations; retained as a robustness limitation outside them.
Point-selector robustness	3	Three bindings use coordinates and are consequently sensitive to layout, screen, and device changes.	Accepted for the fixed emulator and simulator configurations; retained as a portability limitation.
Value-requiring acquisition	3	Search or swipe-until-visible acquisition consumes a value although the corresponding target-level argument is not declared required. The evaluated scenarios supply a value at each exercised use.	Accepted for the evaluated scenarios; omission of the value remains outside the demonstrated input scope.

None of the warning groups violates a required model invariant: the validator reports such violations as errors, in which case S_0 does not pass. The warnings are recorded only as scope and robustness limitations and do not enter scenario-level outcomes.

Retries and process attempts. As summarised in Table 11, the campaign contains 1,280 logical flow executions: 640 generated and 640 manual. Interaction and infrastructure failures were retried once, and four logical flows received a retry, increasing the number of Maestro process attempts to 1,284 without changing the logical-execution denominator. Three manual Android flows reached their declared expected terminal status on retry: $SCN-F3-01-N$ in r_1 , $SCN-F3-01-P$ in r_2 , and $SCN-F3-01-N$ in r_6 . The generated Android flow $SCN-F7-01-P$ in r_9 failed both attempts.

5.2 Shared Compilation Success (RQ1)

RQ1 asks what compilation success rate the compiler achieves when transforming the MobileBDD scenarios $\mathcal{E}$ into syntactically valid Maestro flows. Compilation is shared across platforms: the source corpus is analysed and the target artefacts are generated once per repetition.

S_1 is a source-file-level analysis diagnostic, whereas S_2 records scenario-level generated-artefact validity. RQ1 combines both criteria into the scenario-level compilation result. All four source files passed S_1 in each repetition. At scenario level, all 32 variants passed the complete S_1-S_2 compilation process in every repetition, giving 32/32 per run and 320/320 = 100% overall.

RQ1 therefore establishes compatibility with the evaluated corpus.

5.3 Expected Terminal-Status Conformity (RQ2)

RQ2 is measured directly by S_3 . A scheduled generated-flow observation satisfies S_3 when its retained report is comparison-eligible and its raw `status` equals its `expectedStatus`. The fixed denominator contains all 32 scheduled observations per platform and repetition, so a compilation failure, missing report, `ERROR`, or other ineligible outcome remains a non-success.

Table 14 summarises the ten run-level results. Android recorded one non-success in r_1 and one in r_9 , giving 31/32 in each of those runs; its other eight runs recorded 32/32. Every iOS run recorded 32/32. As a secondary paired count, 318/320 scenario repetitions satisfied S_3 on both Android and iOS.

Table 14: RQ2 results by platform

Platform	Successful	Rate	Run-level range	Non-perfect runs
Android	318/320	99.375%	96.875%–100%	r_1, r_9
iOS	320/320	100%	100%–100%	None

Observed non-successes and repeatability. Both non-successes were generated Android flows and had S_3 as their first failed criterion. In r_1 , `SCN-F3-01-P` had expected status `PASSED` and observed status `FAILED`; the failure was classified as `ORACLE`. In r_9 , `SCN-F7-01-P` had expected status `PASSED` and observed status `FAILED`; both process attempts were classified as `INTERACTION`. No final report contained `ERROR` or missing runtime evidence. The interaction-classified failure is consequently ineligible for the S_4 comparison.

Each affected scenario satisfied S_3 in 9/10 Android runs; every other scenario–platform combination satisfied S_3 in 10/10 runs. The first-failed classification identifies the detection point in the retained evidence, not the causal origin of either failure.

Assertion-completeness sensitivity. The primary rates include 20 observations of `SCN-F7-02-P/N` on each platform. For these variants, the retained generated flows assert only comment visibility. The activity and rating remain part of the executed path without being independently asserted after submission. A conservative post-hoc analysis retained all ten repetitions and the original 32-variant denominator but assigned no conformity credit to these observations. The resulting rates would be $298/320 = 93.125\%$ on Android and $300/320 = 93.75\%$ on iOS. At run level, Android would range from 90.625% to 93.75%, while iOS would remain at 93.75% in every run. This bound supplies neither the missing assertions nor evidence from a corrected artefact or later execution.

5.4 Differential Raw-Status Consistency (RQ3)

RQ3 is measured directly by S_4 . For each platform and repetition, all 32 scheduled pairs remain in the denominator, and a pair contributes to the numerator only when both reports are eligible

and their raw terminal statuses are equal. Unlike S_3 , S_4 does not compare either status with the scenario expectation and can therefore hold when S_3 does not.

All 640 generated-flow reports and all 640 manual-flow reports recorded a terminal raw status. Of the 640 scheduled pairs, 639 were eligible and one was ineligible. Table 15 reports the resulting platform totals.

Table 15: Aggregate differential raw-status consistency

Platform	Scheduled	Equal	Unequal	Ineligible	Rate
Android	320	318	1	1	318/320 (99.375%)
iOS	320	320	0	0	320/320 (100%)
Both	640	638	1	1	638/640 (99.6875%)

Android recorded 31/32 eligible and equal pairs in r_1 , 31/32 in r_9 , and 32/32 in the other eight runs, giving a run-level range of 96.875%–100%. In r_1 , the pair for SCN-F3-01-P was unequal: the generated status was `FAILED` with `ORACLE` classification, whereas the manual status was `PASSED`. In r_9 , the pair for SCN-F7-01-P was ineligible after two interaction-classified generated failures. Every iOS pair was eligible and equal in every run.

Terminal-observable-alignment sensitivity. The primary analysis records 638/640 eligible and equal pairs. This rate includes SCN-F4-01-P/N and SCN-F4-02-P/N, whose generated and manual terminal assertions concern different UI observables. These four variants contribute 80 comparisons across ten repetitions and two platforms.

A post-hoc sensitivity analysis restricts each repetition to the remaining 28 variants without replacing any artefact or execution. Android then has 278 equal, one unequal, and one ineligible pair among 280 scheduled pairs, giving $278/280 = 99.2857\%$. All 280/280 iOS pairs are equal, and the combined rate is $558/560 = 99.6429\%$. The Android run-level range is 96.4286%–100%.

All 80 omitted pairs were equal in the primary analysis, so the restricted rate is slightly lower. Generated/manual terminal-observable alignment covers $28/32 = 87.5\%$ of the corpus. The restricted analysis measures raw-status consistency only within this aligned subset and does not establish semantic agreement; the F7-02 compound-postcondition boundary remains within the subset. This analysis was not part of the original evaluation protocol.

Scenario polarity. Both Android non-successes occurred among positive scenarios: 158 of 160 positive pairs were equal, one was unequal, and one was ineligible. All 160 Android negative pairs were equal. On iOS, all 160 positive and all 160 negative pairs were equal.

The S_4 measure is limited to eligible raw-status equality; assertion text, failure signatures, interaction traces, and complete scenario semantics are outside its scope. The resulting construct-validity limitation is examined in Section 6.3.

5.5 Results Summary and Decision Labels

Table 16 provides a compact overview of the primary results reported in the preceding RQ1–RQ3 sections. The rates use the final retained outcomes and the fixed scheduled denominators defined in Section 3.3; missing or ineligible observations therefore count as non-successes.

Table 16: Answers to RQ1–RQ3

RQ	Result	Critical boundary
RQ1	Compilation succeeded for 320/320 scheduled scenario observations (100%).	$\mathcal{E}$; no evidence of generalisation to unseen scenarios.
RQ2	Expected terminal-status conformity was 318/320 (99.375%) on Android and 320/320 (100%) on iOS.	The criterion checks terminal status, not complete scenario semantics; F7-02 asserts only comment visibility.
RQ3	Android recorded 318 eligible and equal pairs, one unequal pair, and one ineligible pair among 320 scheduled pairs (99.375%); all 320/320 iOS pairs were eligible and equal.	S_4 ignores assertion identity and interaction traces; four F4 variants lack generated/manual terminal-observable alignment.

Decision labels. The study-specific rule in Section 3.3.4 requires the respective rate to reach 90% on each platform. Applied to the primary results in Table 16, H1 (RQ2) and H2 (RQ3) are satisfied on both platforms. RQ1 has no decision threshold.

Retry sensitivity. Four of 1,280 scheduled flows were retried (0.3125%). Three manual Android flows recovered, in r_1 , r_2 , and r_6 . Under first-attempt-only scoring, the Android RQ3 result would have been 315/320 (98.4375%); the recoveries increased the final result to 318/320 (99.375%). Retries did not change RQ2 and increased complete-bundle success from 6/10 to 8/10. H2 remained satisfied.

Decision-rule sensitivity. With the primary rates, H1 and H2 remain satisfied at whole-percentage cut-offs through 99% and fail at 100% because the Android rate is 99.375%. Under conservative F7-02 scoring, H1 is satisfied at 90% and fails at 95%. The restricted F4 analysis does not change H2 at cut-offs up to 99%. These arithmetic outcomes do not validate the author-selected 90% rule; the descriptive counts and their evidence boundaries remain the primary results. Chapter 6 discusses the implications of these scope and measurement boundaries.

6 Discussion

This chapter interprets the findings for RQ1–RQ3. It synthesises their main implications, relates them to the most relevant prior work, and examines the limitations and threats to validity that bound the study’s conclusions.

6.1 Interpretation of Findings

Eight of ten bundles passed; the failures in r_1 and r_9 show imperfect Android stability. Three recovered manual Android flow-run instances make the final RQ3 rate and number of successful complete bundles dependent on the retry policy. The 90% H2 label remains unchanged. The secondary decision labels add no evidence beyond the descriptive results: primary scoring remains stable below a perfect-score requirement, whereas conservative F7-02 scoring changes H1 at the higher tested cut-off. Fixed denominators and run-level variation are therefore more informative than the binary labels.

Accepted AIM findings. The configuration gate passed because the repeated S_0 findings were advisory warnings; a violated required invariant would have closed the gate. The 59 text-selector and three point-selector findings identify potential fragility under changed labels, localisation, layouts, or devices. The three acquisition findings identify inputs for which the evaluated scenarios supplied values although the corresponding target-level arguments were not declared required. The warnings did not prevent compilation or scheduled execution in the fixed configuration; their effects under other configurations remain untested.

Compilation success (RQ1). The complete compilation result establishes fixed-corpus, in-sample compatibility among the MobileBDD language, AIM-based semantic resolution, Maestro generation, and the corpus $\mathcal{C}$. Unseen MobileBDD scenarios remain unevaluated. The negative variants are valid compiler inputs with expected FAILED outcomes and share their interaction paths with the positive variants.

Expected terminal-status conformity (RQ2). The RQ2 results contain two Android non-conformities with different S_3 failure classifications (Section 5.3). SCN-F3-01-P reached an oracle-classified failure where PASSED was expected, whereas SCN-F7-01-P ended with an interaction-classified failure after both attempts. The evidence identifies the detection stage and failure class only; it provides no basis for a common causal explanation. Moreover, S_3 measures only PASSED/FAILED conformity: for F7-02, the retained assertion covers comment visibility, leaving the stated activity and rating components unasserted.

The primary RQ2 counts use final outcomes. They equal the first-attempt counts because the retried generated flow did not recover; the attempt logs nevertheless record both attempts, and H1 remains unchanged. More broadly, platform-specific knowledge resides in the AIM, selectors, recipes, and execution configuration. The result therefore applies to this configured toolchain as a whole. Although both non-successes occurred on Android, observations from one emulator and one simulator under a fixed execution order cannot rank platform reliability.

Differential raw-status consistency (RQ3). The generated-flow outcomes identified under RQ2 also account for the Android S_4 non-successes reported in Section 5.4. Both affected positive variants, whereas every negative pair satisfied S_4 . This pattern is descriptive only; assertion details did not enter the decision.

The fixed denominator matters in r_9 : the report is ineligible, not unequal, but the run still records only 31/32 successful pairs. The missing comparison therefore cannot disappear from RQ3 or its secondary H2 label.

The eligible and equal negative pairs provide status-level evidence only. The implementation accepts equal executable statuses even when the retained failing-assertion details differ, leaving assertion-level and trace-level equivalence unresolved.

The post-hoc sensitivity analysis of terminal-observable alignment (Section 5.4) omits SC N-F4-01-P/N and SCN-F4-02-P/N because their generated and manual terminal assertions concern different UI observables. It retains all ten repetitions for the remaining 28 variants. The restricted result leaves the H2 classification unchanged, while primary H2 remains defined over the complete 32-variant corpus. Neither result establishes semantic agreement.

6.2 Relation to Prior Work

Prior work addresses different parts of the path from behavioural specification to executable mobile UI tests. Direct comparisons of performance or reliability are therefore not justified. The findings can instead be related to earlier work through three evaluation boundaries: semantic mediation and compilation, execution against rendered interfaces, and assessment through expected outcomes and manually authored comparators. Across these boundaries, the contribution of this thesis is not the removal of platform-specific knowledge, but its separation from shared behavioural intent and its explicit treatment during generation and evaluation.

Compilation and semantic mediation (RQ1). The complete RQ1 result is consistent with Slang’s demonstration that structured behavioural descriptions and GUI information can be transformed into executable tests (Bünder & Kuchen, 2019a, pp. 1743–1745). Slang combines behavioural descriptions with wireframes and explicit mappings, whereas MobileBDD uses a constrained behavioural language together with the AIM. Both approaches therefore address information that unrestricted scenario text does not encode, but place that information in different supporting artefacts.

The result also supports prior arguments for an explicit semantic layer between behavioural descriptions and executable automation. Silva et al. use a platform-independent interaction vocabulary to relate behavioural scenarios to UI artefacts (T. R. Silva et al., 2017a, pp. 250–253), while BDDTS and IBDD make implicit or ambiguous scenario information explicit before test generation (Zameni et al., 2024, pp. 199–200). MobileBDD follows the same general principle through typed intermediate representations and AIM-based semantic resolution. In contrast to the formal MBT orientation of BDDTS/IBDD, the resulting representation is resolved directly into Maestro flows for two mobile platforms.

This separation is also related to earlier mobile test-generation approaches. Turner et al. require mappings from abstract interaction-sequence widgets to concrete Android widgets before generating Espresso tests (Turner & Bowen, 2024, p. 7), while Karlsson et al. show that a high-level behavioural model can be reused across Android and iOS (Karlsson et al., 2021, pp. 131, 135). The present findings provide fixed-corpus evidence for a comparable separation: one behavioural source is reused, while platform-specific selectors and operations are resolved through the AIM and generation recipes. Platform dependence is therefore relocated rather than eliminated.

Compiling the same 32 variants in ten repetitions demonstrates repeated compatibility of the frozen corpus, compiler, AIM, and target generator. It does not represent 320 distinct compiler inputs or establish generalisation to unseen scenarios. Nor does it show that the intermediate representations or the AIM are individually necessary; such a claim would require an alternative architecture or an ablation study. Unlike Slang’s evaluation of specification effort (Bünder & Kuchen, 2019a, pp. 1747–1748), RQ1 also provides no evidence that authoring MobileBDD specifications is more efficient than maintaining platform-specific tests.

Rendered-interface execution (RQ2). RQ2 extends the evaluation beyond compilation and target-artefact validation to execution against rendered Android and iOS interfaces. This differs from ViMoTest, which executes behaviour-oriented tests at the ViewModel level (Fuksa et al., 2023, p. 823). ViewModel-level testing reduces dependence on rendering frameworks, while the approach evaluated here includes the final UI, selector, driver, application-state, and device boundaries. The two approaches therefore validate different parts of the system rather than providing competing reliability measurements.

The two generated Android non-conformities illustrate why this distinction matters. Successful source analysis and generation did not ensure conformity with the declared terminal status during execution. This is consistent with prior cross-platform testing work reporting failures caused by platform-specific UI structures, dynamic screens, and element-location conflicts (Menegassi & Endo, 2020, pp. 33–34). It is also consistent with Turner et al.’s observation that generated Espresso tests can require target-specific integration and completion before execution (Turner & Bowen, 2024, p. 11). MobileBDD automates more of this transition through its configured pipeline, but execution remains dependent on the SUT, selectors, automation driver, timing, and platform environment.

Rendered-interface execution consequently increases the observable validation surface but also introduces failure sources absent from ViewModel-level evaluation. Because the retained evidence identifies the detection stage and failure class rather than the underlying cause, the Android observations cannot be attributed to the platform itself or used to rank Android and iOS reliability.

Generated/manual consistency (RQ3). RQ3 introduces a separate assessment boundary by comparing generated and manually authored executable flows. Test-oracle theory distinguishes execution from the decision about whether the observed behaviour is acceptable (Baresi & Young, 2001, pp. 4–5). Prior generated-UI-test approaches similarly encode assertions over observable interface states, while requiring manual input when the source model lacks a concrete expected

outcome (Turner & Bowen, 2024, pp. 9–10). The manual flows in this thesis provide a second executable realisation of the intended interaction paths, but not an independent ground truth.

None of the closest approaches provides a directly comparable generated/manual runtime measure. ViMoTest evaluates generated ViewModel-level tests, Slang evaluates specification effort, Silva et al. assess consistency across UI artefacts, and BDDTS/IBDD provide formal or illustrative evidence. RQ3 therefore adds descriptive evidence at the executable-outcome level. Its dependence on eligibility rules, retry behaviour, terminal observables, and same-author comparators prevents interpretation as semantic equivalence.

Overall positioning. The study combines three boundaries that prior approaches generally evaluate separately: semantic compilation from a shared behavioural source, execution against rendered Android and iOS interfaces, and comparison with manually authored executable flows. Its principal contribution is this staged end-to-end evaluation and the resulting ability to distinguish compilation, runtime conformity, and comparator consistency. The evidence remains restricted to one application, one fixed corpus, one configured virtual device per platform, and status-level outcomes. Broader conclusions require additional systems, independently authored artefacts, and trace- or assertion-level oracles.

6.3 Threats to Validity

The validity assessment focuses on threats arising from the evaluation design and distinguishes implemented controls from limitations that remain. The categories are selected according to the study method, following recommendations to analyse relevant threats in depth and explain why they could not be fully mitigated (Lago et al., 2024, pp. 322–323).

6.3.1 Construct Validity

Construct validity concerns whether the operational measures support the concepts represented by the research questions (Sjøberg & Bergersen, 2023, pp. 1374–1376). The staged criteria deliberately capture limited properties: S_1 measures source parsing and semantic analysis, S_2 measures target-contract validity, S_3 measures conformity between an eligible raw status and the declared expected status, and S_4 measures raw-status equality between eligible generated and manual reports. These measures do not establish complete scenario semantics, equality of interaction traces, assertion identity, command-sequence equivalence, or full behavioural correctness.

The four F4 variants demonstrate this limitation because their generated and manual terminal assertions concern different UI observables. Their exclusion was not specified before the campaign. Accordingly, the complete S_4 result remains primary. The restricted result is reported only in the post-hoc sensitivity analysis of terminal-observable alignment (Section 5.4). Similarly, the F7-02 source postcondition names a rating, a comment, and an activity, while both executable flows independently assert only comment visibility. The resulting observations support the narrower implemented assertion rather than every component expressed by the source postcondition.

Raw failure equality is especially limited for negative variants. Although interaction, infrastructure, and unknown failures are classified separately, an `ORACLE` classification does not establish that generated and manual flows failed at the same assertion or after the same interaction trace. It records that the retained diagnostic was classified as an oracle failure. RQ3 is therefore restricted to differential raw-status consistency, avoiding a broader claim of behavioural agreement as recommended for construct underrepresentation (Sjøberg & Bergersen, 2023, pp. 1376, 1387–1389).

The status classifier introduces an additional measurement boundary. A process with exit code zero can be classified as `PASSED` when no recognised terminal diagnostic is extracted, while failure eligibility depends on the diagnostic classifier. The classifier tests documented in Chapter 4 reduce implementation error at known boundaries but cannot establish that every future Maestro output is interpreted correctly.

The retry policy also forms part of the measured construct: final outcomes after at most one eligible retry are treated equally regardless of whether success was immediate or recovered. The first-attempt-only sensitivity analysis reports the effect of this decision (Section 5.5), but neither scoring scheme measures the intended interaction path directly.

Finally, the compiler, source scenarios, AIM, and manual comparators were authored by the same researcher. Freezing these artefacts before the retained campaign prevents adaptation to final outcomes but does not create independent ground truth. Shared interpretation may increase agreement between generated and manual flows. The comparators must therefore be understood as separately implemented same-author references.

6.3.2 Internal Validity

The study does not estimate a causal treatment effect. Its principal internal threat is development–evaluation leakage. Although separate corpus roots establish scenario disjointness, the evaluation corpus $\mathcal{E}$ was known while the grammar, compiler, and AIM were being developed. Structural similarity between corresponding development and evaluation cases also allowed implementation knowledge to transfer. The pre-campaign freeze prevents adjustment to retained outcomes but cannot make $\mathcal{E}$ prospectively unseen.

Android manual flows were executed from temporary normalised workspaces in which selected commands were modified to accommodate the campaign environment, while generated flows continued to execute unchanged. These deterministic transformations are documented in Section 4.5 and were applied consistently, but they create an asymmetry between generated and manual execution. The RQ3 comparison therefore applies to the executed, normalised manual artefacts rather than to the original files without runtime preparation.

Execution outcomes may also depend on emulator or simulator state, application state, timing, selectors, and execution order. Ten complete repetitions and retained attempt-level evidence allow run-to-run variation under the fixed configuration to be observed, but they do not provide evidence about variation across devices or configurations. Because only one configured virtual device represented each platform, the two Android non-successes cannot be attributed to Android

independently of device, application, selector, or ordering effects.

6.3.3 Conclusion Validity

The repetitions do not enlarge the evaluation corpus or create statistically independent cases. For RQ1, the same 32 fixed variants were compiled repeatedly; for RQ2 and RQ3, repetitions describe execution variation for those variants under the retained configuration. The reported rates are therefore descriptive and do not estimate population parameters.

The final formulations of RQ2 and RQ3 were narrowed after campaign outcomes were available. Retaining every scheduled observation and using fixed denominators limits outcome-driven exclusion, but the analytical framing remains retrospective rather than confirmatory.

H1 and H2 apply an author-selected 90% threshold that was not empirically calibrated. The threshold sensitivity analysis shows how the classifications change under stricter cut-offs, while the first-attempt and F4 analyses expose dependence on other analytical decisions. These labels add no statistical confidence beyond the observed counts. Ten repetitions may also fail to reveal uncommon runtime failures.

6.3.4 External Validity

The corpus was purposively constructed rather than sampled from a defined population. Its 32 evaluation variants represent 16 interaction paths because each positive/negative pair differs only in its postcondition. The variants therefore increase outcome polarity without increasing interaction-path breadth. The interaction categories and dominant-category assignments were produced by one researcher without an independent coding check, and their semantic completeness, realistic frequency, and difficulty distribution remain unmeasured.

The evaluation further covers one application, one compiler version, one automation framework, and one configured Android and iOS target. Generalisation to other applications, domains, platform versions, devices, interaction patterns, or automation frameworks requires independently constructed corpora and additional execution configurations.

6.3.5 Reliability and Reproducibility

The frozen artefacts, scenario identifiers, recorded configurations, repeated execution protocol, and retained run-level reports support inspection and repetition under the documented configuration. They do not establish reproduction by independent researchers. All executions and artefact classifications were performed by the same researcher, and neither the corpus nor the manual oracles received an independent audit. Independent replication is therefore required to evaluate reproducibility beyond the reported environment (Lago et al., 2024, pp. 321–323).

7 Summary & Outlook

This chapter closes the thesis by relating its principal contribution to the limits of the reported evidence and identifying the empirical work needed to assess transferability, runtime reliability, and practical value.

7.1 Findings and Contributions

The central contribution of this thesis is an implemented compiler-based workflow that transforms controlled MobileBDD scenarios into executable Maestro flows. MobileBDD retains the familiar structure of BDD scenarios while restricting executable steps to forms that can be parsed and validated systematically. Application-specific interaction knowledge is localised in the AIM, which defines targets, capabilities, selectors, acquisition strategies, and generation recipes. The implementation realises this separation through an ANTLR parser, a typed Kotlin AST, semantic resolution, and a resolved interaction plan. It validates scenario identifiers, vocabulary, target capabilities, arguments, and screen relations before generating Maestro YAML. In the evaluated configuration, all generated flows were validated and executed without per-flow manual editing.

The empirical contribution is a staged evaluation covering source analysis, target synthesis, terminal-status conformity, and differential raw-status consistency. The evaluation separately examines the transformation boundaries addressed by RQ1–RQ3. It covers the 32 variants in the evaluation corpus $\mathcal{E}$, representing 16 behavioural paths across eight interaction categories, with each path represented by a positive and a negative variant. This structure identifies where a failure was detected in the processing sequence, although it does not by itself establish the failure’s cause.

Answer to RQ1. The compiler achieved a 100% compilation success rate for the evaluation corpus $\mathcal{E}$. Because $\mathcal{E}$ was known and refined during development, the result demonstrates compatibility with this corpus only.

Answer to RQ2. Expected terminal-status conformity was 99.375% on Android and 100% on iOS. These rates apply the S_3 eligibility and expected-status requirements but do not establish assertion completeness or comparative platform reliability (Section 6.1).

Answer to RQ3. Using scheduled pairs as the fixed denominator, differential raw-status consistency was 99.375% on Android and 100% on iOS. S_4 compares executable statuses only; equal statuses do not establish semantic, assertion-level, or trace-level equivalence (Section 6.1).

Overall, the study demonstrates bounded feasibility for the evaluated setup. Section 6.3 details the limits arising from the corpus, repeated observations, and post-measurement RQ framing.

7.2 Limitations

Scope of the abstraction. The separation between MobileBDD, the AIM, and target generation was implemented for one application and one automation backend. Transfer through an AIM-only change and reuse of the resolved interaction plan with another backend remain untested. Related ontology-based approaches require extensions for additional platforms or interaction techniques (T. R. Silva et al., 2017b, pp. 513, 537). Kamalakar et al. (2013, pp. 238–239) identify manually implemented glue code as a bottleneck because scenario-to-step binding stops short of full test generation. The pipeline relocates this application-specific effort into the AIM; it does not remove the need to encode and maintain supported interactions. The evaluation did not establish whether this effort is lower than maintaining conventional step definitions.

Scope of the scenario language. The corpus was written to fit the supported vocabulary, leaving unanticipated interaction paths outside the compilation result. Writing scenarios at the interaction level may increase verbosity because individual UI actions must be specified, while restricting the vocabulary trades flexibility for reduced ambiguity and incompleteness (T. R. Silva et al., 2017a, pp. 256–257). Scenario readability, authoring effort, and maintainability were not measured.

Study and evidence scope. The purposively constructed corpus provides neither representative coverage of mobile-app behaviour nor a basis for statistical generalisation. Although $\mathcal{D}$ and $\mathcal{E}$ are directory- and scenario-disjoint, $\mathcal{E}$ was known during development. The runtime evidence is limited to terminal-status criteria: S_3 requires an eligible generated status that matches the scenario’s declared expectation, while S_4 requires eligible generated and manual statuses that are equal. Ineligible observations remain in the fixed denominators as non-successes. These criteria do not assess complete interaction traces, assertion identity, or full scenario semantics. Section 6.3 analyses these limitations in detail.

Execution scope. All runs used one macOS host, one Android emulator, and one iOS simulator, with both platform environments configured for English. Ten repetitions therefore assess repeatability only within this setup. The four internal retries and two final Android non-successes show remaining runtime variability, but the retained classifications identify detection order rather than causal origin. Execution time, resource use, and long-term flakiness were not measured.

7.3 Outlook

Independent transfer and backend study. Future work should evaluate the compiler on applications from other domains using independently authored scenarios, AIMs, and reference flows. The evaluation corpus should use independently constructed positive and negative interaction paths, with the final set fixed before measurement. The main transfer criterion is whether a new application can be supported through its AIM without changing the grammar or compiler. A second automation backend would test the other side of the architecture: whether the resolved interaction plan is independent of Maestro in practice.

Runtime reliability study. Future campaigns should report first-attempt success separately from the final logical-flow outcome produced by the retry policy. The two generated Android non-successes and the four internal retries from this evaluation should form a named regression set with attempt-level reports, screenshots, and failure classifications. Controlled runs should vary application reset state, device state, execution order, and relevant timing settings to determine which conditions reproduce each failure. Platform and scenario order should be randomised or counterbalanced.

The execution matrix should include several virtual and physical devices, operating-system versions, screen sizes, locales, and host configurations. Display scaling, increased font size, dark mode, and accessibility services should be included where they can alter layout or selector exposure. Longer campaigns should report execution time, first-attempt failure frequency, recovery frequency, final-outcome success, and complete-bundle success. Counts should remain separated by scenario and configuration; repeated observations of the same scenario should not be presented as independent cases.

Stronger oracle evidence. The implemented S_4 comparison should be extended beyond eligible terminal statuses. Generated and manual pairs should also compare executed actions, intermediate checkpoints, assertion signatures, failure locations, and failure classes. Reports classified as interaction or infrastructure failures should remain separate from oracle disagreements so that missing comparisons do not appear as behavioural agreement. Independently authored or externally reviewed reference flows would reduce shared-author bias and provide a stronger basis for distinguishing translation errors from environmental failures.

Practical assessment. A comparative study should evaluate MobileBDD and its AIM against direct Maestro authoring. Relevant measures include scenario and AIM authoring time, maintenance effort after interface changes, defect localisation, and comprehension by technical and non-technical participants. Such a study would show whether moving selectors and interaction recipes into the AIM reduces total maintenance effort or only relocates it. Language extensions, multilingual support, and editor assistance should follow observed authoring problems and unsupported interaction requirements.

Overall, this thesis demonstrates that a controlled BDD language combined with an AIM can generate executable Android and iOS UI tests for the evaluated corpus. Establishing the approach's broader practical value now requires independent transfer studies and comparison beyond terminal execution statuses.

References

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, & tools* (2nd ed., Pearson international ed.). Pearson Addison-Wesley. https://dpvipracollege.ac.in/wp-content/uploads/2023/01/Alfred-V.-Aho-Monica-S.-Lam-Ravi-Sethi-Jeffrey-D.-Ullman-Compilers-Principles-Techniques-and-Tools-Pearson_Addison-Wesley-2007.pdf
- Ali, Z. (2019). *Mobile Application Testing Using Behavior-Driven Development Based Specifications for Android Platform* [Doctoral thesis, Graz University of Technology]. <https://doi.org/10.3217/m5tr1-9dr76>
- Assem, M., Keshk, A., Ismail, N., & Nassar, H. (2007). Specification-driven automated testing of Java Swing GUIs using XML. *2007 ITI 5th International Conference on Information and Communications Technology*, 84–88. <https://doi.org/10.1109/ITICT.2007.4475622>
- Baresi, L., & Young, M. (2001, April 8). *Test Oracles* (CIS-TR-01-02). University of Oregon, Department of Computer Science; Politecnico di Milano. https://www.researchgate.net/publication/2392558_Test_Oracles
- Beck, K. (2002, November 8). *Test-driven development: By example*. Addison-Wesley Professional.
- Binamungu, L. P., Embury, S. M., & Konstantinou, N. (2018). Maintaining behaviour driven development specifications: Challenges and opportunities. *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 175–184. <https://doi.org/10.1109/SANER.2018.8330207>
- Biørn-Hansen, A., Grønli, T.-M., & Ghinea, G. (2019). A Survey and Taxonomy of Core Concepts and Research Challenges in Cross-Platform Mobile Development. *ACM Computing Surveys*, 51(5), 1–34. <https://doi.org/10.1145/3241739>
- Brambilla, M., Cabot, J., & Wimmer, M. (2017). *Model-Driven Software Engineering in Practice* (2nd ed.). Morgan & Claypool. <https://doi.org/10.1007/978-3-031-02549-5>
- Bünder, H., & Kuchen, H. (2019a). A model-driven approach for behavior-driven GUI testing. *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 1742–1751. <https://doi.org/10.1145/3297280.3297450>
- Bünder, H., & Kuchen, H. (2019b). Towards behavior-driven graphical user interface testing. *ACM SIGAPP Applied Computing Review*, 19(2), 5–17. <https://doi.org/10.1145/3357385.3357386>
- Bussenot, R., Leblanc, H., & Percebois, C. (2018). Orchestration of Domain Specific Test Languages with a Behavior Driven Development approach. *2018 13th Annual Conference on System of Systems Engineering (SoSE)*, 431–437. <https://doi.org/10.1109/SYBOSE.2018.8428788>
- Cohn, M. (2004). *User Stories Applied For Agile Software Development*. Addison Wesley.
- Deka, B., Huang, Z., & Kumar, R. (2016). ERICA: Interaction Mining Mobile Apps. *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, 767–776. <https://doi.org/10.1145/2984511.2984581>
- Eke, N., Salihu, I., Usman, A., Ibrahim, R., & Mshelia, Y. (2025). A survey of automated testing techniques for android-based mobile applications. *Nigerian Journal of Technology*, 44(2), 311–337. <https://doi.org/10.4314/njt.v44i2.15>

- Eurostat. (2025a). *Individuals - devices used to access the internet* (Eurostat). Eurostat. https://doi.org/10.2908/ISOC_CI_DEV_I
- Eurostat. (2025b). *Individuals - Frequency of Internet Use* (Eurostat). Eurostat. https://doi.org/10.2908/ISOC_CI_IFP_FU
- Fichte, L., & York, C. (2025). How do smartphone users access the internet? An exploratory analysis of mobile web browser use. *Mobile Media & Communication*, 13(1), 151–170. <https://doi.org/10.1177/20501579241274781>
- Fischer, C. N., Cytron, R. K., & Le Blanc, R. J. J. (2009, November 7). *Crafting a compiler*. Addison-Wesley. Retrieved May 14, 2026, from <https://pages.cs.wisc.edu/~ansari/cc.pdf>
- Frattaroli, V., Le Goar, O., & Philippot, O. (2023). Ecological Impact of Native Versus Cross-Platform Mobile Apps: A Preliminary Study. *2023 38th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 3–8. <https://doi.org/10.1109/ASEW60602.2023.00005>
- Fuksa, M., Speth, S., & Becker, S. (2023). Applicability of the ViMoTest Approach for Automated GUI Testing: A Field Study. *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 821–830. <https://doi.org/10.1109/MODELS-C59198.2023.00131>
- Fuksa, M., Speth, S., & Becker, S. (2025). ViMoTest: A Tool to Specify ViewModel-Based GUI Test Scenarios Using Projectional Editing. *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 752–756. <https://doi.org/10.1109/ICST62969.2025.10988969>
- Georgescu, C., Olsthoorn, M., Derakhshanfar, P., Akhin, M., & Panichella, A. (2024). Evolutionary Generative Fuzzing for Differential Testing of the Kotlin Compiler. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 197–207. <https://doi.org/10.1145/3663529.3663864>
- Heitkötter, H., Hanschke, S., & Majchrzak, T. A. (2013). Evaluating Cross-Platform Development Approaches for Mobile Applications. In J. Cordeiro & K.-H. Krempels (Eds.), *Web Information Systems and Technologies* (pp. 120–138, Vol. 140). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-36608-6_8
- Hölldobler, K., Rumpe, B., & Wortmann, A. (2018). Software language engineering in the large: Towards composing and deriving languages. *Computer Languages, Systems & Structures*, 54, 386–405. <https://doi.org/10.1016/j.cl.2018.08.002>
- Holmes, J., Groce, A., Pinto, J., Mittal, P., Azimi, P., Kellar, K., & O’Brien, J. (2018). TSTL: The template scripting testing language. *International Journal on Software Tools for Technology Transfer*, 20(1), 57–78. <https://doi.org/10.1007/s10009-016-0445-y>
- Hudak, P. (1998). Domain Specific Languages. In *Handbook of Programming Languages, Volume III: Little Languages and Tools* (Vol. 3). Macmillan. <https://cs448h.stanford.edu/DSEL-Little.pdf>
- Inukollu, V. N., Keshamon, D. D., Kang, T., & Inukollu, M. (2014). Factors Influencing Quality of Mobile Apps: Role of Mobile App Development Life Cycle. *International Journal of Software Engineering & Applications*, 5(5), 15–34. <https://doi.org/10.5121/ijsea.2014.5502>

- Irshad, M., Börstler, J., & Petersen, K. (2022). Supporting refactoring of BDD specifications—An empirical study. *Information and Software Technology*, 141, 106717. <https://doi.org/10.1016/j.infsof.2021.106717>
- Jangassiyev, R., Umarova, Z., Ussenova, A., Makhanova, Z., Zhumatayev, N., Amirov, M., & Koishibekova, G. (2024). Comparative analysis of cross-platform development methodologies: A comprehensive study. *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, 23(1), 108. <https://doi.org/10.12928/telkomnika.v23i1.26331>
- Ji, R., Zhu, T., Zhu, X., Chen, C., Pan, M., & Zhang, T. (2023). Vision-Based Widget Mapping for Test Migration Across Mobile Platforms: Are We There Yet? *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 1416–1428. <https://doi.org/10.1109/ASE56229.2023.00068>
- Jongeling, R., Strøm, N. J., Torp, L. P., Kitchen, M., & Carlson, J. (2025). Adopting the C4 model for lightweight architecture modeling – An experience report. http://www.es.mdu.se/pdf_publications/7230.pdf
- Jošt, G., & Taneski, V. (2025). State-of-the-Art Cross-Platform Mobile Application Development Frameworks: A Comparative Study of Market and Developer Trends. *Informatics*, 12(2), 45. <https://doi.org/10.3390/informatics12020045>
- Kamalakar, S., Edwards, S. H., & Dao, T. M. (2013). Automatically Generating Tests from Natural Language Descriptions of Software Behavior: *Proceedings of the 8th International Conference on Evaluation of Novel Approaches to Software Engineering*, 238–245. <https://doi.org/10.5220/0004566002380245>
- Karlsson, S., Causevic, A., Sundmark, D., & Larsson, M. (2021). Model-based Automated Testing of Mobile Applications: An Industrial Case Study. *2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 130–137. <https://doi.org/10.1109/ICSTW52544.2021.00033>
- Khorram, F., Bousse, E., Garmendia, A., Mottu, J.-M., Sunyé, G., & Wimmer, M. (2024). A language-parametric test coverage framework for executable domain-specific languages. *Journal of Systems and Software*, 211, 111977. <https://doi.org/10.1016/j.jss.2024.111977>
- Kong, P., Li, L., Gao, J., Liu, K., Bissyande, T. F., & Klein, J. (2019). Automated Testing of Android Apps: A Systematic Literature Review. *IEEE Transactions on Reliability*, 68(1), 45–66. <https://doi.org/10.1109/TR.2018.2865733>
- Lago, P., Runeson, P., Song, Q., & Verdecchia, R. (2024). Threats to Validity in Software Engineering – hypocritical paper section or essential analysis? *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 314–324. <https://doi.org/10.1145/3674805.3686691>
- Lawrence, R., & Rayner, P. (2019, June 21). *Behavior-Driven-Development with Cucumber*. Addison-Wesley Professional.
- Le-Khanh, T., Pham-Hoang, A., Hoang-Van, Q., Bui-The, C., Cao-Thi-Minh, T., & Hung, P. N. (2025). An Efficient End-to-End User Interface Testing Method for Web Applications. *Comp. Science & Com. Eng.*, 41(1), 97–111. <https://doi.org/10.25073/2588-1086/vnucsce.3629>
- Lima, Y. A., Rodrigues, E. D. M., Basso, F. P., & Oliveira, R. A. P. (2021). Teasy: A domain-specific language to reduce time and facilitate the creation of tests in web applications.

- Anais Do III Workshop Em Modelagem e Simulação de Sistemas Intensivos Em Software (MSSiS 2021)*, 40–49. <https://doi.org/10.5753/mssis.2021.17258>
- Lin, J.-W., Salehnamadi, N., & Malek, S. (2021). Test automation in open-source Android apps: A large-scale empirical study. *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 1078–1089. <https://doi.org/10.1145/3324884.3416623>
- Lv, Z., Peng, C., Zhang, Z., Su, T., Liu, K., & Yang, P. (2022). Fastbot2: Reusable Automated Model-based GUI Testing for Android Enhanced by Reinforcement Learning. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–5. <https://doi.org/10.1145/3551349.3559505>
- Majchrzak, T., Biørn-Hansen, A., & Grønli, T.-M. (2017). Comprehensive Analysis of Innovative Cross-Platform App Development Frameworks. *Proceedings of the 50th Hawaii International Conference on System Sciences*, 6162–6171. <https://doi.org/10.24251/HICSS.2017.745>
- Martinez, M., & Lecomte, S. (2017). Towards the Quality Improvement of Cross-Platform Mobile Applications. *2017 IEEE/ACM 4th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 184–188. <https://doi.org/10.1109/MOBILESoft.2017.30>
- Menegassi, A. A., & Endo, A. T. (2020). Automated tests for cross-platform mobile apps in multiple configurations. *IET Software*, 14(1), 27–38. <https://doi.org/10.1049/iet-sen.2018.5445>
- Muccini, H., Di Francesco, A., & Esposito, P. (2012). Software testing of mobile applications: Challenges and future research directions. *2012 7th International Workshop on Automation of Software Test (AST)*, 29–35. <https://doi.org/10.1109/IWAST.2012.6228987>
- Nguyen, M., Wrede, S., & Hochgeschwender, N. (2025). Automated Behaviour-Driven Acceptance Testing of Robotic Systems. *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1327–1333. <https://doi.org/10.1109/IROS60139.2025.11246121>
- Nie, L., Said, K. S., Ma, L., Zheng, Y., & Zhao, Y. (2023). A systematic mapping study for graphical user interface testing on mobile apps. *IET Software*, 17(3), 249–267. <https://doi.org/10.1049/sfw2.12123>
- Nitze, A., & Schmietendorf, A. (2015). A survey on mobile users' software quality perceptions and expectations. *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 1–2. <https://doi.org/10.1109/ICSTW.2015.7107417>
- Oliveira, G., Marczak, S., & Moralles, C. (2019). How to Evaluate BDD Scenarios' Quality? *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*, 481–490. <https://doi.org/10.1145/3350768.3351301>
- Parr, T. (2010). *Language Implementation Patterns* (1st ed.). The Pragmatic Programmers.
- Parr, T. (2013, January). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
- Rodrigues Da Silva, A. (2015). Model-driven engineering: A survey supported by the unified conceptual model. *Computer Languages, Systems & Structures*, 43, 139–155. <https://doi.org/10.1016/j.cl.2015.06.001>

- Romdhana, A., Merlo, A., Ceccato, M., & Tonella, P. (2022). Deep Reinforcement Learning for Black-Box Testing of Android Apps. *ACM Transactions on Software Engineering and Methodology*, 31(4), 1–29. <https://doi.org/10.1145/3502868>
- Rossouw, C., & Fischer, B. (2024). Grammar-based test suite construction using coverage-directed algorithms over LR-graphs. *Journal of Systems and Software*, 214, 112068. <https://doi.org/10.1016/j.jss.2024.112068>
- Santiago, D., Cando, A., Mack, C., Nunez, G., Thomas, T., & King, T. M. (2013). Towards Domain-Specific Testing Languages for Software-as-a-Service. *Proceedings of the 2nd International Workshop on Model-Driven Engineering for High Performance and Cloud Computing*; 1118, 43–52. <https://ceur-ws.org/Vol-1118/06-paper.pdf>
- Silva, T., Winckler, M., & Trættemberg, H. (2019). Extending Behavior-Driven Development for Assessing User Interface Design Artifacts. *SEKE 2019: Proceedings of the 31st International Conference on Software Engineering and Knowledge Engineering*, 485–488. <https://doi.org/10.18293/SEKE2019-054>
- Silva, T. R. (2022). Towards a Domain-Specific Language to Specify Interaction Scenarios for Web-Based Graphical User Interfaces. *Companion of the 2022 ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, 48–53. <https://doi.org/10.1145/3531706.3536463>
- Silva, T. R., & Fitzgerald, B. (2021). Empirical Findings on BDD Story Parsing to Support Consistency Assurance between Requirements and Artifacts. *Evaluation and Assessment in Software Engineering*, 266–271. <https://doi.org/10.1145/3463274.3463807>
- Silva, T. R., Hak, J.-L., & Winckler, M. (2017a). A Behavior-Based Ontology for Supporting Automated Assessment of Interactive Systems. *2017 IEEE 11th International Conference on Semantic Computing (ICSC)*, 250–257. <https://doi.org/10.1109/ICSC.2017.73>
- Silva, T. R., Hak, J.-L., & Winckler, M. (2017b). A Formal Ontology for Describing Interactive Behaviors and Supporting Automated Testing on User Interfaces. *International Journal of Semantic Computing*, 11(04), 513–539. <https://doi.org/10.1142/S1793351X17400219>
- Silva, T. R., & Winckler, M. (2017). A Scenario-Based Approach for Checking Consistency in User Interface Design Artifacts. *Proceedings of the XVI Brazilian Symposium on Human Factors in Computing Systems*, 1–10. <https://doi.org/10.1145/3160504.3160506>
- Sjøberg, D. I. K., & Bergersen, G. R. (2023). Construct Validity in Software Engineering. *IEEE Transactions on Software Engineering*, 49(3), 1374–1396. <https://doi.org/10.1109/TSE.2022.3176725>
- Smart, J. F. (2015). *BDD in action: Behavior-driven development for the whole software lifecycle*. Manning.
- Soeken, M., Wille, R., & Drechsler, R. (2012). Assisted behavior driven development using natural language processing. In C. A. Furia & S. Nanz (Eds.), *Objects, models, components, patterns* (pp. 269–287). Springer Berlin Heidelberg. https://dl.acm.org/doi/10.1007/978-3-642-30561-0_19
- Solis, C., & Wang, X. (2011). A Study of the Characteristics of Behaviour Driven Development. *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications*, 383–387. <https://doi.org/10.1109/SEAA.2011.76>
- Song, Z., Mansur, S. M. H., Rathnasuriya, R., Fatima, Y., Yang, W., Moran, K., & Lam, W. (2025). Can You Mimic Me? Exploring the Use of Android Record & Replay Tools in Debugging.

- 2025 IEEE/ACM 12th International Conference on Mobile Software Engineering and Systems (MOBILESoft), 32–43. <https://doi.org/10.1109/MOBILESoft66462.2025.00011>
- Stanić, N., & Ćirković, S. (2024). Analysis of Approaches to Developing Kotlin Multiplatform Applications and Their Impact on Software Engineering. *10th International Scientific Conference Technics, Informatic, and Education*, 53–59. <https://doi.org/10.46793/TIE24.053S>
- Statista. (2023). *Mobile Betriebssysteme*. Hamburg. Retrieved August 28, 2026, from <https://de.statista.com/statistik/studie/id/12549/dokument/mobile-betriebssysteme/>
- Talebipour, S., Zhao, Y., Dojcilovic, L., Li, C., & Medvidovic, N. (2021). UI Test Migration Across Mobile Platforms. *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 756–767. <https://doi.org/10.1109/ASE51524.2021.9678643>
- Terhorst-North, D. (2006, September 20). *Introducing BDD*. Dan North & Associates Limited. Retrieved January 18, 2026, from <https://dannorth.net/blog/introducing-bdd/>
- Turner, J., & Bowen, J. (2024). Automating Test Scripts for Android UI Testing. *Computer Science & Information Technology (CS & IT)*, 14(7), 1–13. <https://doi.org/10.5121/csit.2024.140701>
- Umar, M., Hussain, I., Mahmood, T., Mirza, H. T., & Faisal, C. M. N. (2024). Design Strategies to Minimize Mobile Usability Issues in Navigation Design Patterns. *Information*, 15(11), 732. <https://doi.org/10.3390/info15110732>
- Yu, S., Fang, C., Du, M., Ding, Z., Chen, Z., & Su, Z. (2024). Practical, Automated Scenario-Based Mobile App Testing. *IEEE Transactions on Software Engineering*, 50(7), 1949–1966. <https://doi.org/10.1109/TSE.2024.3414672>
- Zameni, T., Van Den Bos, P., Rensink, A., & Tretmans, J. (2024). An Intermediate Language to Integrate Behavior-Driven Development Scenarios and Model-Based Testing. *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*, 199–206. <https://doi.org/10.1109/SANER-C62648.2024.00033>
- Zameni, T., Van Den Bos, P., Tretmans, J., Foederer, J., & Rensink, A. (2023). From BDD Scenarios to Test Case Generation. *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 36–44. <https://doi.org/10.1109/ICSTW58534.2023.00019>

Appendices

A Use of Generative AI

Generative AI tools were used as supporting instruments during the development and writing of this thesis. The author remained responsible for the research design, implementation, evaluation, interpretation of results, selection of sources, and final wording. AI output was treated as a suggestion and was reviewed before use.

A.1 Tools and Purposes

OpenAI Codex Supported technical reviews and debugging related to Kotlin, ANTLR, Maestro, Gradle, utility scripts, and $\text{\LaTeX}$ listings. Proposed changes were checked by inspecting the resulting code, compiling the project, and running the relevant automated tests.

ChatGPT Supported brainstorming, identification of counterarguments, language review of author-written passages, and diagnosis of selected $\text{\LaTeX}$ problems. It was also used to suggest literature-search terms and to summarise selected material for initial orientation. AI-generated summaries were not treated as scholarly sources.

Google Gemini Provided feedback on the coherence and structure of selected draft passages. Its suggestions were assessed and revised by the author and were not used as scholarly evidence.

A.2 Verification and Limitations

AI-generated information was not treated as a citable source. References, quotations, technical claims, and research findings were verified independently. Generated or modified code was accepted only after review and successful execution of the relevant checks. No AI tool conducted the experiments, interpreted the measurements independently, or determined the conclusions.

A.3 Representative Prompts

The following abbreviated prompts represent the interactions that most influenced the work:

- “Review this Kotlin/ANTLR/Maestro implementation for concrete correctness problems. Explain each issue and avoid rewriting unrelated code. Provide further resources which could help to understand the problem. [CODE AND CONTEXT]”
- “Review this passage for gaps in reasoning, unsupported claims, use of correct language and repetition. Do not introduce new facts or references. Don’t come up with custom ideas. [PASSAGE]”
- “Explain the cause of this $\text{\LaTeX}$ overfull $\text{\hbox}$ warning and propose the smallest correction that preserves the content. [WARNING AND SOURCE]”

- “Review the hand-in requirements and this repository’s existing scripts. Implement the smallest reproducible workflow that collects the referenced online resources, creates a catalogue, and packages the hand-in files without stale archive contents. [REQUIREMENTS AND REPOSITORY]”
- “Review these $\text{\LaTeX}$ listings and their shared style definitions. Correct the mixed syntax highlighting with the smallest reusable change without altering the listing contents. [LISTINGS AND STYLE DEFINITIONS]”
- “Recalculate the reported evaluation counts and rates from the retained run bundles. Identify discrepancies and provide the corresponding evidence paths without inventing missing evidence. [RUN BUNDLES AND RESULTS]”

B Git Repositories

The literature and archived online resources used in this thesis are bundled in `handin.zip` in the thesis repository.

Repository	URL
Thesis repository	https://gitlab.ct.fh-salzburg.ac.at/shde-masterthesis/master-thesis-mmt-denis-schuele
Compiler repository	https://gitlab.ct.fh-salzburg.ac.at/shde-masterthesis/bbd-to-maestro
System under test	https://gitlab.ct.fh-salzburg.ac.at/shde-masterthesis/sut

Table 17: Git repositories used in this thesis.

C Fixed Execution Environment

Table 18: Fixed execution environment used for expected terminal-status conformity and differential raw-status consistency.

Aspect	Fixed value
SUT	SalzburgCard
Evaluated revision	branch feat/15-poi-review, commit cf607d8
Evaluated compiler/pipeline revision	commit ef4b5a3e7dfaf093f9eedc6e685ba41a0be7b68f, introduced 27 July 2026
Evidence-bundle revision	commit 91744795269a78f90bc7e0c62d04df8c3681deb1, recorded 28 July 2026
JDK	Amazon Corretto OpenJDK 22.0.2
Kotlin	2.2.21
ANTLR	4.13.0
Gradle	9.3.1, invoked through the Gradle wrapper
Maestro CLI	2.1.0
Canonical build and run command	make pipeline-both
Android execution target	emulator Medium_Phone_API_35
iOS execution target	simulator iPhone 16 Pro, iOS 18.1
Orientation	portrait
Theme	light
Locale	Android: en-US; iOS: primary language en-US, locale en_US@rg=atzzzz (English, Austria region)
Font/display scale	Android: font scale 1.0, 1080 × 2400 px at 420 dpi; iOS: Dynamic Type UIContentSizeCategoryLarge, 1206 × 2622 px at scale 3 (402 × 874 pt)
Permission state	Android: application data and runtime-permission state cleared with adb shell pm clear before every flow; android.permission.INTERNET granted as a normal install-time permission; no runtime permissions requested. iOS: application data cleared by launchApp with clearState: true; the application requested no protected-resource permissions, so no relevant grants or denials were required.
Application state	reset to deterministic baseline before each complete cross-platform pipeline run
Data dependency	bundled local demo dataset; no external backend required
Host system	MacBook Pro, 14-inch (November 2024)
Processor	Apple M4 Max

Continued on next page

Table 18: Fixed execution environment used for expected terminal-status conformity and differential raw-status consistency (continued).

Aspect	Fixed value
Memory	36 GB
Host operating system	macOS Tahoe 26.5
Power state	Connected to external power
Host activity	System sleep disabled during evaluation
Network state	Wi-Fi enabled throughout evaluation
Repetitions	Ten complete sequential cross-platform S_0 – S_4 runs; Android and iOS included in each run

D Supplementary Implementation Listings

This appendix collects expanded implementation excerpts, evidence examples, and operational commands that support Chapter 4. They are retained for technical traceability and reproduction without interrupting the chapter’s description of the principal compiler representations and transformations.

D.1 Typed Kotlin AST

The excerpt below is derived from `modules/language-core/src/main/kotlin/at/schuele/mobilebdd/language/domain/ast/SpecAst.kt` at implementation revision 91744795269a78f90bc7e0c62d04df8c3681deb1.

Listing 8: Selected implemented Kotlin AST types

```

1 data class ScenarioNode(
2     val tags: List<TagNode>,
3     val title: String,
4     val description: List<DescriptionLineNode> = emptyList(),
5     val items: List<ScenarioItemNode>,
6     override val source: SourceLocation
7 ) : FeatureItemNode
8
9 data class StepNode(
10     val keyword: StepKeyword,
11     val body: StepBody,
12     override val source: SourceLocation
13 ) : ScenarioItemNode
14
15 sealed interface StepBody : AstNode
16
17 sealed interface TypedInteractionBody : StepBody {
18     val kind: TypedInteractionKind
19     val headPhrase: String
20     val value: StepArgumentNode?
21     val objects: List<StepObjectNode>
22     val raw: RawStepBody
23 }
24
25 data class TypedActionBody(
26     override val headPhrase: String,
27     override val value: StepArgumentNode?,
28     override val objects: List<StepObjectNode>,
29     override val raw: RawStepBody,
30     override val source: SourceLocation
31 ) : TypedInteractionBody {

```

```

32     override val kind: TypedInteractionKind =
        TypedInteractionKind.ACTION
33 }
34
35 data class TypedAssertionBody(
36     override val headPhrase: String,
37     override val value: StepArgumentNode?,
38     override val objects: List<StepObjectNode>,
39     val assertionPhrase: String,
40     override val raw: RawStepBody,
41     override val source: SourceLocation
42 ) : TypedInteractionBody {
43     override val kind: TypedInteractionKind =
        TypedInteractionKind.ASSERTION
44 }

```

D.2 AIM Configuration

Listing 9: Representative AIM recipe, language-alias, and operation declarations

```

1 recipes:
2   actions:
3     paste_into_field:
4       steps: [tap_action_selector, set_clipboard_value,
5             paste_text]
6     states:
7       visible_target: [assert_visible_target]
8
9 language:
10  actions:
11    save: ["favorites", "bookmarks"]
12  assertions:
13    displayed: ["can be seen"]
14
15 interactions:
16  operations:
17    return:
18      action: open
19      state: returned

```

D.2.1 Semantic Matching and Configuration Precedence

The semantic resolver applies the following matching rules:

1. Operation and assertion aliases are matched case-insensitively at word boundaries and

in decreasing order of phrase length. Assertion expressions are checked before action expressions.

2. A target must match the required semantic kind and one of its declared terms after case and whitespace normalisation. Stable identifiers are not implicit source-language aliases.
3. No match produces an unresolved-target error; several matches produce an ambiguity error. Declaration order is not a tie-breaker.
4. The matched target must explicitly support the capability selected by the interaction profile and provide the required argument and platform bindings.
5. Target screens and capability source restrictions are checked against the resolver's current screen. A successful transition updates that screen from the capability destination.

Imported documents are deep-merged from left to right, after which the importing document takes precedence. Templates are applied in declaration order, and the concrete target overrides their values. Explicit capability configuration overrides inherited configuration; profile recipes supply defaults only when a capability omits its recipe. During generation, a capability-specific selector takes precedence over the target default, the selected Android or iOS branch is applied, placeholders are interpolated from the resolved value, and recipe primitives retain their declared order.

D.3 Pipeline Types, Commands, and Evidence

Listing 10: Result variants and pipeline outcomes of the S1 gate

```

1 sealed interface S1GateResult {
2     val inputPath: String
3     data class Passed(
4         override val inputPath: String,
5         val document: SpecDocument,
6         val plan: ResolvedInteractionPlan
7     ) : S1GateResult {
8         val outcome = PipelineOutcome.PASSED
9     }
10    data class SyntaxFailed(
11        override val inputPath: String,
12        val diagnostics: List<SpecDiagnostic>
13    ) : S1GateResult {
14        val outcome = PipelineOutcome.FAILED
15    }
16    data class SemanticFailed(
17        override val inputPath: String,
18        val document: SpecDocument,
19        val errors: List<SemanticError>
20    ) : S1GateResult {
21        val outcome = PipelineOutcome.FAILED

```

```

22     }
23     data class InputReadError(
24         override val inputPath: String,
25         val reason: String
26     ) : S1GateResult {
27         val outcome = PipelineOutcome.ERROR
28     }
29 }

```

Table 19: Representative syntax and semantic diagnostics produced by S_1 .

Field	Syntax-stage failure	Semantic failure
Outcome	SyntaxFailed	SemanticFailed
Error code	MOBILEBDD_STEP_KEYWORD	UNRESOLVED_TARGET
Message	unknown step keyword 'Giv'. Did you mean 'Given'?	No AIM target matches 'missing activity' as ELEMENT.
Location	syntax-error.feature, line 8, column 1	semantic-error.feature, line 9, column 6

Listing 11: Generated platform-specific Maestro branch

```

1 - runFlow:
2     when:
3         platform: Android
4     commands:
5         - tapOn:
6             id: "android_place_row"
7 - runFlow:
8     when:
9         platform: iOS
10    commands:
11        - tapOn:
12            id: "ios_place_row"

```

Listing 12: Simplified S2 validation evidence

```

1 {
2     "stage": "S2",
3     "validator": "mobile-dev-inc/Maestro check-syntax",
4     "artifactFileName": "SCN-F1-01-P.yaml",
5     "sha256": "...",
6     "status": "PASSED",
7     "checks": [
8         { "name": "artifact-shape", "passed": true },
9         { "name": "generation-completeness", "passed": true },

```

```

10   { "name": "maestro-check-syntax", "passed": true }
11 },
12   "issues": []
13 }

```

The runtime accepts individual flow paths or directories as inputs. It expands each directory to its YAML files and issues the S_3 command in Table 7 separately for each flow. The first generated flow on a platform uses `-reinstall-driver`. Later generated flows, retries, and manual flows use `-no-reinstall-driver`.

Listing 13: Android and iOS execution-target discovery

```

adb devices -l
adb -s emulator-5554 emu avd name
xcrun simctl list devices booted

```

Listing 14: Tag-filtered standalone generation

```

java -jar build/gradle/cli/libs/cli-0.0.0.jar generate \
  --aim datasets/aim/salzburg.aim.yaml \
  --out build/generated-maestro/scenario-corpus \
  --platforms android,ios \
  --tags SCN-F1-01-P \
  datasets/mobilebdd/scenario_corpus

```

Listing 15: Standalone CLI invocation for S_3

```

java -jar build/gradle/cli/libs/cli-0.0.0.jar s3 \
  --config config/maestro/.maestro.conf \
  --platform <android|ios> \
  --device <device-id> \
  --out <run-directory> \
  <flow.yaml|flow-directory>

```

Listing 16: Shortened S_3 report for an expected-fail flow

```

1 {
2   "stage": "S3",
3   "flowPath": ".../SCN-F1-04-N.yaml",
4   "runDirectory": ".../run",
5   "reportPath": ".../run/report.json",
6   "command": [
7     "maestro", "test", "--platform", "android",
8     "--device", "<device-id>",
9     "--no-reinstall-driver", "-e", "APP_ID=at.schuele.salzburg_card",
10    ".../SCN-F1-04-N.yaml"
11  ],
12   "exitCode": 1,

```

```

13  "status": "FAILED",
14  "expectedStatus": "FAILED",
15  "verdict": "PASSED",
16  "failureClassification": "ORACLE",
17  "attempts": [
18    {
19      "attempt": 1,
20      "status": "FAILED",
21      "failureClassification": "ORACLE",
22      "exitCode": 1,
23      "runDirectory": ".../run",
24      "stdoutPath": ".../stdout.log",
25      "stderrPath": ".../stderr.log",
26      "startedAt": "2026-07-28T10:00:00Z",
27      "finishedAt": "2026-07-28T10:00:04Z",
28      "durationMillis": 4000
29    }
30  ],
31  "flakyPass": false,
32  "scenarioTitle": "SCN-F1-04-N ...",
33  "stdoutPath": ".../stdout.log",
34  "stderrPath": ".../stderr.log",
35  "screenshotPath": ".../screenshot.png",
36  "failure": {
37    "failingAssertion": "\"Mozart Birthplace\" is visible",
38    "message": "Assertion is false: \"Mozart Birthplace\" is visible"
39  }
40 }

```

Table 20: Principal fields of a completed S_3 report.

Field group	Stored fields
Identity	stage, flowPath, runDirectory, reportPath, scenarioTitle, stepCount, and scenarioTags
Invocation	command and exitCode
Classification	status, expectedStatus, verdict, and failureClassification
Attempts	attempts with attempt number, status, failure classification, exit code, run and log paths, timestamps, and duration
Retry result	flakyPass
Timing	startedAt, finishedAt, and durationMillis
Logs and artefacts	stdoutPath, stderrPath, debugOutputPath, and screenshotPath
Diagnostics	failure.failingCommand, failure.failingAssertion, failure.message, failure.hint, and error

When a retry occurs, the top-level status, command, exit code, run directory, log paths, timing, and diagnostics describe the final attempt. The `attempts` array retains both executions, including the first failure and the `retry-1` execution.

D.3.1 Report Pairing and S_4 Case Fields

Generated and manual reports are indexed by (*scenarioId*, *platform*). The scenario identifier is read from the scenario title and then from the flow or report path as a fallback. The platform is read from the Maestro command and then from the run path. For duplicate keys, the latest *finishedAt* value is selected and the path is used as a deterministic tie-breaker.

Table 21: Implemented S_4 case classification.

Outcome	Condition
MISSING	The generated/manual report pair is incomplete.
NON_EXECUTABLE	At least one present report is not comparison-eligible. A report is eligible only when its status–classification pair is PASSED/NONE or FAILED/ORACLE.
DISAGREED	Both reports are comparison-eligible and their raw statuses differ.
AGREED	Both reports are comparison-eligible and their raw statuses are equal.

Listing 17: Shortened S_4 report for one eligible agreeing pair

```

1 {
2   "stage": "S4",
3   "outcome": "PASSED",
4   "compiledReportInputs": [".../generated/SCN-F1-01-P/report.json"],
5   "manualReportInputs": [".../manual/SCN-F1-01-P/report.json"],
6   "reportPath": ".../s4-report.json",
7   "total": 1,
8   "agreed": 1,
9   "disagreed": 0,
10  "missing": 0,
11  "nonExecutable": 0,
12  "errors": [],
13  "cases": [
14    {
15      "scenarioId": "SCN-F1-01-P",
16      "platform": "android",
17      "outcome": "AGREED",
18      "compiledReportPath": ".../generated/SCN-F1-01-P/report.json",
19      "manualReportPath": ".../manual/SCN-F1-01-P/report.json",
20      "compiledStatus": "PASSED",
21      "manualStatus": "PASSED",
22      "issue": null
23    }
24  ]
25 }
```

Each case retains the scenario identifier, platform, generated and manual report paths, both raw statuses, the comparison outcome, and an optional issue. Listing 17 shows the serialised standalone output for one eligible agreeing pair. Assertion text and normalised assertion signatures are not compared. They remain diagnostic evidence in the S_3 reports. The run-level report stores the aggregate case counters and references the stage-local evidence without duplicating the individual reports.

E Corpus Provenance and Set-Membership Audit

This appendix records the artefact history used to audit the directory-level corpus assignment and the final measurement evidence. It documents when the relevant artefacts were created, substantively refined, executed, and committed. The chronology is provenance evidence for interpreting the study design; it is not itself an empirical result for RQ1–RQ3.

The operational corpus roots are $\mathcal{D}$: `datasets/mobilebdd/scenario_corpus_validation` and $\mathcal{E}$: `datasets/mobilebdd/scenario_corpus`.

Decision-rule provenance. An earlier form of the 90% decision rule appears in commit `4bc80a8` dated 26 January 2026, before the final runs recorded on 27 July 2026.

E.1 Case-Level Set Contrasts

Table 22 records the coverage role of each evaluation-corpus case and the corresponding contrast represented in the development corpus. It makes the purposive, category-matched operational assignment auditable without implying that the two corpora are independent samples.

Table 22: Evaluation-corpus cases and their development-corpus contrasts

$\mathcal{E}$ pair	category / capability	coverage role in $\mathcal{E}$	contrast represented in $\mathcal{D}$
SCN-F1-01-P/N	navigation / activity discovery	Open an existing catalogue item and reach its detail state.	F1-03 repeats detail navigation with another item; this is mainly a data-value contrast.
SCN-F1-02-P/N	navigation / saved attractions	Reach a populated saved-attractions list after state setup.	F1-04 reaches the same area for a new account and observes the empty state.
SCN-F2-01-P/N	retrieval / activity discovery	Issue a successful exact-name query and identify its result.	F2-03 uses a shorter successful query for another item.
SCN-F2-02-P/N	retrieval / activity discovery	Exercise the no-match boundary and its empty-result observation.	F2-04 is a successful query, contrasting result and no-result states.
SCN-F3-01-P/N	exploration / activity discovery	Browse the catalogue until a later item becomes observable.	F3-03 repeats catalogue exploration with another target item.
SCN-F3-02-P/N	exploration / activity reviews	Explore a detail page far enough to observe its reviews section.	F3-04 applies the same content-region check to another activity.
SCN-F4-01-P/N	element discovery / authentication	Open account access and identify the profile UI.	F4-03 checks a data-specific account element after sign-up.
SCN-F4-02-P/N	element discovery / activity reviews	Start a review and observe both required input controls.	F4-04 checks the same control set for another activity.
SCN-F5-01-P/N	interaction response / saved attractions	Save an activity and observe the immediate saved-state feedback.	F5-04 repeats the state transition with another activity.

Continued on next page

Table 22: Evaluation-corpus cases and their development-corpus contrasts (continued)

$\mathcal{E}$ pair	category / capability	coverage role in $\mathcal{E}$	contrast represented in $\mathcal{D}$
SCN-F5-02-P/N	interaction response / activity discovery	Clear a no-match search and observe catalogue restoration.	F5-03 clears a successful exact-match search, varying the prior result state.
SCN-F6-01-P/N	command invocation / saved attractions	Remove an item and observe its absence from the saved list.	F6-03 repeats removal with another item.
SCN-F6-02-P/N	command invocation / authentication	Sign out and observe removal of account-email state.	F6-04 observes removal of the username instead.
SCN-F7-01-P/N	input composition / authentication	Compose multi-field account input and observe the resulting profile username.	F7-03 uses different credentials and observes the email field.
SCN-F7-02-P/N	input composition / activity reviews	Compose rating and comment input and observe the submitted review.	F7-04 varies activity, rating, comment, and expected review content.
SCN-F8-01-P/N	configuration / activity discovery	Apply an activity-type filter and observe exclusion of another type.	F8-03 reverses the selected and excluded activity types.
SCN-F8-02-P/N	configuration / activity discovery	Apply an area filter and observe exclusion of another area.	F8-04 reverses the selected and excluded areas.

E.2 Selected Artefact Chronology

Table 23 lists selected commits that mark the creation and substantive refinement of the two corpora, manual comparators, AIM, final pipeline, and evaluation evidence. Entries are ordered by commit timestamp; pure moves, renames, and unrelated changes are omitted. The separate corpus directories record the operational assignment of $\mathcal{D}$ and $\mathcal{E}$; they do not imply independent construction or use.

Table 23: Selected Git chronology of corpus, comparator, AIM, pipeline, and result artefacts

state	commit	commit date and time (CEST)	recorded event	type
Recorded after the campaign	91744795269a	28 July 2026 20:22:16	Committed all ten cross-platform execution bundles, including generated flows, validation evidence, runtime reports, logs, and top-level pipeline reports.	Evaluation results
Stabilised	ef4b5a3e7dfa	27 July 2026 14:20:37	Finalised the cross-platform evaluation runtime: Android keyboard dismissal, runtime-failure classification, targeted retries, consolidated driver setup, and pinned-version reporting. Both scenario corpora and manual comparators were unchanged.	AIM / pipeline
Refined	dbe5cca17400	11 July 2026 14:21:05	Extended language and action mappings in two AIM files and finalised character-by-character iOS review input in the validation comparator.	AIM / manual comparator
Created	0de84ab3a650	9 July 2026 10:07:52	Created six application/example AIM files, two reusable AIM kit files, and <code>datasets/mobilebdd/dataset_split.yaml</code> , which records the directory assignments of the two 32-variant corpora	AIM / corpus assignment
Final grammar refinement	c6ca6063d858	20 June 2026 22:14:29	Split the combined grammar into lexer and parser grammars, completing the tracked grammar-content changes made after $\mathcal{E}$ was created.	Grammar / evaluation scenarios
Separated and expanded	eee864eb1d94	20 June 2026 20:09:10	Separated the manual comparator into corpus-specific directories, refined action F7-02, and added 52 validation-comparator files: 16 action flows, four helper flows, 16 expected-pass flows, and 16 mutant flows.	Manual comparator
Refined	637d1b28862b	19 June 2026 13:09:44	Generalised each feature persona in $\mathcal{D}$ and $\mathcal{E}$ to “visitor”; scenario paths and postconditions were unchanged.	Scenario corpora
Refined	be6618111c11	18 June 2026 13:13:26	Converted user stories in both corpora from comments to grammar-visible Gherkin text and changed the review-start postconditions to check the rating and review-comment fields.	Scenario corpora
Created	d5369c57059e	16 June 2026 13:22:39	Created four feature files and a generated TeX listing for each 32-variant corpus, $\mathcal{D}$ and $\mathcal{E}$.	Scenario corpora
Created	75a65a41646b	16 June 2026 10:37:31	Created the initial manual Maestro oracle: 16 action flows, four helper flows, 16 expected-pass flows, and 16 mutant flows under <code>manual_oracle/maestro</code> .	Manual comparator

Each of the ten artefact bundles in commit `91744795269a78f90bc7e0c62d04df8c3681deb1` records one complete cross-platform pipeline run. Every bundle contains 32 generated flows and their validation reports, plus 32 generated-flow and 32 manual-comparator execution reports for each platform. Together, the bundles constitute the retained evidence for the final ten-run measurement campaign.

The final tracked evaluation configuration and implementation revision preceded the campaign. Between that revision and the evidence commit, only the repeated-run trigger changed, after the last retained run had ended; neither scenario corpus, manual comparator, AIM, nor evaluated implementation changed. The reports do not record a Git revision, source-tree hash, or application-bundle hash, so they cannot exclude transient uncommitted or reverted edits or prove byte-identical application binaries across runs. The history therefore supports stability of the tracked campaign artefacts, but not the absence of earlier influence from $\mathcal{E}$.

F Scenario Corpus Summary

The tables in this appendix summarise scenario identifiers, categories, polarities, scenario titles, and expected postconditions. They do not reproduce the complete Given–When–Then paths. Repository-relative paths to the case-matched manual comparator flows are provided in Tables 26 and 27.

F.1 Development Corpus ($\mathcal{D}$)

Table 24: Development corpus derived from source files

id	capability	category	polarity	scenario	expected result
SCN-F1-03-P	activity-discovery	screen-to-screen-navigation	pass	Opening Mozart Birthplace displays its activity details	activity details for "Mozart Birthplace" should be displayed
SCN-F1-03-N	activity-discovery	screen-to-screen-navigation	fail	Opening Mozart Birthplace displays Getreidegasse details	activity details for "Getreidegasse" should be displayed
SCN-F2-03-P	activity-discovery	query-based-retrieval	pass	Searching for Zoo displays Salzburg Zoo	activity "Salzburg Zoo" should be displayed
SCN-F2-03-N	activity-discovery	query-based-retrieval	fail	Searching for Zoo displays Mirabell Gardens	activity "Mirabell Gardens" should be displayed
SCN-F2-04-P	activity-discovery	query-based-retrieval	pass	Searching for Museum displays Mozart Birthplace	activity "Mozart Birthplace" should be displayed
SCN-F2-04-N	activity-discovery	query-based-retrieval	fail	Searching for Museum displays Hohensalzburg Fortress	activity "Hohensalzburg Fortress" should be displayed
SCN-F3-03-P	activity-discovery	content-exploration	pass	Browsing the activity catalog reveals Mirabell Gardens	activity "Mirabell Gardens" should be displayed
SCN-F3-03-N	activity-discovery	content-exploration	fail	Browsing the activity catalog hides Mirabell Gardens	activity "Mirabell Gardens" should not be displayed
SCN-F5-03-P	activity-discovery	interaction-response	pass	Clearing an exact-match search restores the activity catalog	activity "Mirabell Gardens" should be displayed
SCN-F5-03-N	activity-discovery	interaction-response	fail	Clearing an exact-match search preserves the empty state	the message "No results found. Try another query or filter." should be displayed
SCN-F8-03-P	activity-discovery	configuration	pass	Selecting Museum excludes sightseeing activities	activity "Hohensalzburg Fortress" should not be displayed
SCN-F8-03-N	activity-discovery	configuration	fail	Selecting Museum includes a sightseeing activity	activity "Hohensalzburg Fortress" should be displayed
SCN-F8-04-P	activity-discovery	configuration	pass	Selecting Sb New Town excludes activities from Sb Old Town	activity "Getreidegasse" should not be displayed
SCN-F8-04-N	activity-discovery	configuration	fail	Selecting Sb New Town includes an activity from Sb Old Town	activity "Getreidegasse" should be displayed
SCN-F3-04-P	activity-reviews	content-exploration	pass	Exploring Mozart Birthplace details reveals its reviews	the Reviews section should be displayed

Table 24: Development corpus derived from source files (continued)

id	capability	category	polarity	scenario	expected result
SCN-F3-04-N	activity-reviews	content-exploration	fail	Exploring Mozart Birthplace details hides its reviews	the Reviews section should not be displayed
SCN-F4-04-P	activity-reviews	element-discovery	pass	Starting a review for Mozart Birthplace displays review input fields	rating and review comment should be displayed
SCN-F4-04-N	activity-reviews	element-discovery	fail	Starting a review for Mozart Birthplace leaves review input fields hidden	rating and review comment should not be displayed
SCN-F7-04-P	activity-reviews	user-input-composition	pass	Submitting a 5-star review displays it on the activity	the 5-star review "Excellent visit" should be displayed for "Mozart Birthplace"
SCN-F7-04-N	activity-reviews	user-input-composition	fail	Submitting a 5-star review displays a different comment	the 5-star review "Average visit" should be displayed for "Mozart Birthplace"
SCN-F4-03-P	local-authentication	element-discovery	pass	Opening account access after sign-up displays the local account email	account email "vv@vv.vv" should be displayed
SCN-F4-03-N	local-authentication	element-discovery	fail	Opening account access after sign-up displays another local account email	account email "wrong@wrong.ww" should be displayed
SCN-F6-04-P	local-authentication	command-invocation	pass	Signing out removes the local username	the Profile for username "vv" should not be displayed
SCN-F6-04-N	local-authentication	command-invocation	fail	Signing out preserves the local username	the Profile for username "vv" should be displayed

Table 24: Development corpus derived from source files (continued)

id	capability	category	polarity	scenario	expected result
SCN-F7-03-P	local-authentication	user-input-composition	pass	Creating a local account displays the authenticated email	account email "vv@vv.vv" should be displayed
SCN-F7-03-N	local-authentication	user-input-composition	fail	Creating a local account displays another email	account email "wrong@wrong.ww" should be displayed
SCN-F1-04-P	saved-attractions	screen-to-screen-navigation	pass	Opening My List for a new local account displays the empty state	the message "No saved attractions yet" should be displayed
SCN-F1-04-N	saved-attractions	screen-to-screen-navigation	fail	Opening My List for a new local account displays Mozart Birthplace	saved activity "Mozart Birthplace" should be displayed
SCN-F5-04-P	saved-attractions	interaction-response	pass	Saving Mozart Birthplace changes its state to Saved	the activity state "Saved" should be displayed
SCN-F5-04-N	saved-attractions	interaction-response	fail	Saving Mozart Birthplace preserves the unsaved state	the activity state "Save to My List" should be displayed
SCN-F6-03-P	saved-attractions	command-invocation	pass	Removing Mozart Birthplace removes it from My List	saved activity "Mozart Birthplace" should not be displayed
SCN-F6-03-N	saved-attractions	command-invocation	fail	Removing Mozart Birthplace keeps it in My List	saved activity "Mozart Birthplace" should be displayed

F.2 Evaluation Corpus ($\mathcal{E}$)

Table 25: Evaluation corpus derived from source files

id	capability	category	polarity	scenario	expected result
SCN-F1-01-P	activity-discovery	screen-to-screen-navigation	pass	Opening Getreidegasse displays its activity details	activity details for "Getreidegasse" should be displayed
SCN-F1-01-N	activity-discovery	screen-to-screen-navigation	fail	Opening Getreidegasse displays Mozart Birthplace details	activity details for "Mozart Birthplace" should be displayed
SCN-F2-01-P	activity-discovery	query-based-retrieval	pass	Searching for Getreidegasse displays the matching activity	activity "Getreidegasse" should be displayed
SCN-F2-01-N	activity-discovery	query-based-retrieval	fail	Searching for Getreidegasse displays Mozart Birthplace	activity "Mozart Birthplace" should be displayed
SCN-F2-02-P	activity-discovery	query-based-retrieval	pass	Searching for an unknown activity displays the empty state	the message "No results found. Try another query or filter." should be displayed
SCN-F2-02-N	activity-discovery	query-based-retrieval	fail	Searching for an unknown activity displays Getreidegasse	activity "Getreidegasse" should be displayed
SCN-F3-01-P	activity-discovery	content-exploration	pass	Browsing the activity catalog reveals Salzburg Zoo	activity "Salzburg Zoo" should be displayed
SCN-F3-01-N	activity-discovery	content-exploration	fail	Browsing the activity catalog hides Salzburg Zoo	activity "Salzburg Zoo" should not be displayed
SCN-F5-02-P	activity-discovery	interaction-response	pass	Clearing an active search restores the activity catalog	activity "Hohensalzburg Fortress" should be displayed
SCN-F5-02-N	activity-discovery	interaction-response	fail	Clearing an active search preserves the empty state	the message "No results found. Try another query or filter." should be displayed
SCN-F8-01-P	activity-discovery	configuration	pass	Selecting Sightseeing excludes museum activities	activity "Mozart Birthplace" should not be displayed
SCN-F8-01-N	activity-discovery	configuration	fail	Selecting Sightseeing includes a museum activity	activity "Mozart Birthplace" should be displayed
SCN-F8-02-P	activity-discovery	configuration	pass	Selecting Sb Old Town excludes activities from Sb New Town	activity "Mirabell Gardens" should not be displayed
SCN-F8-02-N	activity-discovery	configuration	fail	Selecting Sb Old Town includes an activity from Sb New Town	activity "Mirabell Gardens" should be displayed

Table 25: Evaluation corpus derived from source files (continued)

id	capability	category	polarity	scenario	expected result
SCN-F3-02-P	activity-reviews	content-exploration	pass	Exploring Getreidegasse details reveals its reviews	the Reviews section should be displayed
SCN-F3-02-N	activity-reviews	content-exploration	fail	Exploring Getreidegasse details hides its reviews	the Reviews section should not be displayed
SCN-F4-02-P	activity-reviews	element-discovery	pass	Starting a review displays review input fields	rating should be displayed; review comment should be displayed
SCN-F4-02-N	activity-reviews	element-discovery	fail	Starting a review leaves review input fields hidden	rating should not be displayed; review comment should not be displayed
SCN-F7-02-P	activity-reviews	user-input-composition	pass	Submitting a review displays it on the activity	the 4-star review "Was nice" should be displayed for "Getreidegasse"
SCN-F7-02-N	activity-reviews	user-input-composition	fail	Submitting a review displays a different comment	the 4-star review "Could be better" should be displayed for "Getreidegasse"
SCN-F4-01-P	local-authentication	element-discovery	pass	Opening account access displays the Profile	the Profile should be displayed
SCN-F4-01-N	local-authentication	element-discovery	fail	Opening account access displays My List	My List should be displayed
SCN-F6-02-P	local-authentication	command-invocation	pass	Signing out removes the local account details	account email "aa@aa.aa" should not be displayed
SCN-F6-02-N	local-authentication	command-invocation	fail	Signing out preserves the local account details	account email "aa@aa.aa" should be displayed

Table 25: Evaluation corpus derived from source files (continued)

id	capability	category	polarity	scenario	expected result
SCN-F7-01-P	local-authentication	user-input-composition	pass	Creating a local account displays the authenticated Profile	the Profile for username "aa" should be displayed
SCN-F7-01-N	local-authentication	user-input-composition	fail	Creating a local account displays another username	the Profile for username "bb" should be displayed
SCN-F1-02-P	saved-attractions	screen-to-screen-navigation	pass	Opening My List displays saved attractions	saved activity "Getreidegasse" should be displayed
SCN-F1-02-N	saved-attractions	screen-to-screen-navigation	fail	Opening My List displays an unsaved activity	saved activity "Mozart Birthplace" should be displayed
SCN-F5-01-P	saved-attractions	interaction-response	pass	Saving Getreidegasse changes its state to Saved	the activity state "Saved" should be displayed
SCN-F5-01-N	saved-attractions	interaction-response	fail	Saving Getreidegasse preserves the unsaved state	the activity state "Save to My List" should be displayed
SCN-F6-01-P	saved-attractions	command-invocation	pass	Removing Getreidegasse removes it from My List	saved activity "Getreidegasse" should not be displayed
SCN-F6-01-N	saved-attractions	command-invocation	fail	Removing Getreidegasse keeps it in My List	saved activity "Getreidegasse" should be displayed

Note. The table reproduces the source postconditions. The SCN-F7-02-P/N assertion boundary is defined in Section 3.4.

G S4 Manual Reference-Flow Artefacts

This appendix documents the manual Maestro reference-flow artefacts described in Section 3.4. Two reference-flow corpora are maintained according to the operational assignment of $\mathcal{D}$ and $\mathcal{E}$. The development-corpus reference flows are stored under `datasets/oracle/maestro/validation`; they are used while developing the compiler pipeline and checking the S_4 comparison workflow. The fixed evaluation-corpus reference flows are stored under `datasets/oracle/maestro/evaluation`; they are used for the reported reference-flow consistency results. The directory names are retained technical identifiers.

Both reference-flow sets follow the same organisation. Expected-pass scenarios are stored under `pass/`, deliberately incorrect expected-fail variants under `mutants/`, shared interaction paths under `_actions/`, and reusable setup or navigation flows under `_helpers/`. Each pass/fail pair invokes the same shared action file and changes only the terminal assertion. This keeps the manual comparator focused on differential raw-status consistency: S_4 compares equal raw statuses only after both reports satisfy its comparison-eligibility rule: (PASSED/NONE or FAILED/ORACLE).

G.1 Development-Corpus Reference Flows ($\mathcal{D}$)

The development-corpus reference flows are stored under `datasets/oracle/maestro/validation`. Their scenario identifiers match the development corpus listed in Table 24. Table 26 maps every positive/negative case pair to both YAML entry flows and their shared YAML action flow. All paths are relative to the reference-flow directory above.

Table 26: Manual reference-flow artefacts for the development corpus $\mathcal{D}$

case	expected-pass entry	expected-fail entry	shared action
F1-03	pass/SCN-F1-03-P.yaml	mutants/SCN-F1-03-N.yaml	_actions/F1-03.yaml
F1-04	pass/SCN-F1-04-P.yaml	mutants/SCN-F1-04-N.yaml	_actions/F1-04.yaml
F2-03	pass/SCN-F2-03-P.yaml	mutants/SCN-F2-03-N.yaml	_actions/F2-03.yaml
F2-04	pass/SCN-F2-04-P.yaml	mutants/SCN-F2-04-N.yaml	_actions/F2-04.yaml
F3-03	pass/SCN-F3-03-P.yaml	mutants/SCN-F3-03-N.yaml	_actions/F3-03.yaml
F3-04	pass/SCN-F3-04-P.yaml	mutants/SCN-F3-04-N.yaml	_actions/F3-04.yaml
F4-03	pass/SCN-F4-03-P.yaml	mutants/SCN-F4-03-N.yaml	_actions/F4-03.yaml
F4-04	pass/SCN-F4-04-P.yaml	mutants/SCN-F4-04-N.yaml	_actions/F4-04.yaml
F5-03	pass/SCN-F5-03-P.yaml	mutants/SCN-F5-03-N.yaml	_actions/F5-03.yaml
F5-04	pass/SCN-F5-04-P.yaml	mutants/SCN-F5-04-N.yaml	_actions/F5-04.yaml
F6-03	pass/SCN-F6-03-P.yaml	mutants/SCN-F6-03-N.yaml	_actions/F6-03.yaml
F6-04	pass/SCN-F6-04-P.yaml	mutants/SCN-F6-04-N.yaml	_actions/F6-04.yaml
F7-03	pass/SCN-F7-03-P.yaml	mutants/SCN-F7-03-N.yaml	_actions/F7-03.yaml
F7-04	pass/SCN-F7-04-P.yaml	mutants/SCN-F7-04-N.yaml	_actions/F7-04.yaml
F8-03	pass/SCN-F8-03-P.yaml	mutants/SCN-F8-03-N.yaml	_actions/F8-03.yaml
F8-04	pass/SCN-F8-04-P.yaml	mutants/SCN-F8-04-N.yaml	_actions/F8-04.yaml

G.2 Fixed Evaluation-Corpus Reference Flows ($\mathcal{E}$)

The fixed evaluation-corpus reference flows are stored under `datasets/oracle/maestro/evaluation`. Table 27 maps every positive/negative case pair to both YAML entry flows and their shared YAML action flow. All paths are relative to the reference-flow directory above. The paired entry flows differ in their terminal assertion but invoke the same shared action file, making postcondition-only mutations and path identity auditable. They belong to the fixed artefact set defined in Section 3.2.4 and provide the comparator input for the reported S_4 differential consistency results.

Table 27: Manual reference-flow artefacts for the evaluation corpus $\mathcal{E}$

case	expected-pass entry	expected-fail entry	shared action
F1-01	pass/SCN-F1-01-P.yaml	mutants/SCN-F1-01-N.yaml	_actions/F1-01.yaml
F1-02	pass/SCN-F1-02-P.yaml	mutants/SCN-F1-02-N.yaml	_actions/F1-02.yaml
F2-01	pass/SCN-F2-01-P.yaml	mutants/SCN-F2-01-N.yaml	_actions/F2-01.yaml
F2-02	pass/SCN-F2-02-P.yaml	mutants/SCN-F2-02-N.yaml	_actions/F2-02.yaml
F3-01	pass/SCN-F3-01-P.yaml	mutants/SCN-F3-01-N.yaml	_actions/F3-01.yaml
F3-02	pass/SCN-F3-02-P.yaml	mutants/SCN-F3-02-N.yaml	_actions/F3-02.yaml
F4-01	pass/SCN-F4-01-P.yaml	mutants/SCN-F4-01-N.yaml	_actions/F4-01.yaml
F4-02	pass/SCN-F4-02-P.yaml	mutants/SCN-F4-02-N.yaml	_actions/F4-02.yaml
F5-01	pass/SCN-F5-01-P.yaml	mutants/SCN-F5-01-N.yaml	_actions/F5-01.yaml
F5-02	pass/SCN-F5-02-P.yaml	mutants/SCN-F5-02-N.yaml	_actions/F5-02.yaml
F6-01	pass/SCN-F6-01-P.yaml	mutants/SCN-F6-01-N.yaml	_actions/F6-01.yaml
F6-02	pass/SCN-F6-02-P.yaml	mutants/SCN-F6-02-N.yaml	_actions/F6-02.yaml
F7-01	pass/SCN-F7-01-P.yaml	mutants/SCN-F7-01-N.yaml	_actions/F7-01.yaml
F7-02	pass/SCN-F7-02-P.yaml	mutants/SCN-F7-02-N.yaml	_actions/F7-02.yaml
F8-01	pass/SCN-F8-01-P.yaml	mutants/SCN-F8-01-N.yaml	_actions/F8-01.yaml
F8-02	pass/SCN-F8-02-P.yaml	mutants/SCN-F8-02-N.yaml	_actions/F8-02.yaml

G.3 Representative Evaluation-Corpus Reference Flow

Listings 18, 19, 20, and 21 show a representative pass/fail pair from the fixed evaluation-corpus reference flows, the shared action it delegates to, and the reusable helper used by that action. The same structure is used for the remaining evaluation scenarios listed in Table 27.

Listing 18: Representative expected-pass S4 evaluation-corpus reference flow

```
1 appId: at.schuele.salzburg_card
2 name: SCN-F1-01-P Opening Getreidegasse displays its activity
  details
3 tags: [manual-oracle, expected-pass, flow-navigation]
4 --
5 - launchApp:
6   clearState: true
7 - runFlow: ../_actions/F1-01.yaml
8 - assertVisible: "Getreidegasse"
```

Listing 19: Representative expected-fail S4 evaluation-corpus reference flow

```
1 appId: at.schuele.salzburg_card
2 name: SCN-F1-01-N Opening Getreidegasse displays Mozart
  Birthplace details
3 tags: [manual-oracle, expected-fail, flow-navigation]
4 --
5 - launchApp:
6   clearState: true
7 - runFlow: ../_actions/F1-01.yaml
8 - assertVisible: "Mozart Birthplace"
```

Listing 20: Shared interaction path used by both representative evaluation flows

```
1 appId: at.schuele.salzburg_card
2 --
3 - runFlow: ../_helpers/open-getreidegasse.yaml
```

Listing 21: Reusable helper invoked by the representative evaluation action

```
1 appId: at.schuele.salzburg_card
2 --
3 - tapOn: ".*Search activities or category.*"
4 - inputText: "Getreidegasse"
5 - runFlow:
6     when:
7         platform: Android
8     commands:
9         - tapOn:
10             id: "poi_row_poi_7"
11 - runFlow:
12     when:
13         platform: iOS
14     commands:
15         - pressKey: ENTER
16         - hideKeyboard
17         - tapOn:
18             id: "poi_row_poi_7"
```

This work has the following word count (counted by texcount):

```

File: body.tex
Encoding: utf8
Sum count: 20319
Words in text: 19021
Words in headers: 422
Words outside text (captions, etc.): 642
Number of headers: 128
Number of floats/tables/figures: 24
Number of math inlines: 234
Number of math displayed: 0
Subcounts:
  text+headers+captions (#headers/#floats/#inlines/#displayed)
  736+1+0 (1/0/4/0) Section: Introduction
  76+2+0 (1/0/0/0) Section: Related Work
  1386+23+51 (5/0/0/0) Subsection: Mobile Application Testing
  1581+30+45 (5/1/0/0) Subsection: BDD-Inspired Behaviour Specifications for Test Generation
  1128+30+7 (5/0/0/0) Subsection: Domain-Specific Language for Compiler-Based UI Test
    Generation
  487+19+0 (4/0/0/0) Subsection: Evaluation of Generated UI Tests
  43+5+13 (1/1/0/0) Subsection: Positioning Against Closely Related Approaches
  256+1+0 (1/0/1/0) Section: Methods
  751+18+22 (7/1/8/0) Subsection: Evaluation Design
  1127+14+7 (5/1/13/0) Subsection: Scenario Corpus
  489+15+13 (5/2/22/0) Subsection: Operationalisation and Measures
  347+5+10 (1/0/6/0) Subsection: Manual Comparator Construction for RQ3
  481+15+0 (5/0/4/0) Subsection: Evaluation Procedure
  48+1+7 (1/0/0/0) Section: Implementation
  424+12+193 (6/4/2/0) Subsection: Architecture
  1884+23+82 (8/1/4/0) Subsection: Core Compiler Artefacts
  1977+35+52 (9/4/44/0) Subsection: Pipeline Stages
  161+2+6 (1/1/4/0) Subsection: Implementation Verification
  529+19+66 (6/0/2/0) Subsection: Implementation Decisions and Lessons
  173+12+38 (3/2/0/0) Subsection: Platform-Specific Runtime Stabilisation
  70+1+0 (1/0/0/0) Section: Results
  522+12+18 (4/3/26/0) Subsection: Combined Evaluation Runs
  92+4+0 (1/0/8/0) Subsection: Shared Compilation Success (RQ1)
  290+10+4 (3/1/22/0) Subsection: Expected Terminal-Status Conformity (RQ2)
  366+8+4 (3/1/21/0) Subsection: Differential Raw-Status Consistency (RQ3)
  200+11+4 (4/1/10/0) Subsection: Results Summary and Decision Labels
  34+1+0 (1/0/0/0) Section: Discussion
  513+17+0 (5/0/10/0) Subsection: Interpretation of Findings
  836+18+0 (5/0/0/0) Subsection: Relation to Prior Work
  960+14+0 (6/0/7/0) Subsection: Threats to Validity
  31+2+0 (1/0/0/0) Section: Summary \& Outlook
  297+12+0 (4/0/10/0) Subsection: Findings and Contributions
  304+16+0 (5/0/5/0) Subsection: Limitations
  422+14+0 (5/0/1/0) Subsection: Outlook

```